LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

CHAIR OF THEORETICAL COMPUTER SCIENCE AND THEOREM PROVING



Designing a Competitive Proof Verification System for TPTP-Style Benchmarks

Alexander Giller

Master's Thesis
Computer Science

Supervisor: Prof. Dr. Jasmin Blanchette

Advisor: Tanguy Bozec

Submission Date: October 22, 2026

Disclaimer

I confirm that this Master's thesis was written independently by me, only with the support of the auxiliary tools listed below. All passages taken from literature or other sources are appropriately acknowledged and referenced in the text.

Munich, October 22, 2026 Author

Note on the Use of AI Tools

Artificial intelligence (AI) tools, including large language models (LLMs) such as Claude Sonnet 5.5, were used in the preparation of this thesis to support translation and linguistic editing, including improvements to grammar, spelling, clarity, and overall readability. In addition, AI-based tools were used for code completion, data analysis, and the generation of figures and visualizations.

All AI-generated or AI-assisted outputs were reviewed and critically evaluated by the author. The author takes full responsibility for the accuracy, interpretation, and final presentation of the content of this thesis.

Abstract

Modern automated theorem provers are highly effective, but the trustworthiness of their results depends on the independent verification of the proof objects they generate. This thesis presents the design and implementation of a Python-based Proof-verifier for TPTP/TSTP derivations, developed according to the specifications of ProoVer 2026, a proof-checking competition held as part of the FLoC Olympics 2026 in Lisbon (Cailler & Guilloud 2026).

Generally a wrongly accepted proof is by far the worst outcome in proof verification, and therefore the scoring scheme of the competition also punishes wrong acceptances ten times harder than it rewards a correct acceptance (Cailler & Guilloud 2026). The ProoVer 2026 competition ruleset also specifies a three-valued verdict, consisting of *VerifiedGood* for a valid proof, *VerifiedBad* for an invalid proof, and *Unknown* if the checker did not come to a definitive conclusion. This scheme is furthermore extended by the extra verdict *Timeout*, if the verifier is not able to definitively determine the verdict within the given time limit of 30 seconds. The developed verifier of this thesis accepts a proof only if every step has been validated positively, rejects it only if it has found a concrete defect, and otherwise answers *Unknown*. This policy is implemented as a modular pipeline of parsing and normalization, structural and dependency validation, and rule-specific inference checking. Structural, propositional, quantifier, and clausal rules as well as Skolemization are checked internally and produce a diagnostic for each failing step. All remaining steps are delegated to an external prover, in this thesis Vampire, which is only allowed to see the premises of the step but not the proof as a whole.

Of the 91 problem/proof pairs of the competition committee, the verifier accepts all 44 valid proofs, rejects 46 of the 47 invalid proofs, and answers *Unknown* for the remaining one. The median run time of 1.48 s per proof is within the given limit of 30 seconds wall clock time (Cailler & Guilloud 2026). This corresponds to 136 of 138 points, where most importantly no proof is wrongly accepted. The same also holds for the 9 proofs of the originally published set of 100 that were not scored in the competition. A generated suite of 109 rule-level test proofs and 379 mutants of valid proofs, in which every mutant that is no longer valid is rejected, indicates that the checks are not only tuned to fit the benchmark. Furthermore, the verifier accepts no invalid proof even when the external prover is removed. This thesis contributes a soundness argument, a modular verification architecture, a practical implementation of the verifier, and an empirical analysis of its performance and robustness under simulated competition conditions.

Keywords: Automated Theorem Proving, Proof Verification, TPTP, TSTP, Fail-Closed Checking, Skolemization, ProoVer 2026.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Research Questions and Scope . . . 1
- 1.3 Contributions and Structure . . . 2

2 Background **2**
- 2.1 Automated Theorem Proving and Proof Certification . . . 3
- 2.2 What Is a Proof Verifier? . . . 4
- 2.3 First-Order Logic . . . 6
- 2.4 Normal Forms, Skolemization, and Equisatisfiability . . . 6
- 2.5 The TPTP World . . . 7
  - 2.5.1 The FOF Language . . . 8
  - 2.5.2 TSTP Derivations . . . 9
  - 2.5.3 The SZS Ontologies . . . 11
- 2.6 Soundness of Refutations . . . 12
- 2.7 GDV: Semantic Derivation Verification . . . 13
- 2.8 Competition Setting: ProoVer 2026 . . . 15

3 Related Work **16**
- 3.1 Approaches to Proof Checking . . . 16
- 3.2 Systems of ProoVer 2026 . . . 17
- 3.3 GDV and Nörgler . . . 17
- 3.4 GDV as the Reference Point of This Thesis . . . 19
- 3.5 Vampire as a Step Validator . . . 20
- 3.6 Proof Checking Beyond TPTP . . . 21
- 3.7 Positioning of This Thesis . . . 21

4 Requirements **22**
- 4.1 Task and Assumptions . . . 22
- 4.2 Scoring and Its Consequences . . . 23
- 4.3 Functional Requirements . . . 23
- 4.4 Non-Functional Requirements . . . 25
- 4.5 Traceability . . . 25

5 Design **26**
- 5.1 Design Principles . . . 26
- 5.2 Architecture Overview . . . 27
- 5.3 Data Model . . . 27
- 5.4 Global Checks . . . 27
- 5.5 Step Dispatch and the Checker Interface . . . 29
- 5.6 Rule Checkers . . . 30
- 5.7 Rule Checks by Example . . . 30
- 5.8 Checking Skolemization . . . 34

5.9 Delegation to an External ATP . . . 36
5.10 Verdicts and Soundness Argument . . . 36
5.11 Resource Control . . . 38
5.12 Diagnostics . . . 38
5.13 Design Decisions and Alternatives . . . 38
5.14 Summary . . . 38

6 Implementation **39**
6.1 Overview . . . 39
6.2 Internal Representation . . . 40
6.3 Parser . . . 41
6.4 Driver and Dispatch . . . 41
6.5 Global Checks . . . 42
6.6 Structural, Propositional, and Quantifier Rules . . . 43
6.7 Skolemization . . . 44
6.8 Equality and Clausal Reasoning . . . 44
6.9 Delegation to the External ATP . . . 45
6.10 Resource Control . . . 46
6.11 Test Suite . . . 47
6.12 Tooling for Reporting and Evaluation . . . 47
6.13 Selected Algorithms . . . 48
6.14 Worked Examples . . . 49
6.15 Defects Found During Testing . . . 52
6.16 Deviations from the Design . . . 53

7 Evaluation **54**
7.1 Setup . . . 54
7.2 Correctness and Score . . . 56
7.3 Rule-Level Test Suite . . . 57
7.4 Rule Coverage (RQ1) . . . 57
7.5 Architecture Metrics (RQ2) . . . 61
7.6 Run Time and Memory (RQ3) . . . 61
7.7 Robustness (RQ3) . . . 63
7.8 Ablations . . . 65
7.9 Error Analysis . . . 66
7.10 Threats to Validity . . . 67
7.11 Summary . . . 68

8 Discussion **68**
8.1 Interpretation of the Results . . . 68
8.2 Scoring Policy and Its Limits . . . 69
8.3 Trusted Computing Base and Soundness . . . 69
8.4 Design Trade-offs . . . 70
8.5 Transfer to Other Provers: A Case Study with Vampire . . . 71
8.6 Limitations . . . 74

9 Conclusion and Future Work **75**
9.1 Summary . . . 75
9.2 Answers to the Research Questions . . . 75
9.3 Contributions Revisited . . . 76
9.4 Future Work . . . 76

A Reproduction **76**
A.1 Requirements . . . 77
A.2 Checking Proofs . . . 77
A.3 Tests and Evaluation . . . 78

B Example Reports **78**

C Results per Proof **78**

Bibliography **89**

1 Introduction

1.1 Motivation

Automated theorem proving (ATP) is a key technology and finds use in a widespread field ranging from formal reasoning in mathematics to software verification and also in knowledge-based systems (Robinson & Voronkov 2001). As ATP systems grow more complex, they also become more likely to contain bugs, so that a simple answer such as "the conjecture is a theorem" is hard to trust completely. A common countermeasure is to let the prover generate a derivation, also called a *proof object*, that justifies its answer step by step, and to validate this generated *proof object* with a separate and much simpler program, the *proof checker/proof verifier*. As a consequence trust is moved from a large, fast-changing and hard to understand prover to a small and well-understood checker, an idea that goes back to the de Bruijn criterion and the LCF approach of interactive theorem proving (Section 2.1).

In the TPTP ecosystem (Sutcliffe 2024), derivations are exchanged in the TSTP format (Sutcliffe 2010, 2007). A TSTP derivation is a directed acyclic graph of formulas, where each node records the previous inference rule that produced it and the premises it depends on. The provers that use the TPTP format can vary in the rules they use, in how precisely they annotate steps, and in how they treat various transformations such as clausification and Skolemization. A proof checker, which aims to verify the proof objects of such provers, must be sound, meaning that it should never accept an invalid step, while also covering all the rules that occur in real prover outputs and staying within strict time and memory limits. The established checker for TSTP derivations is GDV (Sutcliffe 2006b,c,a,d), which verifies each inference step by calling trusted external ATP systems and interpreting their returned outputs (Sutcliffe 2008a). This makes it general, and really reliable, but also slow and highly dependent on these external ATP systems. The approach of modern checkers therefore includes treating frequent rules and transformations internally and rejecting wrong steps with model finders (Chapter 3).

ProoVer 2026, the first edition of a competition for proof checkers, held as part of the FLoC Olympics 2026 in Lisbon (Cailler & Guilloud 2026), turns this setting into a benchmark. Like the CASC competition for provers (Sutcliffe 2016), it is based on the TPTP ecosystem, but the systems under evaluation are checkers, instead of provers, and it is also only focused on first-order logic (FOF) problems. The checkers receive a FOF problem and a TSTP proof, some of which intentionally contain subtle errors, and must classify the proof as valid or invalid, or abstain, if they are not able to decide definitively. The scoring scheme is strongly asymmetric: a wrongly accepted proof costs ten times as much as a correctly accepted proof gains. A checker that cannot decide a step therefore faces a real choice between accepting, rejecting, and abstaining, and the existing checkers give no systematic account of how to make it.

1.2 Research Questions and Scope

The objective of this thesis is to design and implement a Python-based verifier for TSTP derivations that is sound and competitive. *Sound* means that each accepted inference step is justified by the formal semantics of its rule, whereas *competitive* means that the verifier achieves a high total score under the ProoVer 2026 scoring scheme (Section 4.2) within the per-proof 30 second wallclock time limit. The scope is limited to FOF problems and the inference rules that occur in the ProoVer

2026 benchmarks. The task, the scoring scheme, and the resource limits are described in Chapter 4 and the thesis focuses on three main research questions.

RQ1 Which inference rules and formula fragments must a verifier support to cover the ProoVer 2026 benchmarks? *Measure:* share of steps that are validated internally, by the external prover, or not at all.

RQ2 How can a modular architecture keep the trusted part of the verifier small while remaining adaptable? *Measure:* size and test coverage of the individual checkers, and the places in the code that must be changed to add a rule.

RQ3 How do correctness, running time, and memory consumption behave on realistic and on malformed or inconsistent derivations? *Measure:* score, verdict matrix, and run time on the benchmark set and on mutated proofs.

1.3 Contributions and Structure

The contributions of this thesis are: (i) a verdict policy for proof checking that is derived from the competition's scoring scheme and results in a three-valued, fail-closed verdict, under which a proof is accepted only if every step has been validated with true (Chapters 4 and 5) (ii) a modular proof checking pipeline for TSTP derivations with internal rule-checking for specific rules, a specific Skolemization check, and a restricted fallback to the external prover Vampire otherwise, together with the Python implementation and diagnostics that recognize failed steps and help solve the occurring issues (Chapters 5 and 6) and (iii) an evaluation on the ProoVer 2026 benchmarks and a suite of mutated proofs showing that the verifier reaches 136 of 138 points on the 91 proofs of the competition committee, with no wrong acceptance and a median run time of 1.48 s per proof, together with an analysis of the remaining limitations (Chapters 7 and 8).

The thesis is structured as follows. Chapter 2 introduces first-order logic, the TPTP/TSTP formats, and the ProoVer2026 competition in detail, while Chapter 3 discusses related work, like the other Checkers of the competition, the existing established GDV checker and the used external ATP Vampire. Chapter 4 derives the requirements for the proof verifier and Chapters 5 and 6 present the architecture and its implementation. Following this comes the Chapter 7, which shows and evaluates the experimental results. Chapter 8 discusses implications and limitations of this thesis, and Chapter 9 answers the research questions and concludes with a potential view into future work and possible room for improvement.

2 Background

This chapter provides the technical foundations of the thesis and the conditions set by the competition. First, section 2.1 explains the need and importance of proof certification. Subsequently, section 2.2 defines what a *proof verifier* is and which properties it should have. Sections 2.3 and 2.4 recall first-order logic and the normal-form transformations, that can appear in the derivations, in particular Skolemization, which played a crucial role in the ProoVer2026 competiton. The section 2.5 then introduces the concrete formats of the TPTP world: the first-order language (FOF), the format of derivations (TSTP), and the SZS ontologies used during the competition. Section 2.6 combines the

two views and shows under which conditions a checked derivation proves its conjecture. Section 2.7 describes GDV, the established derivation verifier of the TPTP ecosystem, and finally section 2.8 explains the ProoVer 2026 competition and its benchmark ruleset.

2.1 Automated Theorem Proving and Proof Certification

Automated theorem provers search for formal derivations that prove the validity or unsatisfiability of different logical statements. Their search procedures combine many techniques, for example saturation, clause selection heuristics, and indexing, and are therefore highly complex. For first-order logic the dominant systems are saturation provers based on resolution and superposition, such as Vampire (Riazanov & Voronkov 2002) and E (Schulz et al. 2019). They are developed and compared in the yearly CASC competition, where especially Vampire has been dominating, winning every edition in the FOF Segment since 2002 (Sutcliffe 2016). But trusting such a system blindly is difficult, because a single implementation error can lead to a wrong answer without any visible symptoms (Taprogge et al. 2026b,a).

The most commonly used countermeasure is proof verification, which addresses this issue by separating *proof search* from *proof checking*. A prover, like Vampire, generates a proof object, and an independent checker validates each inference step against a trusted set of rules. The most important property of a checker is *soundness*, meaning it should never accept an invalid proof as valid. If the possibility of such behavior would exist, the consequences could be severe, as it would undermine the reliability of any results derived from the prover. In general, the checker should be much simpler compared to the complexity of the prover, and it should be possible to inspect the code, recognize mistakes or add and change functionalities for the common user. Therefore, proof verification shrinks the TCB from the prover to the checker, which is typically orders of magnitude smaller (Chihani et al. 2013; Taprogge et al. 2026a).

The de Bruijn criterion and the LCF (Logic for Computable Functions) approach. The principle of keeping the trusted checker small compared to the system that produces the proofs goes back to two traditions of interactive theorem proving. De Bruijn's Automath project introduced proof objects that an independent program can check, and Barendregt and Wiedijk later named the resulting requirement the *de Bruijn criterion*: a system satisfies it if its proofs can be checked by a small and simple program, one that a mathematician or logician could inspect or even write himself (Barendregt & Wiedijk 2005). The *LCF approach*, introduced with Edinburgh LCF (Gordon et al. 1979), reaches the same goal by different means. Theorems are values of an abstract data type whose only constructors are the primitive inference rules of the logic, so that every theorem, however complex the program that produced it, has been built by a small trusted *kernel*. For example HOL Light, an interactive Theorem Prover, keeps its kernel at just a few hundred lines of code (Harrison 2009). Both approaches separate a large, untrusted part, such as proof search, tactics, and automation, from a small trusted part that alone decides what counts as a valid proof (Pollack 1998).

Checking the proofs of automated provers transfers this principle to a new setting: the prover plays the role of the untrusted search, and the checker the role of the kernel. But the analogy is not perfect, as a LCF kernel only has to know a handful of primitive rules, whereas a TSTP derivation consists of coarse steps with a wide variety of rule names whose justification is left to the checker. A checker therefore has to decide between two options, which both contradict the de-Bruijn criterion.

Either it implements many rules itself, which increases its size for every additional rule, that is checked internally and makes it larger than a kernel, or it delegates some of the rules, that have to be verified to another external system, which puts a full proof search back into the hands of the provers, whose dependency it aims to get rid of. This tension between the size of the trusted checker and the number of rules it covers runs through the whole thesis, as it presents the main dilemma of the design and implementation of the verifier.

Apart from additional trust, separating search from checking has other practical benefits, like improved diagnosability. Instead of receiving a binary success or failure result, users obtain explicit intermediate steps that can be replayed and evaluated independently. Furthermore, it also makes benchmark comparisons less dependent on solver-specific implementation details, which is crucial for benchmark-driven research.

2.2 What Is a Proof Verifier?

The literature uses several terms for tools that examine proofs: *proof checker*, *derivation verifier*, *certifier*, and *proof reconstruction* and they are all not used consistently. For example, GDV is called a *derivation verifier* (Sutcliffe 2006e), whereas the ProoVer competition and the system Nörgler speak of *proof checkers* (Cailler & Guilloud 2026; Taprogge et al. 2026b). In this thesis, *verifier* and *checker* are synonyms for the following notion.

Definition. Let a *problem* be a finite set of annotated formulas with the roles `axiom` and `conjecture`, and let a *derivation* be a finite set of annotated formulas that carry sources and inference records (Section 2.5.2). A *proof verifier* is a program that maps a pair $(\mathit{Problem}, \mathit{Derivation})$ to a *verdict*. The verdicts used in the ProoVer setting are `VerifiedGood` (the derivation is a correct proof of the conjecture), `VerifiedBad` (it is not), and the abstentions `Timeout` and `Unknown` (Cailler & Guilloud 2026; Taprogge et al. 2026b). The verifier does *not* search for a proof, it only judges one that was produced by another system. A derivation is *valid* for a problem if

1. it is well-formed: its names are unique, every parent exists, and the dependency graph is acyclic (*structure*)
2. its leaves are faithful to the problem: every imported axiom and conjecture is a copy of a formula of the problem, and no further axioms are assumed (*faithfulness*)
3. every inference step is correct with respect to its declared status, i.e. entailed by its parents for `thm` and satisfiability-preserving for `esa` (*logical correctness*, Section 2.6)
4. it is a refutation: it negates the conjecture, uses only this negation and the axioms as premises, and derives `$false`.

These four groups correspond to the taxonomy of checks used in the Nörgler paper: global and local structure, logical correctness, and faithfulness to the original problem (Taprogge et al. 2026b). If a derivation is valid, then by Equation 1 the conjecture is entailed by the axioms.

Soundness and completeness. A verifier is *sound* if it returns `VerifiedGood` only for valid derivations. In the sense of this thesis, a checker is *complete* for a class of derivations if it returns `VerifiedGood` for every valid derivation of that class. Soundness is the property that justifies trusting a verifier at all, and it is what the competition punishes most heavily (Section 2.8). A note on terminology: the GDV documentation uses the two words from the point of view of *faults*. There, "a complete verifier finds every fault, and a sound verifier finds only faults" (Sutcliffe 2026a). Both definitions follow the same logical principle, namely that a procedure is sound if its positive answers are true and complete if it gives every true positive answer. The definitions only differ in what counts as the positive answer. For GDV the positive answer is "this derivation contains a fault", for this thesis it is `VerifiedGood`. A verifier that finds every fault never accepts an invalid derivation, so GDV's *complete* corresponds to *sound* here, and a verifier that finds only faults never rejects a valid derivation, so GDV's *sound* corresponds to our *complete*. The GDV documentation considers the first property the more important one, which agrees with the priority given here. Completeness on a given benchmark set is not a correctness requirement, but it determines how many benchmarks a checker can verify. The thesis therefore reports it separately as coverage.

How steps are validated. Verifiers differ mainly in the evidence they require for an individual inference step.

Semantic verification. The step is re-proved by a trusted automated prover: from the parents, the prover must derive the conclusion. This is the approach of GDV (Sutcliffe 2006e) and of Nörgler (Taprogge et al. 2026b). It works for any rule but adds the prover to the TCB and requires proof search, which may fail or time out.

Rule replay. The verifier implements the inference rules itself and recomputes the conclusion from the parents. The TCB is the verifier, but every rule must be implemented and trusted. Replay is the standard in propositional proof checking with DRAT and LRAT (Wetzler et al. 2014).

Certification. The derivation, or the verifier's own check of it, is translated into a term that a small kernel of a proof assistant or logical framework checks, for example in LambdaPi/Dedukti (Sutcliffe et al. 2025; Delahaye et al. 2013). This yields the strongest guarantee at the price of translation effort.

Practical systems combine these strategies. The ProoVer competition explicitly asks for such a hybrid: contestants "implement a proof checker that can call an external prover for some steps, while verifying specific proof steps internally" (Cailler & Guilloud 2026).

Why verification is not decidable in general. Whether a conclusion follows from its parents is an entailment problem in first-order logic, which is only semi-decidable. A verifier that transfers such questions to a prover therefore cannot always answer within a time limit, which leads to the situation that a correct step may not be proved, and an incorrect step may not be refuted. For this reason a verifier needs the possibility to abstain from a definitive verdict via the verdicts `Timeout` and `Unknown`, and it needs a policy for the correct usage of both. The policy adopted in this thesis is *fail-closed*, which means that a step or proof that cannot be established is never reported as `VerifiedGood`. Chapter 5 derives this policy from the scoring scheme of the competition.

2.3 First-Order Logic

The derivations considered in this thesis are over classical first-order logic. We recall only the notions needed within the scope of this thesis, for a full treatment see Fitting (1996).

Syntax. A signature consists of function symbols and predicate symbols with fixed arities, where constants are function symbols of arity zero. *Terms* are built from variables and function symbols. An *atom* is a predicate symbol applied to terms (or an equation $s = t$). *Formulas* are built from atoms, $\top$ and $\bot$ (written `$true` and `$false` in TPTP), the connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$, and the quantifiers $\forall$ and $\exists$. A formula without free variables is a *sentence*. A *literal* is an atom or its negation, and a *clause* is a disjunction of literals. A quantifier $\forall x\, \varphi$ *binds* x in φ and two formulas that differ only in the names of bound variables are called *alpha-equivalent*, a notion that the verifier uses to compare formulas (Fitting 1996).

Semantics. An interpretation assigns a domain, a function to each function symbol, and a relation to each predicate symbol. A sentence is *satisfiable* if some interpretation makes it true, and a set of sentences Γ *entails* a sentence φ, written $\Gamma \models \varphi$, if every interpretation that satisfies all of Γ also satisfies φ. Entailment and unsatisfiability are connected by

$$\Gamma \models \varphi \quad \text{iff} \quad \Gamma \cup \{\neg\varphi\} \text{ is unsatisfiable.} \tag{1}$$

Most ATP systems exploit this *refutation principle*: to prove a conjecture, they negate it, add it to the axioms, and derive a contradiction. This explains why TSTP proofs of theorems typically end in $\bot$ and contain a negated conjecture (Fitting 1996).

Polarity. The *polarity* of an occurrence of a subformula is positive if it lies under an even number of negations and in the consequent positions of implications, and negative otherwise. The antecedent of an implication $A \Rightarrow B$ is a negative position, since $A \Rightarrow B$ is equivalent to $\neg A \vee B$. A quantifier occurring at a negative position behaves like its dual: $\neg\forall x\, \varphi$ is equivalent to $\exists x \neg\varphi$. Checks that depend on whether a quantifier is *effectively* existential or universal, such as the Skolemization check below, must therefore track polarity through negations and implications.

For example, in $(\exists x\, p(x)) \Rightarrow q$ the existential quantifier lies in the antecedent and therefore at a negative position, so the formula is equivalent to $\forall x\, (p(x) \Rightarrow q)$. On the other hand, in $\neg\forall x\, p(x)$ the universal quantifier is effectively existential, since the formula is equivalent to $\exists x \neg p(x)$. Below an equivalence $A \Leftrightarrow B$ each operand occurs at a positive and at a negative position at the same time, because $A \Leftrightarrow B$ is equivalent to $(A \Rightarrow B) \wedge (B \Rightarrow A)$. There a quantifier has no well-defined polarity, and the equivalence has to be eliminated before the quantifier can be treated.

2.4 Normal Forms, Skolemization, and Equisatisfiability

Many provers do not work on arbitrary formulas, but first transform them into a normal form, usually conjunctive normal form (CNF) of a quantifier-free matrix. This *clausification* consists of eliminating $\Rightarrow$ and $\Leftrightarrow$, pushing negations inward (negation normal form, NNF), renaming variables apart, moving quantifiers outward (prenex form), eliminating existential quantifiers, and distributing $\vee$ over $\wedge$. Real provers use small refinements. To avoid an exponential blow-up, subformulas

are replaced by fresh predicates or functions together with a defining equivalence (definitional or Tseitin-style clausification). These definitions again introduce new symbols and are equisatisfiable, not equivalent, to their input.

The elimination of existential quantifiers is called *Skolemization*. An existential variable y in the scope of universal variables $x_1, \ldots, x_n$ is replaced by a term $f(x_1, \ldots, x_n)$ with a fresh *Skolem function* f (a fresh constant if $n = 0$):

$$\forall x_1 \ldots \forall x_n \exists y \, \varphi(x_1, \ldots, x_n, y) \rightsquigarrow \forall x_1 \ldots \forall x_n \, \varphi(x_1, \ldots, x_n, f(x_1, \ldots, x_n)).$$

The result is in general *not* logically equivalent to the original formula, since it mentions the new symbol f. It is only *equisatisfiable*, which means that the original formula is satisfiable if and only if the skolemized formula is. For refutation-based proofs this is enough, because only unsatisfiability matters (Equation 1).

Skolemization has a consequence for checking that is central to this thesis. For existential quantifiers at positive positions, the skolemized formula leads to the original one, but not the other way round. This one direction is a necessary condition for a correct step. A Skolem term with too few arguments, for example f instead of $f(x)$, yields a *stronger* formula that still entails the original one but is no longer equisatisfiable. Therefore, a Skolemization step cannot be verified by an entailment check between parent and conclusion. The checker must instead check and validate the shape of the transformation, the dependencies of the Skolem term on the universal variables in scope, and the *freshness* of the introduced symbol, i.e. that f occurs neither in the problem nor in the formulas derived before (Nonnengart & Weidenbach 2001). This is exactly why GDV treats Skolemized formulas as a separate case (Section 2.7). Similarly, instantiating a universal formula with a fresh constant, as in Herbrand-style reasoning, introduces new symbols that must be tracked. This is why the symbol-introduction annotations in ProoVer 2026 files (Section 2.8) are highly relevant for verification.

Example. Consider the axiom $\forall x \exists y \, parent(y,x)$, stating that everyone has a parent. Its correct Skolemization is $\forall x \, parent(f(x),x)$, where f maps each x to one of its parents. The formula $\forall x \, parent(c,x)$ with a Skolem constant c instead of $f(x)$ looks similar, but it claims that a single individual is the parent of everyone. It entails the original axiom, so a check that proves the parent from the conclusion succeeds, yet the step does not preserve satisfiability: together with the axiom $\forall x \neg parent(x,x)$ the original formulas are satisfiable (two individuals that are each other's parent), whereas the Skolemized formula yields $parent(c,c)$ and thus a contradiction. A verifier that accepted the step would therefore accept a refutation of a satisfiable set of formulas. Freshness matters for the same reason. If the existential variable of $\exists y \, p(y)$ is replaced by a constant a that already occurs in an axiom $\neg p(a)$, the result $p(a)$ contradicts that axiom, although $\{\exists y \, p(y), \neg p(a)\}$ is satisfiable. Missing dependencies, wrong Skolem terms, and reused symbols are among the most frequent defects of the invalid benchmark proofs of the ProoVer2026 competition (Chapter 7).).

2.5 The TPTP World

The TPTP (Thousands of Problems for Theorem Provers) library provides a standardized collection of benchmark problems across multiple logics and domains. It has become a central infrastructure for evaluating ATP systems under comparable conditions (Sutcliffe 2017). Problems are encoded

in a common syntax and distributed with metadata that supports reproducible experiments. The language comes in several dialects: untyped first-order formulas (`fof`), clause normal form (`cnf`), typed first-order (`tff`), and higher-order (`thf`). This thesis concentrates on `fof`, which is used in the ProoVer 2026 benchmarks (Chapter 4). The infrastructure around the library consists of the TSTP solution library, which stores proofs and models produced by ATP systems for the TPTP problems (Sutcliffe 2026e), the SZS ontologies (Section 2.5.3), and online tools such as SystemOnTPTP, which runs ATP systems and tools on a problem on remote machines (Sutcliffe 2000).

2.5.1 The FOF Language

FOF stands for first-order form: formulas of untyped classical first-order logic with equality, in which quantifiers and connectives may be nested arbitrarily. This distinguishes it from the `cnf` dialect, in which only clauses with implicitly universally quantified variables are allowed. A TPTP file is a sequence of *annotated formulas*, whose general shape in the TPTP grammar is (Sutcliffe 2026c)

```
fof(Name, Role, Formula, Source, UsefulInfo).
```

`Name` is a unique identifier (a lowercase word, a quoted word, or a non-negative integer), `Role` states the intended use of the formula, and `Formula` is the logical content. `Source` and `UsefulInfo` are optional annotations. They are absent in problem files, but essential in derivations and therefore solution files. Lines beginning with `%` and blocks between `/*` and `*/` are comments.

Formula roles. The roles that occur in this thesis are the following, while the grammar lists further ones, for example `lemma` and `theorem` (Sutcliffe 2026c).

Role	Meaning
`axiom`	A formula that may be assumed.
`hypothesis`	An assumption of the problem, which is treated like an axiom but must keep its role when it is imported into a proof.
`conjecture`	The statement to be proved from the axioms.
`negated_conjecture`	The negation of the conjecture, introduced by refutation provers.
`plain`	A derived formula without a special role, used for intermediate steps.
`definition`	The definition of a new symbol.

Concrete syntax. Variables are written with an initial uppercase letter, whereas predicates, functions, and constants start with a lowercase letter or are quoted. Table 1 lists the connectives, quantifiers, and defined symbols of FOF. Three syntactic details matter for a parser. First, a quantifier is followed by a variable list in brackets, a colon, and a *unit* formula, so a quantified body that is a binary formula needs parentheses: `![X]: (p(X) => q(X))`. Second, the associative connectives `&` and `|` may be chained, but they must not be mixed without parentheses, and the non-associative connectives such as `=>` and `<=>` take exactly two operands. Third, equality and inequality are *infix* atoms (`s = t`), unlike all other predicates. In the BNF of the TPTP language this is expressed by the nonterminals `fof_unitary_formula`, `fof_binary_assoc`, and `fof_binary_nonassoc` (Sutcliffe 2026c).

Table 1: Logical symbols of the FOF language and their TPTP syntax (Sutcliffe 2026c).

Notion	Symbol	TPTP	Example
negation	$\neg$	`~`	`~p(X)`
conjunction, disjunction	$\wedge$, $\vee$	`&`, `|`	`p(a) & q`
implication, reverse implication	$\Rightarrow$, $\Leftarrow$	`=>`, `<=`	`p => q`
equivalence, exclusive or	$\Leftrightarrow$, $\oplus$	`<=>`, `<~>`	`p <=> q`
NOR, NAND		`~|`, `~&`	`p ~| q`
universal, existential quantifier	$\forall$, $\exists$	`!`, `?`	`![X]: p(X)`
equality, inequality	$=$, $\neq$	`=`, `!=`	`f(X) != c`
truth constants	$\top$, $\bot$	`$true`, `$false`	`$false`

Example problem. The smallest problem of the benchmark (`PRV000+1.p`) consists of an axiom and a conjecture that are identical:

```
fof(a1, axiom, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W)).
fof(c, conjecture, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W)).
```

2.5.2 TSTP Derivations

TSTP (Thousands of Solutions from Theorem Provers) complements TPTP by providing a library and a convention for solver outputs, including proofs. A *derivation* is written in the same language as a problem, as a list of annotated formulas, and uses the annotation fields to record where each formula comes from (Sutcliffe et al. 2006; Sutcliffe 2026d). The `Source` field takes one of three forms:

`file(FileName, FormulaName)` marks a *leaf* that is taken from a problem file, for example `file('Problems/PRV000+1.p', a1)`.

`introduced(Type, [UsefulInfo])` marks a leaf that was *introduced* by the prover instead of being part of the problem, with a type such as `definition`, `tautology`, or `assumption`. A `definition` must declare the new symbol with `new_symbols(definition, [Symbol])`.

`inference(Rule, [UsefulInfo], [Parents])` marks a *derived* formula. `Rule` is an arbitrary word, since the names of inference rules are not standardized. `Parents` lists the names of the formulas from which the formula was inferred.

The list `UsefulInfo` of an inference record holds additional terms. The two most important ones are the following (Sutcliffe 2026d).

- `status(S)` states the logical relationship between a formula and its parents (see below). An inference record should contain exactly one status.
- `new_symbols(Kind, [Symbols])` documents symbols that the step introduces, for example `new_symbols(skolem,[sK0])`. For Skolemization the additional term `skolemize(Var, Term)` names the eliminated variable and the Skolem term, for example `skolemize(X3,sK0)`.

Status values. The status of an inference is one of the success values of the SZS ontology (Section 2.5.3), from which three are used in derivations.

Status	SZS name	Meaning
`thm`	Theorem	The formula is a logical consequence of its parents. Used by most rules.
`cth`	CounterTheorem	The *negation* of the formula is a consequence of its parents. Used when negating the conjecture. The single parent must have role `conjecture`.
`esa`	EquiSatisfiable	The formula is equisatisfiable with its single parent. Used for Skolemization.

Structure of a derivation. A derivation is a directed acyclic graph (DAG): the leaves are formulas of the problem or introduced formulas, the derived nodes are inferred from their parents, and the roots are the final formulas. For a refutation, the root is `$false` (Sutcliffe 2026a). Figure 4 in Chapter 5 shows the graph of a complete benchmark proof. Further conventions apply to derivations that are to be verified. An inference should list only the parents that it actually uses. Each Skolemization step eliminates one existential variable and is recorded in its own annotated formula. Symbols introduced by a step must be globally new. They are by convention named `sK`*n* for Skolem symbols and `sF`*n* or `sP`*n* for defined functions and predicates (Sutcliffe 2026d). These conventions are what make the status of a Skolemization step checkable: without a single eliminated variable and a declared new symbol, a verifier has little to examine (Taprogge et al. 2026b). In practice, provers do not always follow them. Vampire, for example, reports the introduction of Skolem symbols as separate `introduced(definition,...)` formulas, attaches no status to most steps, and eliminates several variables in one `skolemisation` step (Section 8.5). An excerpt of Vampire's proof of `PRV000+1` shows the difference to the benchmark style (Figure 1 below):

```
fof(f3,negated_conjecture,
  (~? [X0] : ! [X1] : ? [X2] : ! [X3] : r(X0,X1,X2,X3)),
  inference(negated_conjecture,[status(cth)],[f2])).
fof(f4,plain,
  (! [X0] : ? [X1] : ! [X2] : ? [X3] : ~r(X0,X1,X2,X3)),
  inference(ennf_transformation,[],[f3])).
fof(f5,plain,
  (? [X0] : ! [X1] : ? [X2] : ! [X3] : r(X0,X1,X2,X3)
   => ! [X1] : ? [X2] : ! [X3] : r(sK0,X1,X2,X3)),
  introduced(definition,[],[skolem_symbol_introduction])).
fof(f7,plain,(! [X1] : ! [X3] : r(sK0,X1,sK1(X1),X3)),
  inference(skolemisation,
    [status(esa),new_symbols(skolem,[sK0,sK1])],[f1,f6,f5])).
```

A complete example. Figure 1 shows the smallest proof of the ProoVer 2026 benchmark set, the derivation for the problem of Section 2.5.1, where already its last step needs a semantic check.

Verifying such a proof requires:

- parsing and normalizing formula representations,

```
fof(a1,axiom, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W),
    file('Problems/PRV000+1.p',a1) ).
fof(c,conjecture, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W),
    file('Problems/PRV000+1.p',c) ).
fof(neg,negated_conjecture,
    ![A]: ?[B]: ![C]: ?[D]: ~r(A,B,C,D),
    inference(negated_conjecture,[status(cth)],[c]) ).
fof(bot,plain, $false,
    inference(consequence,[status(thm)],[neg,a1]) ).
```

Figure 1: The ProoVer 2026 proof `PRV000+1.s` (comments and SZS delimiters omitted). Axiom and conjecture are imported from the problem file, `neg` negates the conjecture, and one `consequence` step derives `$false` from `neg` and `a1`.

- resolving symbol and variable scopes correctly,
- validating rule applications (e.g., propositional, clausal, instantiation, and structural steps),
- ensuring dependency consistency and acyclicity where required.

From an implementation viewpoint, these tasks are tightly coupled. For example, a checker can only validate an `instantiate` step after resolving quantifier structure in the parent formula. Similarly, validating `skolemize` requires both formula transformation checks and side-condition checks on generated symbols. As a result, practical proof verification combines syntactic processing, semantic rule checks, and graph-level consistency checks.

2.5.3 The SZS Ontologies

Solver results are communicated through the SZS ontologies (Sutcliffe 2008b, 2026b). They are four families of status values for logical data, of which three matter here.

Success ontology. Describes what has been established about the relationship between the axioms *Ax* and the conjecture *C* of a problem. For problems with a conjecture the main values are *Theorem* (every model of *Ax* is a model of *C*), *ContradictoryAxioms* (no interpretation is a model of *Ax*), and *CounterSatisfiable* (some model of *Ax* falsifies *C*). For problems without a conjecture they are *Satisfiable* and *Unsatisfiable*. The same ontology provides the status values of inference steps: *Theorem*, *CounterTheorem*, and *EquiSatisfiable* are exactly the values `thm`, `cth`, and `esa` above.

NoSuccess ontology. Describes why no success value was established, for example *Timeout* (the CPU limit was reached), *GaveUp* (the system gave up because of incompleteness), or *Error*.

DataForm ontology. Describes the nature of the logical data that justifies a result, for example *Proof*, *CNFRefutation*, *Model*, *FiniteModel*, or *Saturation*.

A solver reports its result in comment lines of the form

```
% SZS status Theorem for PRV000+1
% SZS output start Proof for PRV000+1
...
% SZS output end Proof for PRV000+1
```

The first line states the success or no-success value exactly once, where the data (here a proof) is delimited by the `output start` and `output end` lines, and the data form follows the keyword `start`. A verifier uses these delimiters to find the derivation inside a larger solver output. The ProoVer competition reuses the SZS style for its own verdicts (for example `% SZS status VerifiedGood`), as does this thesis.

2.6 Soundness of Refutations

Section 2.2 defined a sound verifier as one that accepts only valid derivations. This section shows why the conditions of that definition are enough, that is, why a valid derivation proves its conjecture, and which criterion a checker has to apply to each kind of step.

Step status. The status field distinguishes the logical strength of a step (Section 2.5.2). A step with status `thm` claims that its conclusion is entailed by its parents, whereas a step with status `esa` (equisatisfiable) only claims preservation of satisfiability, as for Skolemization (Sutcliffe et al. 2006). In the ProoVer 2026 files the statuses `thm`, `esa`, and `cth` occur, with `cth` marking the step that negates the conjecture. A checker must apply the appropriate criterion to each step: entailment for `thm` steps, and shape and side conditions for equisatisfiability-preserving steps.

Step-wise soundness. Let Γ be the set of axioms and the negated conjecture of a derivation, and let D be a derivation of $\bot$ from Γ. The aim is to conclude from the correctness of the individual steps that Γ is unsatisfiable, and hence, by Equation 1, that the conjecture is entailed by the axioms. This conclusion follows from *step-wise soundness*, which has to be stated separately for the two kinds of steps. A step with status `thm` is sound if its conclusion is entailed by its premises. A step with status `esa` is sound if it preserves satisfiability: every model of its premises can be extended, by interpreting the fresh symbols it introduces, to a model of the premises and the conclusion. If all steps are sound and the dependency graph is acyclic, an induction over the graph yields the invariant that Γ together with all formulas derived so far is satisfiable whenever Γ is. This is impossible once $\bot$ has been derived, so Γ is unsatisfiable. The argument has a precondition that is easy to overlook: the derivation must actually *contain* a derivation of $\bot$. A derivation in which every step is sound but that never reaches $\bot$ proves nothing, so the check that a refutation ends in `$false` (EIF in Table 2) is part of soundness and not only of well-formedness. Section 5.10 carries out this argument for the verifier. In practice, systems adopt conservative behavior for unsupported constructs: unknown inference rules are rejected, flagged, or isolated to avoid unsound acceptance. This design choice trades permissiveness for trustworthiness, which is generally desirable in certification-oriented workflows (Pollack 1998).

2.7 GDV: Semantic Derivation Verification

GDV is the established tool for verifying derivations in the TPTP world. It was developed by Geoff Sutcliffe and is described in Sutcliffe (2006e), with additional material on the TPTP web site (Sutcliffe 2026a). Because the competition systems and this thesis build on its ideas, we describe it in some detail.

Motivation. ATP systems are complex software and therefore likely to contain bugs, which can make their outputs unsound. GDV addresses this by having *trusted* ATP systems check each inference of a derivation semantically, and by combining this with a *structural* verification that each inference is used correctly within the whole derivation (Sutcliffe 2006e).

Structural verification. The structural checks do not look at the logical content of formulas but at the relationships between inferences. The documentation lists nine checks in three groups (Sutcliffe 2026a): *sanity checks* (the parents of derived formulas exist, the derivation is acyclic, and refutations have a false root), checks for *explicit splitting* (both sides of refuting splits have a false root, and split refutations are independent), and checks for the *use of assumptions* (assumptions are recorded, propagated, and discharged).

Semantic verification. GDV generates a *verification obligation* for every formula of the derivation, including the leaves, and discharges the obligations with a trusted ATP system. The kind of obligation depends on the relationship between a formula and its sources (Sutcliffe 2026a):

Leaf formulas. A leaf that is a copy of an input formula needs no proof. Some ATP systems preprocess their input, so that a leaf is not a copy but derived from the input. In such cases the aim is to prove the leaf from the input problem. The documentation notes that undocumented clausification is a complication, since the resulting clauses include those of the negated conjecture and clauses containing Skolem functions.

Logical consequences. The derived formula must be proved from its parents, with theory axioms added if needed.

Relevant logical consequences. Contradictory parents entail any formula trivially. Where this must be excluded, the obligation is to show that the parents are satisfiable, which is only semi-decidable in first-order logic. The documentation names finite model generation, saturation, and failure of an unsatisfiability test as approaches. Relevance is not required in some refutation steps.

Negated logical consequences. For a negated conjecture, the obligation is to prove the negated leaf from its parent.

Inverse logical consequences. For a Skolemized formula, the obligation is reversed: the *parent* is proved from the Skolemized formula. The documentation labels this check incomplete (Sutcliffe 2026a), in line with the observation of Section 2.4 that Skolemization is not an entailment.

Table 2: Taxonomy of checks for TSTP refutations, after Taprogge et al. (2026b). The abbreviations are those of the paper.

Group	Checks
Global structure	EIF: the proof ends in `$false`; IAA: inference parents are acyclic; NUF: no unused formulas; NAU: names are unique.
Local structure	PAA: each formula has an annotation; RIA: the role is admissible; IPE: inference parents exist earlier; NSC: only the negated conjecture has status `cth`; ONU: only the negated conjecture is used as a parent; DCP: a formula is `$false` or a parent.
Logical correctness	ICN: the conjecture is correctly negated; ICE: a step is correctly entailed by its parents; ICS: a step is correctly Skolemized; APS: the problem's axioms are satisfiable; PIS: the premises of an inference are satisfiable.
Faithfulness	COO: a file-sourced formula is a copy of the problem formula; NAA: no axioms are added beyond the problem file.

The trusted systems are called through SystemOnTPTP (Sutcliffe 2000). Optionally, GDV can additionally invoke model finders for two further checks that detect trivially satisfied inferences: that the axioms of the problem are satisfiable and that the premises of an inference are satisfiable (Taprogge et al. 2026b).

Trust. The documentation discusses the circularity of the approach: a trusted system that wrongly proves an invalid step makes the verifier accept a faulty derivation, which is unsound in the sense of Section 2.2, and a trusted system that fails to prove a valid step makes it reject a correct one, which is incomplete. In the fault-oriented terminology of the documentation, the two cases are called the other way round. The proposed way out is empirical: every system acts as trusted system for the derivations of the others, so either all are wrong or all are right, and a *weak* trusted system is preferable because it only needs to discharge small obligations (Sutcliffe 2026a). GDV-LP removes the trust in the ATP by translating the proofs that discharge the obligations into LambdaPi terms, which a small kernel checks, which completes a pipeline from proof search through verification to certification (Sutcliffe et al. 2025).

Limitations observed in the literature. The authors of Nörgler, which follows the semantic approach of GDV, report the following limitations (Taprogge et al. 2026b). Semantic verification relies on proof search and is therefore undecidable in general. Skolemization steps are unverifiable without annotations. Therefore, GDV falls back to a best-effort check by entailment in both directions, which cannot be disabled and whose failure does not reject the proof. Some checks were missing in the version studied: that only the negated conjecture carries the status `cth` and that only the negated conjecture is used as a parent of other steps (both were added to GDV later). Calling external provers via a remote service adds latency. Finally, many provers do not follow the TPTP conventions for derivations, so that strict checks may reject proofs that are not actually wrong.

Check taxonomy. Taprogge et al. (2026b) organize the checks of GDV-style verifiers into four groups, which we use in later chapters to compare verifiers (Table 2).

2.8 Competition Setting: ProoVer 2026

ProoVer is a new competition for proof checkers in the TPTP ecosystem, inspired by CASC, that evaluates the correctness and robustness of checkers (Cailler & Guilloud 2026). Participants receive both correct and deliberately flawed proofs and must decide for each whether it is valid. The rules of the 2026 edition constrain what a competitive verifier may do (Cailler & Guilloud 2026):

- All proofs are refutations: the conjecture is negated and the proof ends with `$false`.
- A fixed set of *specified* rules must be verified *internally* by the checker. These are the import of axioms and conjectures (the `file` directive must match the problem file, and the formula must be alpha-equivalent to the referenced one), `negated_conjecture` (status `cth`, parent of role `conjecture`, formula the negation of the parent), and `skolemize` (status `esa`, one new Skolem symbol declared with `new_symbols(skolem, [sK])`, the eliminated variable and the Skolem term stated with `skolemize(Var, sK(...))`, and the Skolem term depending on the universal variables in scope).
- *Unspecified* rules need not be hard-coded as long as they are checked by an external ATP call, which is permitted only for steps with status `thm` or `cth`. The ATP may only be invoked on a single step together with its premises and most importantly it is forbidden to re-check the whole conjecture independently as a sanity check.
- The input consists of a problem file and a TSTP proof file whose header names the problem (`Proof: path/to/problem_file.p`); according to the rules, the verifier never needs to parse the header beyond that.

The benchmark of the 2026 edition was announced with 100 problems, half of them with a valid and half with an invalid proof, each evaluated independently with a wall-clock limit of 30 s per problem and one checker per CPU (Cailler & Guilloud 2026). Of the 100 problem/proof pairs that were initially provided, nine no longer appear in the results published by the organizers and were apparently not scored in the competition: the scores of the participating systems refer to the remaining 91 pairs (Cailler & Guilloud 2026). The benchmark used in this thesis therefore consists of these 91 pairs (Section 7.1). A correct verdict on a bad proof earns $+2$, on a good proof $+1$, giving up scores 0, rejecting a good proof costs -1, and accepting a bad proof costs -10. The total is the sum over all problems, and ties are broken by the average time on solved problems (Cailler & Guilloud 2026). The asymmetry in this scheme is the reason for the fail-closed policy of Section 2.2.

Competition scenarios impose additional engineering constraints beyond pure logical correctness. A competitive verifier must process diverse benchmark sets, cope with differences in prover output style, and provide stable performance under time and memory limits. Architecture decisions such as modular rule checkers, efficient internal data structures, and clear diagnostics are therefore not only implementation details, but they directly affect scientific usability and benchmark competitiveness.

The ProoVer 2026 benchmark files follow a regular TSTP-style profile: each proof file starts with comment metadata, and the derivation is enclosed in the SZS delimiters of Section 2.5.3. Typical files begin with `axiom` and `conjecture` items linked to the corresponding problem source,

followed by a `negated_conjecture` step, derived `plain` steps, and finally a contradiction step deriving `$false` via `inference(consequence,...)`.

Two concrete examples illustrate the spectrum. The small file `PRV000+1.s` has a compact structure: source formulas, negated conjecture, and one final consequence step to contradiction. In contrast, `PRV010+1.s` contains a long derivation with many rule families, including `excluded_middle`, `double_negation`, `instantiate`, `add_hypothesis`, `existential_gen`, `remove_double_negation`, `split_conjunct`, `reflexivity`, `rename_variable`, `weaken`, `commute`, `skolemize`, and `consequence`. This diversity motivates a modular checker architecture with explicit per-rule validation logic. Table 10 in Chapter 5 lists these rules with their intended reading, and Table 9 groups them by the principle of their check.

Another ProoVer 2026-specific aspect is metadata for generated symbols. Besides the Skolemization annotations of Section 2.5.2, some instantiation steps include Herbrand-symbol annotations like:

```
new_symbols(herbrand,[m0])
```

These annotations are not only decorative, but they describe symbol-introduction events that a robust checker should parse and validate consistently against the transformed formula, in particular with respect to the freshness condition from Section 2.4. Section 5.8 describes how the verifier checks `skolemize` steps and their `new_symbols` annotations.

For `herbrand` symbols the verifier performs no separate check, because an instantiation with an arbitrary term is entailed by its parent (Section 5.8).

Taken together, these conditions mean that a practical verifier has to balance three goals: high assurance through explicit rule-level validation, compatibility with the heterogeneous TSTP-style derivations of the benchmark, and efficient execution within the time limit. They motivate the requirements and design decisions of the following chapters.

3 Related Work

This chapter positions the thesis with respect to existing work on checking proofs of automated reasoners. Section 3.1 organizes the approaches by how a checker establishes the validity of an inference step. Section 3.2 surveys the systems of ProoVer 2026, which address the same task as this thesis. Section 3.3 discusses the two most closely related systems, GDV and Nörgler, in more detail. Section 3.6 briefly covers proof checking outside the TPTP ecosystem, and Section 3.7 states how this thesis differs from the work discussed.

3.1 Approaches to Proof Checking

Checkers for TSTP derivations can be grouped by how they decide that a single inference step is correct.

Semantic checking with external provers. The checker verifies the global and local structure of the derivation and then asks a trusted ATP to confirm that each step's conclusion follows from its premises. Incorrect steps may additionally be refuted by a model finder, which searches for a

countermodel. This approach reuses mature provers and is applicable to any inference rule, but it increases the trusted computing base by the ATP and depends on its running time.

Internal replay. The checker contains its own implementation of the rules, for example of resolution or superposition, and replays each step without external tools. This reduces external dependencies and can be fast, but every supported rule has to be implemented and trusted.

Translation to a logical framework or proof assistant. The derivation, or the output of an ATP checking its steps, is translated into a proof term or script that a small kernel of a proof assistant or logical framework verifies. This yields strong guarantees, since only the kernel must be trusted, at the cost of translation effort and run time.

Hybrids. Practical systems often combine these strategies, for example by checking Skolemization steps structurally and delegating entailment steps to an ATP. The systems in Section 3.2 illustrate the full spectrum.

3.2 Systems of ProoVer 2026

ProoVer 2026 is the first edition of the ProoVer competition and was held on 27 July 2026 in Lisbon, alongside CASC-J13 at FLoC 2026 (Cailler & Guilloud 2026). Systems receive FOF problems with TSTP proofs and must classify each proof as verified, failed, or uncertain. Correct verdicts earn points, abstaining scores zero, and wrongly accepting a bad proof is penalized most heavily (Pro 2026). Ten systems were entered, eight in the regular division and two (GDV and GDV-LP) in a demonstration division. Table 3 summarizes them according to their system descriptions in the proceedings (Pro 2026).

Three observations are relevant for this thesis. First, most entrants delegate entailment steps to external provers (E, Vampire) and use model finders such as Mace4 to reject wrong steps; only CheckProof and GAPT rely on internal reasoning engines. Second, Skolemization is a recurring special case: several systems, including ProofGuard and PyCheck, check it structurally, with a freshness check on the introduced symbol, instead of via entailment, in line with the discussion in Section 2.4. Third, two systems, ProofGuard and PyCheck, are written in Python. They are therefore the closest points of comparison for this thesis regarding the implementation language and are discussed in Section 3.7.

3.3 GDV and Nörgler

GDV. The Derivation Verifier GDV is the established tool for checking TSTP derivations (Sutcliffe 2006e) and in Section 2.7 its techniques are already described in detail. In short, it combines a *structural* verification of the derivation DAG with a *semantic* verification in which trusted ATP systems, called through SystemOnTPTP, discharge one obligation per formula (Sutcliffe 2006e, 2026a). In ProoVer 2026 it took part in the demonstration division as GDV 2.0, which is described there as a four-phase verification: structural checks, checks of the leaf formulas against the problem, rule-specific checks, and finally the verification of inference steps by calling trusted ATPs (Pro 2026). GDV is thus the reference for the semantic checking approach. Its dependence on external

Table 3: Systems entered in ProoVer 2026, as described in their system descriptions. "Ext." lists external tools the system relies on at run time.

System	Language	Core approach	Ext. tools
CheckProof 0.1	C	Internal replay; own saturation engine	none stated
GAPT 2.20	Scala	Import as LK proof; internal prover Escargot	none (internal)
GDV 2.0	C	Four-phase semantic verification	remote ATPs via SystemOnTPTP
GDV-LP 2.0	C, OCaml tools	GDV extended by LambdaPi translation	ZenonModulo, lambdapi, other Dedukti checkers
mrs-proover 0.2.0	Rust	Structural checks plus ATP ladder	mrs (internal), E, Vampire
Nörgler 1.1	Scala	Semantic verification with countermodel search	E, Vampire, Mace4
ProofCheck 1.0	C++	Structural checks plus internal clausal engine	E, Mace4, Vampire
ProofGuard 1.0	Python 3	Conservative checker with internal structural checks	E (bundled)
PyCheck 0.1	Python (reuses PyRes)	Structural and semantic checks	E
VaLeaDate 0.1	Scala, Lean	Per-step Vampire call; whole proof checked in Lean	Vampire, Lean (VampLean)

provers makes it comparatively slow and ties its trust to the ATPs it calls. The documentation of GDV discusses this circularity and recommends weak trusted systems that only discharge small obligations (Sutcliffe 2026a). GDV-LP (Sutcliffe et al. 2025) addresses the trust problem by translating the proofs of individual obligations into LambdaPi terms, so that the ATP no longer needs to be trusted.

Nörgler. Nörgler (Taprogge et al. 2026b) is a light-weight checker for TSTP refutations that explicitly builds on and extends the approach of GDV. It organizes the checks into global structure, local structure, logical correctness, and faithfulness to the problem (Table 2), uses an internal procedure for Skolemization steps, external provers (E and Vampire) for entailment, and the model finder Mace4 to reject incorrect steps. It adds two checks that the studied version of GDV lacked, namely that only the negated conjecture carries the status `cth` (NSC) and that only the negated conjecture is used as a parent of other steps (ONU). Important to note is that GDV later added both of these missing checks in later versions. In the evaluation of its authors on 1976 correct and 1952 falsified FOF proofs, both Nörgler and GDV verify 1972 of the correct proofs. Of the falsified proofs, GDV rejects 1845, whereas the configurations of Nörgler reject between 1845 and 1952, the

latter if the model finder is always used. The average time per correct proof is between 0.08 s and 0.31 s for Nörgler and 0.54 s for GDV (Taprogge et al. 2026b). These results show that a careful treatment of Skolemization and parallel checking can close much of the performance gap to internal replay, and they provide a useful baseline for the evaluation in Chapter 7. A direct comparison on the same benchmark was not possible within this thesis: the evaluation of Nörgler uses proofs from the TSTP library, which differ in rule vocabulary from the ProoVer proofs (Section 8.5).

3.4 GDV as the Reference Point of This Thesis

GDV and Nörgler solve the same problem as this thesis, and the ProoVer files follow the TPTP derivation conventions (SZS statuses, `new_symbols` and `skolemize` annotations) for which GDV is the established verifier. We therefore use GDV as the reference against which the design of the verifier is explained. The comparison has two parts: where the verifier follows GDV, and where it deliberately differs.

What is shared. All three verifiers split the task into structural and semantic checks, work on the derivation DAG, require refutations, treat the negated conjecture specially, and use an ATP only to discharge entailment questions. All three give up (`Timeout` or `Unknown`) rather than accept a step they cannot establish. The verdicts `VerifiedGood`, `VerifiedBad`, `Timeout`, and `Unknown` of Nörgler and of this thesis come from the ProoVer design (Taprogge et al. 2026b).

Where the verifier differs.

- **Internal replay before delegation.** GDV discharges the obligation of every inference with a trusted ATP. The ProoVer rules prescribe instead that a set of specified rules, among them axiom and conjecture import, `negated_conjecture`, and `skolemize`, be verified *internally*, and that an ATP may only be called on a single step with its premises and only for the status `thm` or `cth` (Cailler & Guilloud 2026). This verifier extends the internal part to the structural, propositional, and quantifier rules of the benchmark and uses the ATP as a restricted fallback.
- **Skolemization.** GDV checks a Skolemized formula by proving the *parent* from it, an inverse consequence that its documentation calls incomplete (Sutcliffe 2026a); without the `new_symbols` and `skolemize` annotations it falls back to entailment in both directions, and the failure of this fallback does not reject the proof (Taprogge et al. 2026b). This verifier checks the shape of the transformation, the dependencies of the Skolem term on the universal variables in scope, and the freshness of the symbol (Section 5.8).
- **Faithfulness.** Nörgler requires that a formula taken from the problem file is a syntactically equal copy (COO). The ProoVer rules and this verifier accept formulas that are alpha-equivalent to the referenced one (Cailler & Guilloud 2026).
- **Scope of checks.** GDV and the Nörgler paper aim at an *informative* verification, which additionally requires that no formula is unused and that axioms and premises are satisfiable. The competition scores only whether a proof is correct, so this verifier omits these checks (see below).

Table 4: Checks of GDV, Nörgler, and this verifier. “n.s.”: not stated in the sources. “added later”: missing in the studied version of GDV, added afterwards.

Check	Group	GDV	Nörgler	This verifier
EIF	global structure	yes	yes	yes
IAA	global structure	yes	yes	yes
NAU	global structure	n.s.	yes	yes
NUF	global structure	yes	omitted	no
IPE	local structure	yes	yes	yes
ONU	local structure	added later	yes	yes
DCP	local structure	yes	omitted	no
ICN	logical correctness	yes	yes	yes
ICE	logical correctness	ATP	ATP	internal rules, ATP fallback
ICS	logical correctness	ASk, inverse	ASk	internal
APS, PIS	logical correctness	optional	omitted	no
COO	faithfulness	yes (copy)	yes (equal copy)	yes (alpha-equivalent)

Check-by-check comparison. Table 4 compares the three verifiers along the check taxonomy of Table 2. The entries for GDV and Nörgler are taken from the Nörgler paper and the GDV documentation (Taprogge et al. 2026b; Sutcliffe 2026a), whereas the entries for this verifier were derived from its source code. Two entries, EIF and NAU, were missing in an earlier version of the verifier, which was detected through this comparison, later confirmed on small hand-made derivations, and then fixed. Most differences are intentional. NUF, DCP, APS, and PIS belong to the properties that make a proof *informative*. Nörgler omits them as well, and they do not affect whether a proof is correct (Taprogge et al. 2026b). Two rows concern correctness and deserve attention. First, EIF requires that the derivation ends in `$false`. By the soundness argument of Section 2.6 this is part of correctness, since sound steps alone do not make a refutation. The comparison showed that an earlier version of the verifier did not check this: a derivation that stops after the negated conjecture and never derives `$false` was accepted (benchmark proof TST036). The verifier now requires a derived step, i.e. one that is neither an axiom nor a conjecture and carries an inference, whose formula is `$false`. Second, NAU: the TPTP conventions require unique formula names (Sutcliffe 2026d), but the earlier version resolved a parent to the first definition of a name and checked later steps with the same name on their own (benchmark proof TST044). The verifier now rejects every derivation in which a name is defined more than once. Both checks are cheap, run before any prover is called, and among the 176 benchmark proofs they flag exactly the two seeded faults above; only the verdict of TST036 changes, since TST044 was already rejected for other reasons.

3.5 Vampire as a Step Validator

Vampire (Riazanov & Voronkov 2002; Kovács & Voronkov 2013) appears in this thesis in two different roles, which should be kept apart. As a *proof producer* it is the source of the derivations that a checker has to read. In Section 8.5 it is examined how far the verifier developed here carries over to such output. And on the other hand as a *step validator*, where it functions as a black box that decides whether a single inference step is logically valid, which is the role discussed here.

The idea is not new. GDV (Sutcliffe 2006e) already verifies a derivation by asking an automated prover whether the conclusion of each inference step follows from its parents and interprets the SZS status of the answer. Most ProoVer 2026 entrants that check entailment follow the same scheme (Table 3): Nörgler, mrs-proover, ProofCheck, and VaLeaDate call Vampire, usually alongside E and, to reject wrong steps, a model finder such as Mace4. VaLeaDate (Pro 2026) calls Vampire for every step and additionally checks the whole proof in Lean, and the short paper on Lean and Vampire proofs (Bodingbauer et al. 2026) addresses the related question of validating Vampire's own output in a proof assistant. Proof reconstruction in interactive provers, such as Sledgehammer (Böhme & Nipkow 2010), is the same idea in a stronger form: the answer of the prover is not trusted but replayed in a kernel.

Using a prover as a validator has two properties that determine how a verifier must be built. First, the prover is only *complete in the limit*. A positive answer (*Theorem*) is evidence for a valid step, but a timeout says nothing, so a verifier cannot treat "no answer" as "invalid". Second, every call costs time that is charged against the per-proof limit of the competition, and the cost varies by orders of magnitude between steps. Our verifier therefore uses Vampire in CASC mode only for steps that no internal checker handles and treats a timeout as "not confirmed". The consequences for each different verdict are listed separately in Section 6.16.

How good such a validator is on the steps of the competition proofs is measured in Section 7.8. Among the 155 compared steps, Vampire confirms 146 of the internal acceptances, and 0 acceptances remain unconfirmed because Vampire times out. Conversely, Vampire would accept 2 steps that the internal checkers reject, both in invalid proofs. The prover is thus a useful cross-check but not an oracle: it can neither replace the internal checkers (it is too slow on some steps) nor settle the cases in which they disagree.

3.6 Proof Checking Beyond TPTP

The idea of independently checking solver output is much older than ProoVer and is established in neighboring areas. For propositional satisfiability, proof formats such as DRAT and LRAT allow solvers to produce certificates that small verified checkers validate (Wetzler et al. 2014). For SMT, the Alethe format and checkers such as Carcara play this role (Schurr et al. 2021). For first-order proofs, the Dedukti logical framework and its translators, for example from Zenon Modulo, enable verification by a small kernel (Delahaye et al. 2013). In interactive theorem proving, proof reconstruction replays the output of external provers inside the kernel of a proof assistant, as in Sledgehammer for Isabelle/HOL (Böhme & Nipkow 2010). GAPT (Ebner et al. 2016) provides infrastructure for importing and transforming proofs of various provers. These approaches generally give stronger guarantees than the direct checking considered here, but they require a proof format that is far more detailed than TSTP, or additional translation steps. In contrast, TSTP derivations contain coarse inference steps whose justification is left to the checker, which is the setting of this thesis.

3.7 Positioning of This Thesis

Table 5 compares this thesis with the closest systems along the dimensions that matter for its goals.

The discussion above shows that checking TSTP derivations is not a new problem and that several competing designs exist. This thesis does not claim to be better than these systems in

Table 5: Comparison of the closest related systems with this thesis, according to their system descriptions (Pro 2026)(n/a: not stated in the description.)

Dimension	GDV	Nörgler	ProofGuard / PyCheck	This thesis
Implementation language	C	Scala	Python	Python
Needs external ATP for entailment	yes	yes	yes (E)	yes (Vampire)
Internal Skolem check	–	yes	yes	yes
Reports the offending step	n/a	n/a	n/a	yes
Mutation-based testing	n/a	n/a	ProofGuard	yes

general. Instead, it explores a particular trade-off: a small, modular Python checker in which every rule check is an independently testable function, internal checks are tried before any external prover is called, and the verdict policy is derived explicitly from the competition's scoring scheme (Section 4.2). Compared with GDV and Nörgler, which send most entailment steps to external provers, it validates the structural, propositional, quantifier, and Skolemization steps itself and uses an ATP only as a fallback. This reduces run time and the dependence on the ATP, but increases the amount of checker code that has to be trusted (Chapter 8). Compared with the other Python-based entrants ProofGuard and PyCheck, the differences lie in rule coverage and testing. ProofGuard is likewise written in plain Python and is deliberately conservative, but it checks only the negated conjecture and the Skolemization internally and sends all other theorem steps to E. It was tested on the published examples, on proofs generated by E, and on mutated variants. PyCheck reuses the parser and data structures of PyRes and combines structural checks with calls to an external prover. In contrast, this thesis checks 20 rules internally, which is why 99.1 % of the derived steps do not need a prover (Chapter 7), and it uses Vampire instead of E. The evaluation in Chapter 7 substantiates these differences with measurements.

4 Requirements

This chapter derives the requirements for the verifier from the ProoVer 2026 task and from the properties of TSTP derivations discussed in Chapter 2. Section 4.1 fixes the task and its assumptions. Section 4.2 analyses the scoring scheme, which determines how the verifier has to behave when it is unsure, and derives a decision policy from it. Sections 4.3 and 4.4 list the functional and non-functional requirements. Section 4.5 relates them to the modules and tests of the implementation.

4.1 Task and Assumptions

The verifier receives a FOF problem file and a TSTP proof file and must decide whether the proof is a valid refutation of the negated conjecture, i.e. a derivation of `$false` from the axioms and the negated conjecture (Cailler & Guilloud 2026). The proofs use the roles `axiom`, `conjecture`, `negated_conjecture`, and `plain` and the statuses `thm`, `esa`, and `cth`. A part of the benchmark proofs contains deliberate, subtle errors. Only a small number of rules is specified by the organizers: Skolemization, the negated conjecture, and the import of axioms and conjectures. All other steps may be checked by an external ATP, but only if they have status `thm` or `cth` and only with the step's premises. The organizers state that handling edge cases is part of the task.

Let P be a problem with axioms A and conjecture C, and let a proof π be a finite list of annotated formulas. The verifier must decide whether π is a *valid refutation* of P: (i) every `axiom` and `conjecture` step of π is faithful to P; (ii) the dependency graph of π is acyclic and C is used only by the step that negates it; (iii) every other step is sound in the sense of Section 2.6; and (iv) π derives `$false`. It answers *verified*, *failed*, or *uncertain*, whose SZS statuses in the competition are `VerifiedGood`, `VerifiedBad`, and `Unknown` if the Checker can not decide, or `Timeout` if the time limit is exceeded (Pro 2026). The competition input is only restricted to FOF. CNF and typed or higher-order input are out of scope. We make the following assumptions:

- The input files are syntactically close to the TPTP BNF, but may be large (thousands of steps) and deeply nested.
- The problem file is trusted, the proof file is not.
- An external ATP may or may not be available at run time and therefore the verifier must behave safely in both cases.
- The organizers specify only a few rules and leave edge cases open. As a consequence, the verifier follows the conservative reading, which never accepts without evidence, when the specification is silent.

4.2 Scoring and Its Consequences

In ProoVer 2026 a correct rejection of an invalid proof scores $+2$, a correct acceptance of a valid proof $+1$, giving up 0, a wrong rejection -1, and a wrong acceptance -10 (Pro 2026). The asymmetry has a direct consequence for the verifier's behavior in uncertain situations. Let p be the probability that a given proof is valid, given everything the verifier has found out. The expected scores of the three possible answers are

$$\begin{aligned} E[\text{accept}] &= p \cdot 1 + (1-p) \cdot (-10) = 11p - 10, \\ E[\text{reject}] &= p \cdot (-1) + (1-p) \cdot 2 = 2 - 3p, \\ E[\text{abstain}] &= 0. \end{aligned}$$

Accepting is thus better than abstaining only if $p > 10/11 \approx 0.91$, and rejecting is better than abstaining only if $p < 2/3$. For $2/3 < p < 10/11$, giving up is optimal. The verifier therefore must not guess. It accepts only if every step has been positively verified, it rejects only if it has found a concrete defect, and it abstains in all other cases, for example if a required tool is missing, the prover gives no answer, or no checker exists for a rule. A failed syntactic check on a rule with an ATP fallback is handled conservatively and reported as a defect unless the prover confirms the step (Section 5.9). Figure 2 visualizes the three expected scores. The probability p is of course not observable; the analysis does not compute it but motivates the asymmetric thresholds and the fail-closed policy that gives rise to the soundness requirement below.

4.3 Functional Requirements

Table 6 lists the functional requirements. Each requirement has an identifier that is used for traceability in Section 4.5 and in the evaluation.

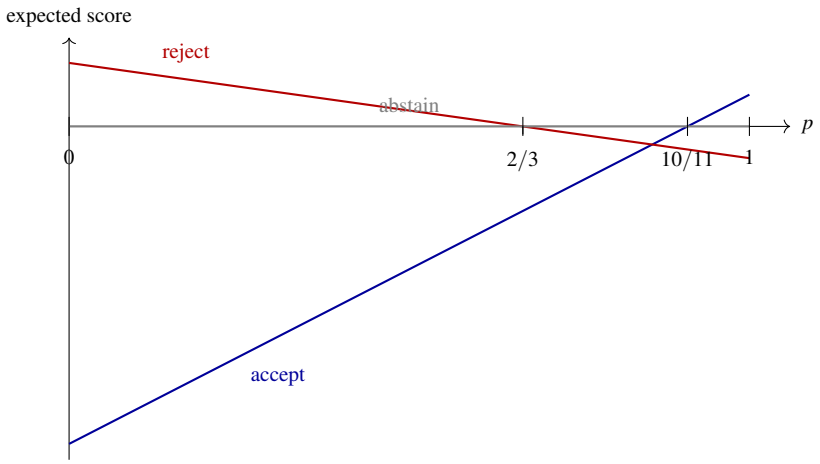


Figure 2: Expected score of accepting, rejecting, and abstaining as a function of the probability p that the proof is valid, under the ProoVer 2026 scoring scheme. Accepting beats abstaining only for $p > 10/11$, rejecting only for $p < 2/3$.

Table 6: Functional requirements.

ID	Requirement
FR1	Parse FOF problem and proof files, including roles, `file(...)` sources, `inference(...)` annotations with status, parents, `new_symbols`, and `skolemize` information.
FR2	Check that each formula imported with a `file(...)` source (`axiom`, `conjecture`, and other input roles such as `hypothesis`) is alpha-equivalent to the formula of the cited problem file and has the same role.
FR3	Check that the `negated_conjecture` step has status `cth`, has a `conjecture` as parent, and its formula is the negation of that conjecture.
FR4	Check that all cited parents exist and that the dependency graph is acyclic.
FR5	Check that the `conjecture` is not used as a premise by any step other than the negation step.
FR6	Check Skolemization steps: status `esa`, exactly one new Skolem symbol, a Skolem term that depends exactly on the universal variables in scope, a conclusion that results from replacing the eliminated variable by this term, and freshness of the symbol with respect to the problem and earlier steps.
FR7	Check each remaining inference step with a rule-specific checker whenever one exists (structural, propositional, quantifier, junction, clausal, and equational rules).
FR8	For steps without a rule-specific checker, and for the rules `weaken`, `commute`, and `excluded_middle` if their syntactic check fails, delegate the step to an external ATP, but only if its status is `thm` or `cth` and only with the step's premises.
FR9	Check that the derivation ends in `$false`.
FR10	Produce one of three verdicts per proof: verified, failed, or uncertain, and print it as SZS status (`VerifiedGood`, `VerifiedBad`, `Unknown`, or `Timeout`). A two-valued output can be requested.
FR11	Report, for each failing or uncertain step, its name, the rule, and a human-readable reason.

Table 7: Non-functional requirements.

ID	Requirement
NFR1	**Soundness (fail-closed).** A proof is verified only if every step has been positively validated. Missing tools, tool errors, unparsable output, and unknown rules never lead to acceptance.
NFR2	**Time.** A proof must be processed within the 30-second wall-clock limit, including proofs with several thousand steps. The checker enforces its own limit and then reports `Timeout`.
NFR3	**Robustness.** Malformed input, deeply nested formulas, and unexpected annotations must not crash the verifier or lead to an unjustified verdict.
NFR4	**Determinism.** The verdict does not depend on process scheduling or on the order of independent checks and the only source of variation is the run time of the external prover.
NFR5	**Diagnostics.** Every failure is traceable to a step and a reason (FR11).
NFR6	**Extensibility.** A new rule can be added without modifying the other checkers.
NFR7	**Testability.** Each checker is a pure function of the proof model and can be tested in isolation, with the target for the line coverage of the full checker modules is 85 %.
NFR8	**Portability.** Python 3.11 or newer and the parser library `lark` suffice for the core. Additionally the ATP is configured through environment variables and is optional.

ID	Requirement
FR12	Check a whole directory of proofs, compare the verdicts with the expected results, and compute the competition score.

4.4 Non-Functional Requirements

Table 7 lists the non-functional requirements. They do not describe what the verifier checks but how it has to behave while checking: safely when it is unsure, within the time limit, without crashing on unusual input, and in a way that can be tested and extended. The most important one is NFR1, the fail-closed property, which follows directly from the scoring analysis of Section 4.2; NFR2 and NFR3 follow from the competition setting with its time limit and its deliberately malformed proofs. NFR6 and NFR7 concern the architecture and are evaluated in Section 7.5.

4.5 Traceability

Table 8 shows which module implements each requirement and which test files exercise it. It is the basis for the claim that the requirements are covered and is revisited in Chapter 7.

Table 8: Traceability from requirements to modules and tests.

Requirement	Module	Tests
FR1	`parser/`	`test_parser`, `test_parser_transformer`, `test_ast_nodes`

Requirement	Module	Tests
FR2	`file_dir`, `alpha_eq`, `main.py`	`test_file_dir`, `test_alpha_equivalence`, `test_scoring_and_runtime`
FR3	`negated_conjecture_checker`	`test_negated_conjecture_checker`
FR4, FR5	`dependency_checker`	`test_dependency_checker`
FR6	`skolem_checker`	`test_skolem_checker`
FR7	`structural_`, `propositional_`, `junction_`, `clausal_`, `instantiate_`, `existential_gen_checker`	matching `test_*` files, `test_checker_review_regressions`, `test_extra_rule_cases`
FR8	`external_atp_checker`	`test_external_atp_checker`, `test_unchecked_inference_rules`
FR9	`main.py` (`check_proof_file`)	`test_refutation_and_names`
FR10, FR11	`main.py` (`classify_verdict`, `report_proof_check`)	`test_ternary_verdict`, `test_main_atp_tristate`, `test_proof_checker_summary`
FR12	`main.py` (batch mode), `scoring/`	`test_proof_checker_summary`, `test_scoring_and_runtime`
NFR2	`main.py` (worker process), `deep_recursion`	timing measurements (Chapter 7)
NFR1, NFR3	all	`test_mutation_fuzzing`, `test_advanced_edge_cases`
NFR7	all checker modules	coverage measurement (Section 7.5)

5 Design

This chapter presents the architecture of the verifier. It describes components, data model, and decision rules independently of implementation details, which follow in Chapter 6.

5.1 Design Principles

Four principles guide the design, each derived from the requirements of Chapter 4.

Fail closed (NFR1). Acceptance is the only outcome that requires positive evidence. Every error, absence of a tool, or unknown construct leads to rejection or abstaining, never to acceptance.

Rule locality (NFR6, NFR7). Each inference rule is checked by a separate function that sees only the step and its parents and returns a list of issues. Rule checkers do not know of each other.

Cheap checks first (NFR2). Syntactic checks run before any external tool is called. An ATP is only a fallback for steps that cannot be validated syntactically.

Diagnostics as data (NFR5). Checkers return structured issues instead of printing or raising, so that reports, summaries, and tests use the same information.

Relation to the de Bruijn criterion. The principles are a compromise with the ideal of a small trusted checker (Section 2.1). An LCF-style kernel would reduce every step to a few primitive rules, but the ProoVer proofs, use more than thirty rule names with no reduction to primitive rules given, and the competition prescribes that some of them are checked internally. The design therefore implements each rule as its own small checker, but lets all checkers rely on the same few operations: alpha-equivalence, substitution, template matching, and unification. These shared operations play the role of a kernel in a weak sense, because a defect in them affects every rule, while a defect in a single rule checker stays local to that rule. These rule checker modules are kept in separately and tested most intensively. The external prover is the part that departs furthest from the criterion, since it is a full proof search, but the cheap-checks-first principle keeps the number of steps that depend on it small.

5.2 Architecture Overview

Figure 3 shows the module and dependency view of the checking pipeline. A proof is processed in a separate worker process with a wall-clock limit. The parser produces a proof model, while the global checks validate provenance and the dependency graph. Later on, a dispatcher routes each step to the checker of its rule, with the ATP as fallback, if no internal rule checker module for the specific step can be found and the structure of the step allows **it**. Finally, the verdict aggregation combines all issues into one of three verdicts (theoretically four verdicts, but Timeout is returned if the dispatching did not finish, so it is not taken into account here).

5.3 Data Model

The parser turns each file into a list of annotated formulas. An *annotated formula* has a name, a role, a formula, and a justification, which is either a source (a `file` reference to the problem) or an *inference record*. An inference record consists of the rule name, a list of parent names, and a list of additional information items: the status, the `new_symbols` declaration, and, for Skolemization, the eliminated variable with its Skolem term. Formulas are represented as immutable abstract syntax trees with nodes for variables, constants, function terms, atoms, equalities, negations, binary connectives, n-ary conjunctions and disjunctions, and quantified formulas. The use of n-ary conjunctions and disjunctions makes flattening of nested connectives explicit and simplifies the junction and clausal checks. Roles, statuses, and rule names are enumerations, so that unknown values are detected at parse time or at dispatch time instead of silently ignored.

5.4 Global Checks

Three checks involve the whole proof and run before the step checks.

Provenance (FR2). Each formula imported from the problem, that is, each `axiom`, `conjecture`, or other input formula with a `file(...)` source, cites a problem file and a formula name. The checker looks up the formula in the problem file and compares it with the step's formula up to alpha-equivalence, i.e. up to consistent renaming of bound variables, and compares the roles. Alpha-equivalence is implemented as a simultaneous traversal of both trees with two mappings

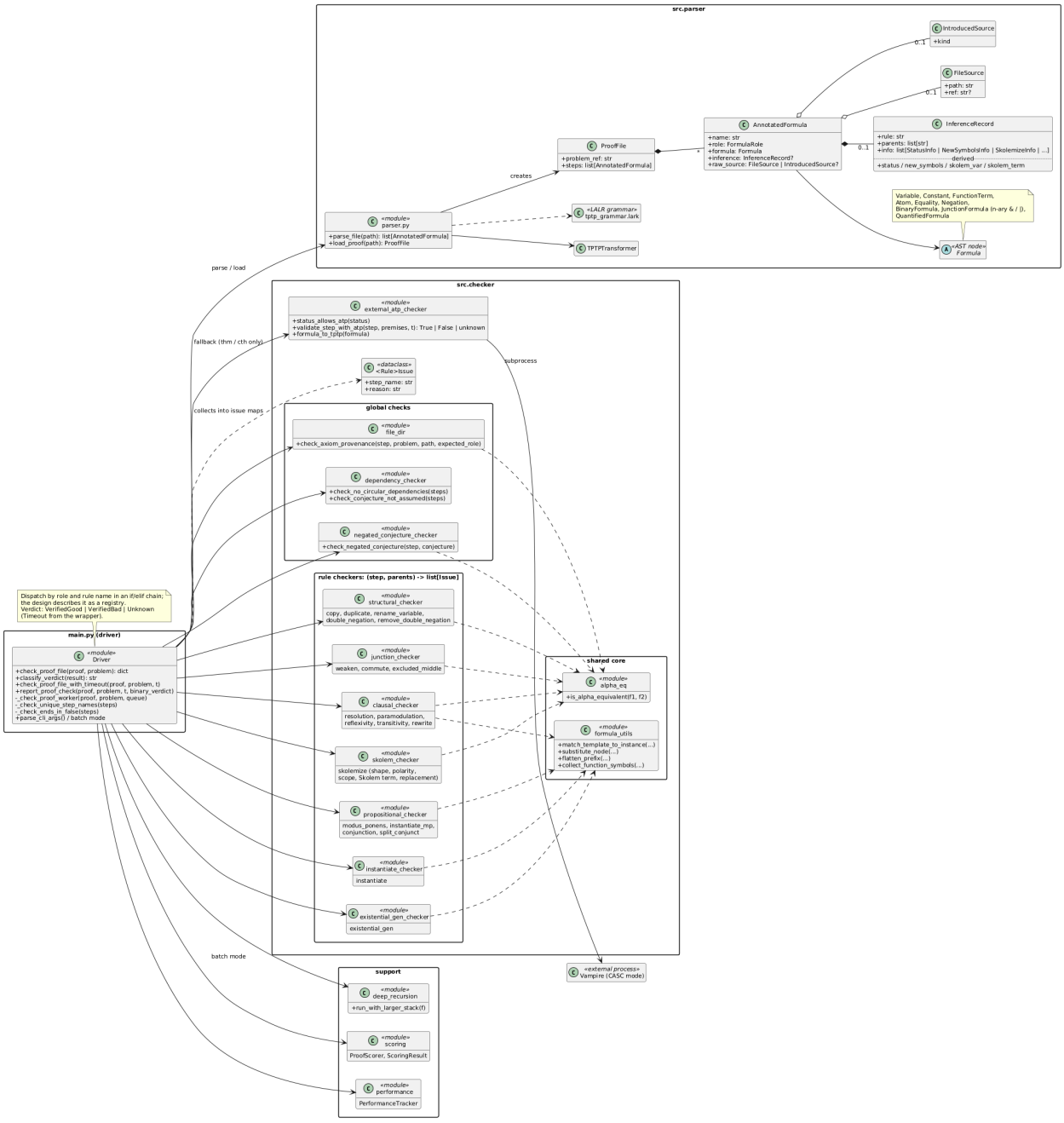


Figure 3: Architecture view of the verifier modules and their main dependencies. Parsing, checking, and verdict aggregation run in a worker process with a wall-clock limit.

between the bound variables of the two formulas, so that no renaming of the trees is necessary. Before the traversal, consecutive quantifiers of the same kind are merged into one prefix and nested conjunctions and disjunctions are flattened.

Dependency graph (FR4, FR5). The graph with an edge from each step to its parents must be acyclic. Cycles are detected with a depth-first search with three colors, where pendent references are reported by the step checks. In a second pass, no step except the negation step may cite a `conjecture`, because otherwise the proof would assume the statement it has to refute.

Termination (FR9). The derivation must contain a step with formula `$false` that depends, directly or indirectly, on the negated conjecture. The implementation only checks that a derived step `$false` exists. It does not require that the step depends on the negated conjecture, so that a problem with contradictory axioms (test case TST071) is not rejected.

5.5 Step Dispatch and the Checker Interface

All step checkers have the same shape: they take the step and its parents and return a list of issues. Any issue occurring during the proof checking process contains the step name and a reason. A checker never throws an exception for invalid proofs, however exceptions indicate a defect of the verifier or an input it cannot process and lead to the verdict *failed* (Section 5.10). The dispatcher selects the checker by the rule name of the step and performs the checks common to all rules before calling it: the number of parents must match the rule's arity, every parent must exist, and the status must be the one the rule requires. Rule checkers therefore need to concentrate on the logical content.

Dispatch is designed as a registry that maps rule names to checkers together with their arity and required status. Adding a rule means registering one entry and writing one function (NFR6). In the implementation, the registry is realized as an `if`/`elif` chain with the same effect (Section 6.16).

A proof as a graph. Figure 4 shows how a complete proof is processed, using the valid benchmark proof `PRV089`. The proof refutes the conjecture $\exists Z\,(p(Z) \wedge q(Z))$ from the axioms $\exists Y\,p(Y)$ and $\forall X\,q(X)$. Each node is an annotated formula, and each edge leads from a parent to the step derived from it. The graph is acyclic, as the dependency check requires, the conjecture `c` is used only by the negation step `negc`, and the single sink is the step `bot` with the formula `$false`, as the termination check requires. The colors show the result of the dispatch: the three formulas taken from the problem are checked for provenance, `negc` by the negated-conjecture check, `skA` by the Skolemization check, and the three intermediate steps `instB`, `s`, and `s2` by internal rule checkers.

The proof also shows how the steps depend on each other beyond their own rule. `skA` replaces the existential variable of `a1` by the Skolem constant *sK0*, which must be fresh. `instB` then instantiates the universal axiom `a2` with exactly this constant, `s` combines the two facts, and `s2` generalizes *sK0* back to an existential variable, which yields the conjecture itself. The two branches of the graph meet only in `bot`, whose parents are the conjecture and its negation. Since they are complementary, the final step is decided by the sound shortcut for `consequence` steps (Section 6.9), and the whole proof is verified without a single call to the external prover. About half of the valid proofs of the benchmark are verified in this way (Section 7.7). The order of the steps in the file

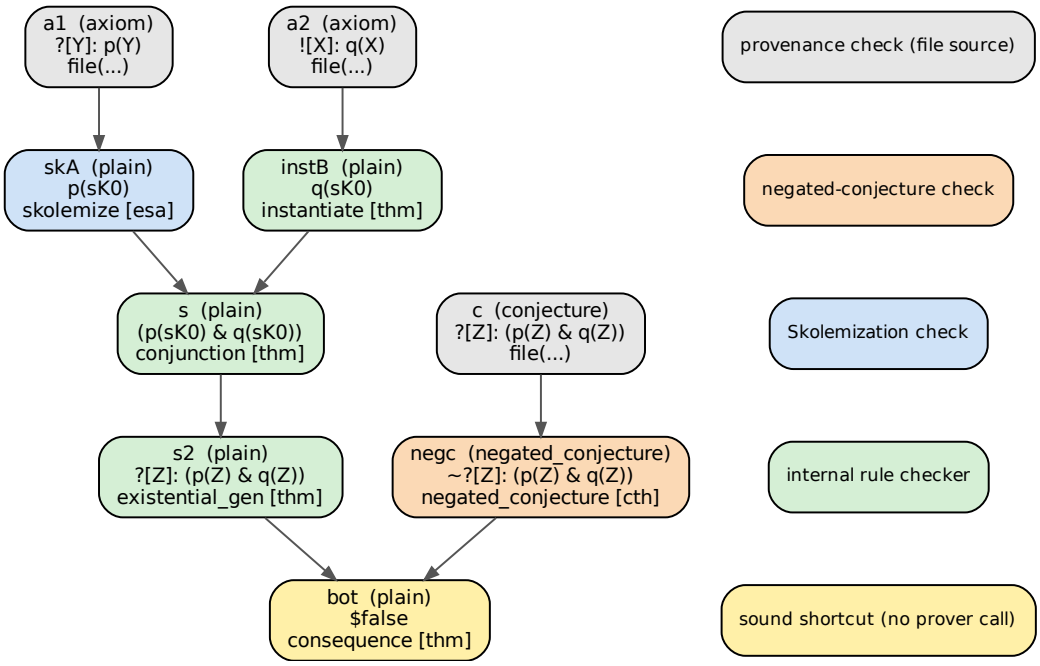


Figure 4: Dependency graph of the valid benchmark proof `PRV089`. Each node is an annotated formula with its name, role, formula, and justification and each edge leads from a parent to the step derived from it. The color shows which check of the verifier validates the node. Generated from the proof file with `scripts/gen_dag_figure.py`.

plays no role for the checks: each step is judged only by its own parents, so that the graph, not the file order, determines what has to hold.

5.6 Rule Checkers

Table 9 groups the supported rules into families. Most families reduce to alpha-equivalence and structural matching, which keeps them simple and fast.

5.7 Rule Checks by Example

The principles of Table 9 are easiest to see on concrete steps. This section takes one valid step of each family from the benchmark proofs and, where the benchmark contains one, an invalid step of the same family, and states what the checker establishes. Premises are written above the line and the conclusion below it, while the names in parentheses are the proof and step names in the benchmark files.

Table 9: Rule families and the principle of their checks.

Family	Rules	Principle
Structural	`copy`, `duplicate`, `rename_variable`, `double_negation`, `remove_double_negation`	Conclusion is alpha-equivalent to the (possibly negation-wrapped or unwrapped) parent.
Propositional	`modus_ponens`, `conjunction`, `split_conjunct`	Conclusion matches a conjunct of the parents or the consequent of an implication, also after universal instantiation.
Quantifier	`instantiate`, `existential_gen`	Template matching with abstract variables: conclusion is a substitution instance (resp. generalization) of the parent.
Junction	`weaken`, `commute`, `excluded_middle`	Shape check after flattening of conjunctions and disjunctions, sometimes ATP fallback for unusual variants.
Clausal / equational	`resolution`, `paramodulation`, `reflexivity`, `transitivity`, `rewrite`	Unification of literals or terms, construction of the expected resolvent or rewritten clause, comparison up to alpha-equivalence and quantifier permutation.
Negated conjecture	`negated_conjecture`	Conclusion is the negation of the parent conjecture, up to renaming of bound variables.
Skolemization	`skolemize`	See Section 5.8.
ATP-delegated	`consequence`, `add_hypothesis`, and every other rule with status `thm` or `cth`	Entailment from the step's premises by an external ATP (Section 5.9).

Structural rules. In `PRV010`, the step `s6` wraps its parent in a double negation, and `s25` renames a bound variable:

$$\frac{q(b) \vee \neg q(b)}{\neg\neg\,(q(b) \vee \neg q(b))}\ \texttt{double_negation} \qquad \frac{\exists X_9\,(q(X_9) \vee \neg q(X_9))}{\exists X_{15}\,(q(X_{15}) \vee \neg q(X_{15}))}\ \texttt{rename_variable}$$

For both rules the checker removes or compares the outer structure and decides the rest by alpha-equivalence without the involvement of unification or any prover. A conclusion that differs from its parent in anything but the names of bound variables, for example in a constant or in the order of two quantifiers, is rejected.

Propositional rules. `PRV027` applies modus ponens to ground formulas, and `PRV017` combines it with a universal instantiation:

$$\frac{p(a) \Rightarrow q(a) \qquad p(a)}{q(a)}\ \texttt{modus_ponens}$$

$$\frac{\forall X\,(\mathit{greek}(X) \Rightarrow \mathit{human}(X)) \qquad \mathit{greek}(\mathit{socrates})}{\mathit{human}(\mathit{socrates})}\ \texttt{instantiate_mp}$$

Table 10: Inference rules occurring in `PRV010+1.s` and their intended reading. The readings are derived from the benchmark proofs, since the competition does not specify the rules in full.

Rule	Intended reading
`excluded_middle`	introduces $\varphi \vee \neg\varphi$
`double_negation`	introduces $\neg\neg\varphi$
`remove_double_negation`	removes $\neg\neg\varphi$
`instantiate`	replaces a universal variable by a term
`existential_gen`	generalizes a term to an existential quantifier
`split_conjunct`	derives one conjunct from a conjunction
`reflexivity`	introduces $t = t$
`rename_variable`	renames a bound variable
`commute`	swaps the operands of a commutative connective
`weaken`	adds disjuncts, $\varphi \vdash \varphi \vee \psi$
`add_hypothesis`	derives $H \Rightarrow \varphi$ from φ, while other shapes go to the ATP
`skolemize`	replaces an existential variable by a fresh Skolem term
`consequence`	the conclusion follows from the listed parents. $\bot$ from complementary parents is checked directly, everything else by the ATP

The checker identifies the implication among the parents, treats its universally quantified variables as abstract variables, and matches the antecedent against the other parent and the consequent against the conclusion with one common substitution, here $X \mapsto \mathit{socrates}$. Either order of the parents is accepted. In the invalid proof `PRV023`, the step `s` cites itself as a parent: the dependency check reports the cycle, and the modus ponens check finds no implication whose antecedent matches the other parent, so the step is rejected for two independent reasons.

Quantifier rules. `PRV010` instantiates a universal quantifier in `s7` and introduces an existential one in `s10`:

$$\frac{\forall X_4\, \exists X_5\, \neg q(c)}{\exists X_5\, \neg q(c)}\ \texttt{instantiate} \qquad \frac{q(b) \vee \neg q(b)}{\exists X_9\, (q(X_9) \vee \neg q(X_9))}\ \texttt{existential_gen}$$

For `instantiate` the parent is the template: its universally bound variables may be replaced by terms, and the checker searches for a substitution that turns the parent into the conclusion. Here X_4 does not occur in the body, so the step only drops the quantifier. For `existential_gen` the roles are reversed: the conclusion is the template, and the checker searches for the term of the parent, here b, that the new existential variable abstracts. The invalid proof `PRV056` contains a typical seeded error:

$$\frac{\forall X\, \forall Z\ \mathit{rel16}(X, Z, \mathit{sK0}(X, Z))}{\mathit{rel16}(m_0, n_0, \mathit{sK0}(n_0, m_0))}\ \texttt{instantiate}$$

Matching binds $X \mapsto m_0$ and $Z \mapsto n_0$ at the first two arguments, so the third argument would have to be $\mathit{sK0}(m_0, n_0)$. The stated $\mathit{sK0}(n_0, m_0)$ swaps the arguments, no consistent substitution exists, and the step is rejected. The final `consequence` step of this proof would pass the prover, since the wrong instance still contradicts the negated conjecture. In this case only the rule check of the

intermediate step reveals the error.

Junction rules. `PRV010` also contains the three junction rules:

$$\frac{q(b) \vee \neg q(b)}{\neg q(b) \vee q(b)} \quad \texttt{commute} \qquad \frac{\forall X_0\, \forall X_2\, \exists X_1\, t}{(\forall X_0\, \forall X_2\, \exists X_1\, t) \vee a = b} \quad \texttt{weaken}$$

For `commute` (`s38`) the checker verifies that the two operands are swapped, and for `weaken` (`s34`) that the disjuncts of the parent form a prefix of the disjuncts of the conclusion. The step `s5` introduces $q(b) \vee \neg q(b)$ with the rule `excluded_middle` and cites the axiom `s2` only to link the step into the proof. The checker then verifies the shape $\varphi \vee \neg\varphi$ and ignores the content of the parent, because the formula is a tautology. These three rules are the only ones for which a failed shape check is not final: the step is then passed to the prover (Section 5.9), because the rule names are also used for unusual but valid variants.

Clausal and equational rules. `PRV020` contains a resolution step with quantified clauses:

$$\frac{\forall X\,(p(X) \vee q(X)) \qquad \forall X\,(\neg p(X) \vee s(X))}{\forall X\,(q(X) \vee s(X))} \quad \texttt{resolution}$$

The checker renames the variable of the second clause apart, unifies $p(X)$ with the complementary literal $p(X')$, applies the unifier $X' \mapsto X$ to the remaining literals, rebuilds the universally quantified clause $\forall X\,(q(X) \vee s(X))$, and compares it with the conclusion. The invalid proof `PRV067` states the resolvent $r(a) \vee t(b)$ of the analogous clauses $\forall X\,(p(X) \vee r(X))$ and $\forall X\,(\neg p(X) \vee t(X))$. The actual resolvent is $\forall X\,(r(X) \vee t(X))$, which instances have the same argument in both literals, and $r(a) \vee t(b)$ is neither among them nor a consequence of the parents. The equational rules work on terms. `PRV018` chains two equations and then rewrites with the result:

$$\frac{a = b \qquad b = c}{a = c} \quad \texttt{transitivity} \qquad \frac{p(c) \qquad a = c}{p(a)} \quad \texttt{paramodulation}$$

For `paramodulation` the checker tries both directions of the equation and every occurrence of the rewritten side. It replaces c by a in $p(c)$, which leads to the conclusion. In the invalid proof `PRV007`, the equation $\forall Y\, m(Y) = f(Y)$ is applied to $p(sk0(m(a)))$, which yields $p(sk0(f(a)))$. The stated conclusion $\forall Z\, p(sk0(f(Z)))$ generalizes the constant a to a variable, which no rewriting step can do.

Skolemization. `PRV004` Skolemizes an axiom with four quantifiers, annotated with `skolemize(Y,sK0(X,Z))` and `new_symbols(skolem,[sK0])`:

$$\frac{\forall X\, \forall Z\, \exists Y\, \forall W\; r(X,Z,Y,W)}{\forall A\, \forall B\, \forall C\; r(A,B,sK0(A,B),C)} \quad \texttt{skolemize}$$

The existential variable Y lies in the scope of X and Z but not of W, so the Skolem term must take exactly X and Z as arguments. The checker computes this set, compares it with the annotation, replaces Y by $sK0(X,Z)$, and compares the result with the conclusion up to the renaming of

X, Z, W to A, B, C. Finally, *sK0* must neither occur in the problem nor have been introduced before. Section 5.8 lists the conditions in general, and Section 6.14 follows an invalid Skolemization through the implementation.

Steps decided by the prover. The only derived step of `PRV000` (Figure 1) is a `consequence` step:

$$\frac{\begin{array}{c}\forall A\,\exists B\,\forall C\,\exists D\,\neg r(A,B,C,D)\\ \exists X\,\forall Y\,\exists Z\,\forall W\; r(X,Y,Z,W)\end{array}}{\bot}\quad \texttt{consequence}$$

The premises are not complementary literals but quantified formulas whose contradiction requires instantiating both with the right witnesses, which no internal checker attempts. The step therefore goes to Vampire, which answers `ContradictoryAxioms`. The invalid proof `PRV002` shows the other outcome: its negated conjecture is not the negation of the conjecture, which the negated-conjecture check reports, and the final `consequence` step is answered with `CounterSatisfiable`, so the prover rejects it as well.

Summary. The examples show the division of labor. The internal checks are syntactic, need no search, and point to the exact step and reason. All invalid steps above are rejected by them, and in `PRV007`, `PRV056`, and `PRV067` the defect is visible only to the rule check, because the final step of the proof is accepted by the prover. The prover is needed for the generic `consequence` steps, where the benchmark gives no syntactic rule to check against.

5.8 Checking Skolemization

Skolemization cannot be validated by entailment (Section 2.4). The checker instead validates the following conditions, which together imply that the step is an instance of Skolemization:

1. The rule is `skolemize`, the step has exactly one parent, and its status is `esa`.
2. The step declares exactly one new symbol of kind `skolem`.
3. The Skolem term is a constant or a function applied to variables only, and these variables are exactly the universally quantified variables in whose scope the eliminated existential quantifier occurs.
4. Replacing the existential quantifier and its variable by the Skolem term in the parent formula yields the conclusion, up to alpha-equivalence.
5. The Skolem symbol is fresh: it neither occurs in the problem file nor was introduced by an earlier Skolemization step.

Conditions 3 and 5 are the side conditions that make the transformation equisatisfiability-preserving. Without the freshness condition, two steps could use the same symbol for different witnesses, which is unsound.

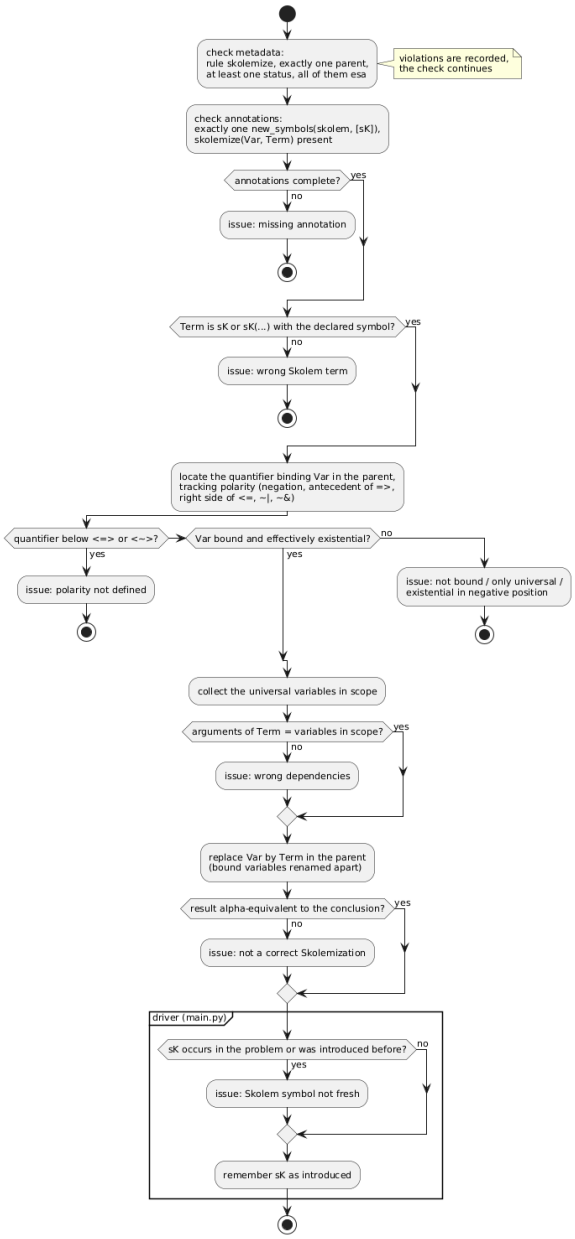


Figure 5: Validation flow for a `skolemize` step.

Polarity. Whether an existential quantifier may be eliminated, and which universal variables are in its scope, depends on its polarity. An existential quantifier in the antecedent of an implication or under a negation is effectively universal, so replacing it by a Skolem term would be unsound. The checker therefore tracks the polarity while it traverses the parent formula and rejects Skolemizations of quantifiers in negative position. Below an equivalence or an exclusive disjunction a quantifier occurs in both polarities, with such a Skolemization being rejected as well.

Herbrand symbols. Some `instantiate` steps carry a `new_symbols` annotation of kind `herbrand`. Instantiating a universal formula with an arbitrary term is entailed by that formula, so such a step is sound without a freshness condition and is validated like any other `thm` step. The competition does not state further conditions for Herbrand symbols, so the verifier does not treat them specially.

5.9 Delegation to an External ATP

Steps that have no rule-specific checker are validated by an external ATP, in accordance with the competition rules. The same fallback applies to the junction rules if the syntactic check fails, to avoid false rejections of valid but unusual variants. The delegation obeys the following policy:

- **Eligibility.** Only steps with status `thm` or `cth` are delegated. Other statuses, in particular `esa`, are never delegated.
- **Isolation.** The ATP receives exactly the premises of the step as axioms and the step's formula as conjecture. It never sees the original conjecture or the rest of the proof.
- **Interpretation.** The answer is read from the SZS status line of the output. The statuses `Theorem`, `Unsatisfiable`, and `ContradictoryAxioms` confirm the step. The statuses `CounterSatisfiable` and `Satisfiable` reject it; any other status leaves it unresolved.
- **Failure.** If the ATP is not found, times out, fails, or returns no SZS status, the step is *unresolved*. An unresolved step makes the verdict *uncertain*, never *verified*.
- **Failed syntactic check.** For rules with an ATP fallback (`weaken`, `commute`, and `excluded_middle`), a failing syntactic check does not make the step invalid at once, since the check covers only the standard shapes of the rule, the ATP decides. If it does not confirm the step, the original issue is reported, which is the conservative choice. For all other rules, a failing check makes the step invalid.
- **Budget.** Each ATP call is limited by the per-proof time limit, with the worker process additionally enforcing the limit for the proof as a whole.

5.10 Verdicts and Soundness Argument

Each step ends in one of three states: *valid* (positively validated), *invalid* (a concrete defect was found), or *unresolved* (a required check could not be completed). The verdict of the proof is determined by the following rule, in this order:

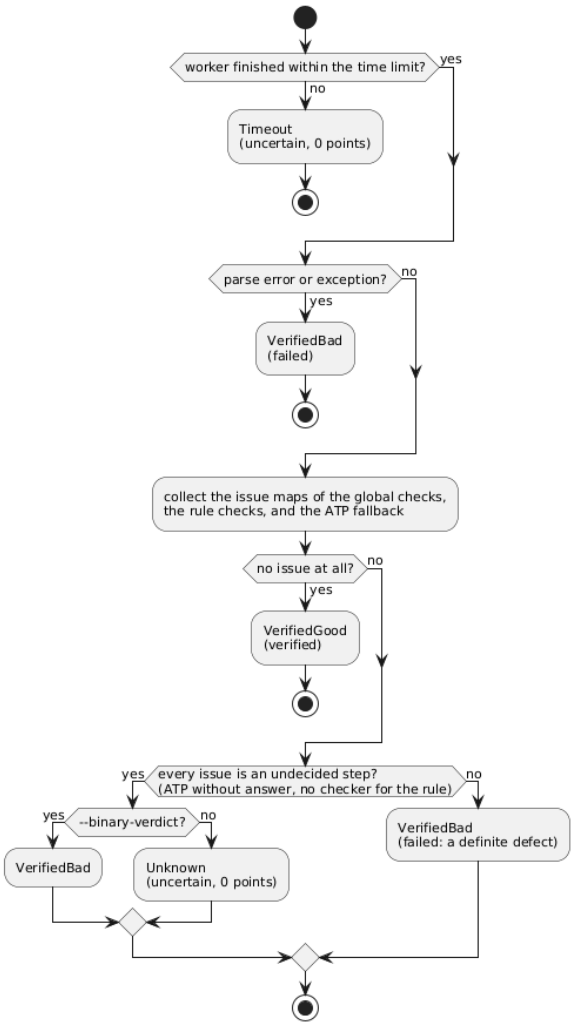


Figure 6: Decision flow for aggregating step-level outcomes into the final proof verdict.

1. If any step is invalid, a global check fails, or the proof cannot be processed (for instance because of a syntax error), the verdict is *failed*. An unparsable file is not a valid TSTP derivation.

2. Otherwise, if any step is unresolved or the time limit is exceeded, the verdict is *uncertain*.

3. Otherwise the verdict is *verified*.

This implements the policy derived in Section 4.2. Table 11 relates the verdicts to the SZS status printed by the verifier and to the score. The competition requires one of the statuses `VerifiedGood`, `VerifiedBad`, `Unknown`, and `Timeout` (Pro 2026). The command-line option `--binary-verdict` maps `Unknown` to `VerifiedBad` for a two-valued output.

Soundness argument. Suppose the verdict is *verified*. Then (i) every axiom and conjecture is faithful to the problem, (ii) the negated conjecture is the negation of the conjecture, (iii) the

Table 11: Verdicts, printed SZS status, and score under the ProoVer 2026 scheme.

Verdict	SZS output	Score if proof valid	Score if proof invalid
verified	`VerifiedGood`	$+1$	-10
failed	`VerifiedBad`	-1	$+2$
uncertain	`Unknown` or `Timeout`	0	0

dependency graph is acyclic, (iv) the derivation ends in `$false`, and (v) every other step is validated either by a rule check that shows entailment from the parents, by the Skolemization conditions, or by an ATP confirmation of entailment. Let Γ consist of the axioms and the negated conjecture, and let Γ_i be Γ together with the formulas of the first i steps in a topological order of the dependency graph. We show by induction that Γ_i is satisfiable whenever Γ is. The case $i = 0$ is trivial. If step i has status `thm`, its conclusion is entailed by its premises, which belong to Γ_{i-1}, so every model of Γ_{i-1} is a model of Γ_i. If step i is a Skolemization, its Skolem symbol is fresh, and its conclusion arises from the parent by replacing the existential variable by a Skolem term over the universal variables in scope; hence every model of Γ_{i-1} can be extended by a suitable interpretation of the Skolem symbol (a choice function) to a model of Γ_i. The last step derives `$false`, so Γ_n is unsatisfiable, and hence so is Γ. Since Γ consists of the axioms and the negation of the conjecture, Equation 1 shows that the conjecture follows from the axioms. The argument holds under the assumptions that the rule checkers are correct and that the ATP is sound. In this thesis, the first assumption is supported by extensive testing (Chapter 7) and the second is made explicit.

5.11 Resource Control

The wall-clock limit is enforced by running each proof in a separate worker process that is terminated when the limit is exceeded. The verdict is then *uncertain* and the status `Timeout`. Process isolation also contains crashes and memory exhaustion of a single proof. Formulas in the benchmark can be deeply nested. Since all tree traversals are recursive, the worker runs in a thread with a 64 MB stack and a recursion limit of 100 000. The dependency analysis, which operates on the whole graph rather than on a formula, uses an iterative search to be independent of proof length.

5.12 Diagnostics

Every issue records the step name and a reason in plain language, for example that a copied formula does not match its parent. The report lists issues grouped by check and finishes with a summary line in the SZS format required by the competition. Because issues are data, the same information drives the console report, the summary table over many proofs, and the assertions in the tests.

5.13 Design Decisions and Alternatives

5.14 Summary

The design translates the requirements into a pipeline of parsing, global checks, rule-local step checks with a restricted ATP fallback, and a verdict rule that is derived from the competition's

Table 12: Main design decisions and the alternatives considered.

Decision	Alternative	Rationale
Syntactic checks first, ATP as fallback	ATP for every step	Faster and independent of an external tool for the vast majority of steps; the ATP is only needed for rare or unspecified rules.
Rule-local checker functions	One generic entailment check	Precise diagnostics and testable units; a generic check can neither explain failures nor handle `esa` steps.
Own AST with grammar-based parser	Reuse of an existing TPTP parser	Full control over annotations such as `new_symbols` and `skolemize` and over the AST with n-ary junctions; an LALR grammar with cached tables keeps parsing fast. A general-purpose parser would have to be extended or wrapped to keep these annotations and the n-ary junctions, which is about as much work as a grammar for the required TPTP subset.
Three-valued verdict	Binary verdict	Matches the scoring asymmetry (Section 4.2).
Process isolation per proof	In-process timeout	Reliable termination of non-cooperative code and containment of crashes.

scoring. Its central property is that acceptance requires positive evidence for every step, which makes the soundness argument of Section 5.10 possible. Chapter 6 describes how the components are implemented in Python.

6 Implementation

This chapter describes how the design of Chapter 5 is implemented in Python. Section 6.1 gives an overview of the code base. Sections 6.2 and 6.3 describe the data model and the parser, Section 6.4 the driver that dispatches the steps and computes the verdict. Sections 6.5 to 6.9 cover the checkers, Section 6.10 the resource control, and Sections 6.11 and 6.12 the test suite and the tooling used for the evaluation. Section 6.13 presents selected algorithms in detail, and Section 6.14 follows one valid and one invalid proof through the implementation. Section 6.15 reports the defects that testing uncovered and how they were corrected, and Section 6.16 lists where the implementation falls short of the design.

6.1 Overview

The implementation is written in Python 3.12 and uses the parser library `lark` as its only runtime dependency. It consists of about 5 300 lines of code (4 160 in the `src` package and 1 120 in the driver `main.py`) in four layers (Table 13): the parser and the AST, the rule checkers with shared utilities, the driver, and the tooling for scoring and performance measurement. The separation

Table 13: Module structure of the implementation (approximate line counts).

Layer	Module (`src/...`)	Lines
Parser and AST	`parser/parser.py`, `parser/tptp_grammar.lark`, `parser/ast_nodes.py`	560
Constants	`var_mapping.py` (enumerations for roles, statuses, rules, connectives)	90
Global checks	`checker/file_dir.py`, `dependency_checker.py`, `negated_conjecture_checker.py`	500
Structural and propositional	`structural_checker.py`, `propositional_checker.py`, `junction_checker.py`	430
Quantifier rules	`instantiate_checker.py`, `existential_gen_checker.py`, `skolem_checker.py`	760
Clausal and equational	`clausal_checker.py`	470
Shared utilities	`alpha_eq.py`, `formula_utils.py`	400
ATP fallback	`external_atp_checker.py`	190
Driver	`main.py`, `deep_recursion.py`	1 130
Tooling	`scoring/`, `performance/`	650

follows the responsibilities of the design. Syntax handling is independent of logical verification, and each rule family has its own module, so that new constructs or rules can be added without touching the others.

Three principles guided the implementation, and they explain several decisions in the following sections. First, *checkers are pure functions*: a checker receives the step and its parent steps and returns a list of issue objects. It does not print anything, read files, or modify the AST. This makes every checker testable with a few formulas and independent of the parser (Section 6.11). Second, *checks are conservative*: a checker accepts a step only if it recognizes the exact shape that the rule definition demands, and a step in another shape is either rejected or, if its status permits, passed to the external prover (Section 6.9). An unrecognized shape is therefore never accepted without evidence. Third, *findings are data*: all issues are collected in maps keyed by step name instead of aborting at the first error, so that one run reports every defect of a proof.

The dependencies between the modules form a directed acyclic graph with the AST at the bottom: `ast_nodes` and `var_mapping` are imported by every other module, and `alpha_eq` and `formula_utils` by the checkers. The checkers do not import each other, and only `main.py` imports the parser, all checkers, and the ATP module. No checker imports the driver, which prevents circular dependencies and means that a checker can be loaded in a test without starting any part of the verifier.

6.2 Internal Representation

Formulas are represented by Python data classes defined in `ast_nodes.py`. Terms are represented by `Variable`, `Constant`, and `FunctionTerm`, atomic formulas by `Atom` and `Equality`, and compound formulas by `Negation`, `BinaryFormula`, `JunctionFormula`, and `QuantifiedFormula`. Conjunctions and disjunctions are `JunctionFormula` objects with a list of operands rather than nested binary trees, which simplifies the junction and clausal checks, since many rules operate on

lists of literals. A `QuantifiedFormula` stores the quantifier and the list of bound variables, which the instantiation, generalization, and Skolemization checks rely on.

A proof step is an `AnnotatedFormula` with name, role, formula, and either a source or an `InferenceRecord`. The record contains the rule name, the parents, and a list of information items: the status, the `new_symbols` declaration (kind and symbols), and, for Skolemization, the `skolemize` information with the eliminated variable and the Skolem term. Roles, statuses, rule names, quantifiers, and connectives are `StrEnum` classes in `var_mapping.py`. This avoids inconsistencies caused by ad hoc string handling and makes the set of recognized values explicit.

6.3 Parser

The parser is generated by Lark from the grammar `tptp_grammar.lark` using the LALR algorithm with cached tables. The parser object is created once per process through a cached helper. A `TPTPTransformer` maps the grammar productions to the data classes above. During this mapping some constructs are normalized: for instance, the connective `<=` is converted into a canonical implication direction, so that later checks have fewer cases. The parser also handles source annotations, inference records, and `include` directives. The function `load_proof` combines parsing with the extraction of the referenced problem and returns a `ProofFile` object.

Because the grammar is recursive, deeply nested formulas lead to deep recursion in the transformer and in all checkers. Section 6.10 describes how this is handled.

6.4 Driver and Dispatch

The driver is the function `check_proof_file` in `main.py`. It parses the problem file and the proof, builds an index from step names to steps, runs the global checks, and then iterates over the steps in file order. For each step it selects a checker by the step's role and rule name in an `if/elif` chain: axioms, conjectures, and every other formula with a `file(...)` source go to the provenance check, the negated conjecture to its dedicated checker, the rules of Table 9 to their modules, `skolemize` to the Skolem checker, and every other step with status `thm` or `cth` to the external ATP. Steps with another status and no checker, and formulas with an `introduced(...)` source, are reported as undecidable. A step without any justification counts as a defect. Figure 7 summarizes this end-to-end call sequence including timeout and ATP fallback branches. The design describes this dispatch as a registry that maps a rule name to its checker, arity, and required status. The implementation uses the chain instead, which works but is not table-driven (Section 6.16).

The driver does not stop at the first error. Each check writes its findings to a result map keyed by step name. There is one map per rule family (for example `skolem_issues` or `resolution_issues`), one for the dependency checks, and one for the ATP fallback. The maps are returned to the caller, so that the console report, the summary table, and the tests all work on the same data (Section 5.12). The driver keeps two sets for the freshness check of Skolem symbols: the function symbols of the problem file, collected once, and the Skolem symbols introduced so far. A `skolemize` step is rejected if its new symbol is in one of these sets.

Verdict. The checker `check_proof_file` returns the issue maps together with the flag `all_ok`, which is true if all maps are empty. The function `classify_verdict` maps the issue maps to

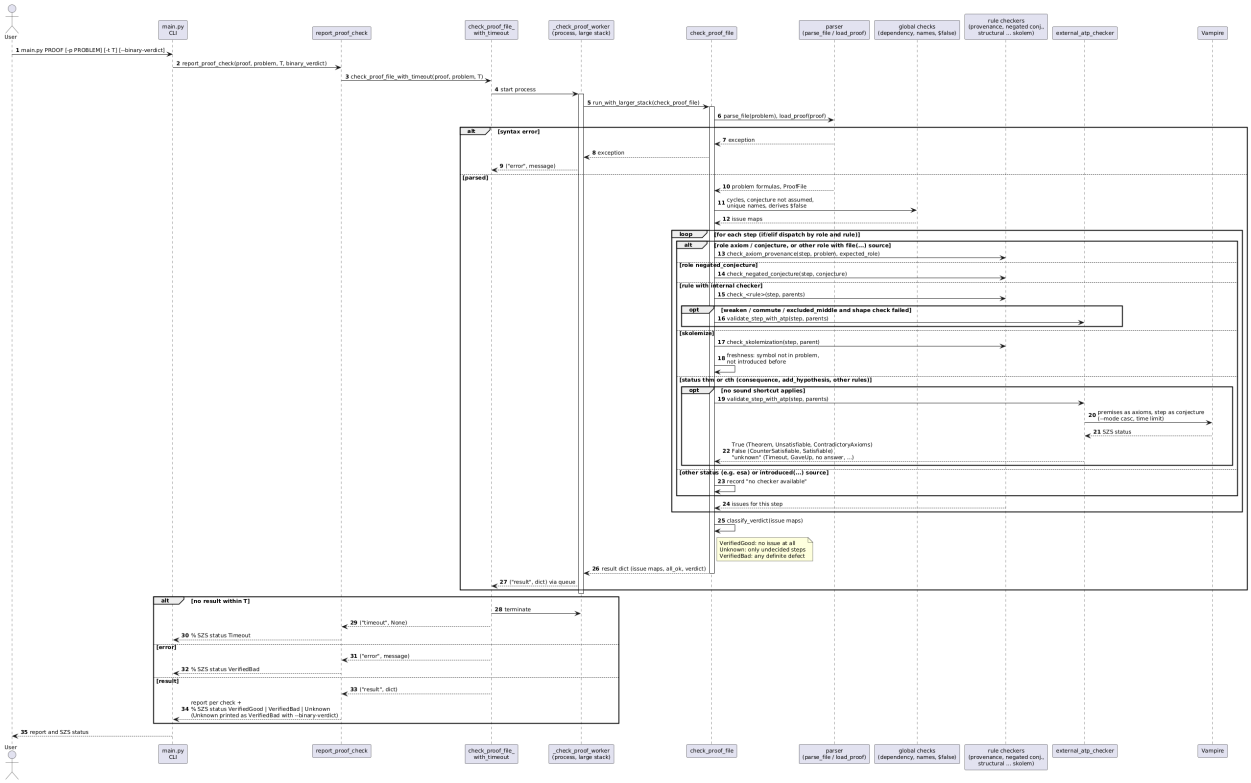


Figure 7: Call sequence of `check_proof_file` including parser, global checks, rule dispatch, ATP fallback, and verdict aggregation.

the three-valued verdict of Table 11 and stores it in the result under `verdict`. The verdict is `VerifiedGood` if all maps are empty. It is `Unknown` if the only findings are steps that could not be decided, either because the prover gave no result (timeout, prover missing, no SZS status) or because no checker exists for the rule. In every other case it is `VerifiedBad`. A mixture of undecided and refuted steps is therefore `VerifiedBad`. The reporting function prints the verdict as the SZS status. A timeout of the whole proof yields `Timeout`, and a crash or an unparsable file yields `VerifiedBad`. With the option `--binary-verdict`, `Unknown` is printed as `VerifiedBad`, and the scoring module treats `Unknown` like `Timeout` with zero points.

6.5 Global Checks

The provenance check in `file_dir.py` verifies that an imported formula (an axiom, a conjecture, or another input role such as `hypothesis`) carries a `file(...)` annotation, that the cited file and formula name exist in the problem, that the role matches, and that the formula is alpha-equivalent to the original. Alpha-equivalence is implemented in `alpha_eq.py` as a simultaneous traversal with two mappings between bound variables. Before the traversal, consecutive quantifiers of the same kind are merged into one prefix and nested junctions of the same connective are flattened (Figure 8). The comparison is used for provenance, for the structural rules, and for many quantifier rules.

Two further global checks are implemented in the driver itself, because they concern the proof as a whole. The first rejects a proof in which two steps have the same name. Without it, the index

```
def is_alpha_equivalent(f1, f2):
    return _alpha_eq(normalize(f1), normalize(f2), {}, {})

def _alpha_eq(a, b, map1, map2):
    if type(a) is not type(b): return False
    if a is a Variable:                        # bound or free
        if a in map1 or b in map2:             # bound: mappings must agree
            return map1.get(a) == b and map2.get(b) == a
        return a == b                          # free: names must be equal
    if a is a Quantified formula:
        require same quantifier and arity
        extend map1, map2 with the bound variables of a and b
        result = _alpha_eq(a.body, b.body, map1, map2)
        restore map1, map2                     # scoping
        return result
    otherwise: compare symbols, then recurse over the children in order
```

Figure 8: Structure of the alpha-equivalence check (pseudocode, simplified).

from names to steps would silently keep the first definition, and a parent reference could then refer to a different formula than the author intended. The check counts the occurrences of each name and reports every name that occurs more than once together with the number of definitions. The second check requires that the proof derives `$false`: at least one step that is neither an axiom nor a conjecture must have the formula `$false` and an inference record. Without this check, a proof that stops after some correct steps would be accepted, as all of its steps are valid, although it does not refute anything. The condition "has an inference record" excludes an axiom or conjecture that is literally `$false`. Both checks write their findings into their own maps (`duplicate_name_issues` and `no_false_issues`), which enter the verdict like the others.

The dependency checks in `dependency_checker.py` detect cycles with an iterative depth-first search with three colors and report the cycle. Dangling parents are ignored there, because the step checks report them. A second function rejects any step other than the negation step that cites a `conjecture`. The negated conjecture checker verifies role, status `cth`, the parent, and that the formula is the negation of the conjecture, up to renaming of bound variables.

6.6 Structural, Propositional, and Quantifier Rules

The structural rules (`copy`, `duplicate`, `rename_variable`, `double_negation`, `remove_double_negation`) are checked in `structural_checker.py`: the checker verifies the metadata and that the conclusion has the expected relation to its parent, mostly by alpha-equivalence. The propositional rules (`modus_ponens`, `conjunction`, `split_conjunct`) and the junction rules (`weaken`, `commute`, `excluded_middle`) act on the top-level connective structure, and here the n-ary junction nodes make the checks simpler. The syntactic checks for `weaken`, `commute`, and `excluded_middle` cover only the standard shapes of the rules. If such a check fails and the step has status `thm`, the driver asks the external ATP before it reports a failure, so that valid but unusual variants are not rejected.

The `instantiate` and `existential_gen` checks use template matching (`match_template_to_instance` in `formula_utils.py`): the quantified parent is treated as a template with abstract variables, and the checker searches for a consistent substitution that turns it into the conclusion,

or the other way around for generalization. Shared utilities for flattening quantifier prefixes, collecting function symbols, and applying substitutions are collected in `formula_utils.py`, so that all checkers interpret formulas consistently.

6.7 Skolemization

The Skolem checker (`skolem_checker.py`, the largest checker) implements the conditions of Section 5.8. It first validates the shape of the step: rule `skolemize`, exactly one parent, at least one status and only the status `esa` (a repeated `status(esa)` occurs in a valid benchmark proof), exactly one `new_symbols` declaration of kind `skolem` with one symbol, and a `skolemize` information item that names the eliminated variable. It then locates the quantifier that binds the eliminated variable in the parent, analyzing its syntactic and its effective quantifier kind while tracking the polarity (negations flip it, the antecedent of an implication flips it, and the operands of `~|` and `~&` flip it), and computes the universal variables in whose scope it occurs. Finally it checks that the Skolem term is built from the declared symbol and exactly these variables, and that applying the replacement to the parent yields the conclusion up to alpha-equivalence. Before this comparison, bound variables are renamed apart to avoid capture. The freshness of the symbol is not decided in the checker itself but in the driver, which holds the symbol sets across steps (Section 6.4).

6.8 Equality and Clausal Reasoning

The module `clausal_checker.py` implements `resolution`, `paramodulation`, `reflexivity`, `transitivity`, and `rewrite`. Resolution decomposes the parents into lists of disjuncts, searches for complementary literals, computes unifiers where necessary, builds the resolvent, and compares it to the stated conclusion. Quantified clauses are handled by flattening the leading universal prefix and rebuilding it for the conclusion. Paramodulation rewrites occurrences of one side of an equation from one parent in the other parent and compares the result with the conclusion. Bound variables are renamed apart before rewriting to avoid variable capture. Reflexivity checks for a trivial equality $t = t$, and transitivity checks that two parent equalities chain to the conclusion.

The `rewrite` rule is the most restrictive check. In the benchmark format, rewrite steps cite only one parent and omit the demodulating equation, so the intended transformation cannot be reconstructed. The checker therefore accepts a step only if its conclusion is the parent formula up to renaming of bound variables, which is sound, because the conclusion is then trivially entailed. A genuine rewrite that changes the formula is rejected. This is incomplete but fail-closed, and no valid proof of the benchmark contains such a step. We did not delegate these steps to the prover, because the invalid proof `PRV024` contains rewrite steps whose conclusion follows from the parent but not by the cited rule, and an entailment test would accept it (Section 7.8).

Overview of the rule checkers. Table 14 summarizes, for each rule family, what the internal check establishes and what it leaves to the prover. It also shows where precision is limited: the internal check of a rule is exact for the standard shape of the rule, and the fallback decides all other cases.

Table 14: Rule families, their internal checks, and the behavior on a failed check.

Rules	Internal check	On failure
`copy`, `duplicate`, `rename_variable`	Conclusion alpha-equivalent to the parent (duplicate: both parents equal)	Issue
`double_negation`, `remove_double_negation`	Conclusion differs from the parent by exactly one double negation at the top	Issue
`modus_ponens`, `conjunction`, `split_conjunct`	Conclusion is the consequent of the implication whose antecedent is the other parent; conjunction of the parents; a conjunct of the parent	Issue
`weaken`, `commute`, `excluded_middle`	Standard shapes: added disjunct, permuted operands, $A \vee \neg A$	ATP fallback if status is `thm`
`instantiate`, `existential_gen`	Template matching with a consistent substitution and capture check	Issue
`skolemize`	Shape, polarity, scope, Skolem term, replacement (Section 6.7); freshness in the driver	Issue
`resolution`, `paramodulation`	Unification of complementary literals (equation sides), resolvent compared to the conclusion	Issue
`reflexivity`, `transitivity`	$t = t$; chain of two equalities	Issue
`rewrite`	Conclusion equals the parent up to renaming	Issue
`consequence`, `add_hypothesis`, all other rules with status `thm` or `cth`	None	Decided by the ATP

6.9 Delegation to the External ATP

The module `external_atp_checker.py` implements the fallback of Section 5.9. The function `status_allows_atp` implements the eligibility rule: only the statuses `thm` and `cth` may be delegated. The function `validate_step_with_atp` serializes the premises as axioms and the step as conjecture into a TPTP problem (with quoting of symbols and variable names that are not valid TPTP tokens), writes it to a temporary file, and starts the prover as a subprocess with a time limit. The default prover is Vampire in CASC mode. The command can be configured through the environment variables `EXTERNAL_ATP_COMMAND` and `EXTERNAL_ATP_BINARY`, and the module translates file paths if the prover is launched through WSL. The answer is read from the SZS status line of the output (Figure 9).

The function returns a tri-state value, so that the caller can distinguish a refutation from an unresolved call. Only the answers `Theorem`, `Unsatisfiable`, and `ContradictoryAxioms` count as confirmation. The last one is accepted because a final step `$false` with parents that are contradictory in themselves is a valid refutation, and Vampire reports exactly this status when

```
validate_step_with_atp(step, premises, timeout) -> True | False | "unknown"
    write premises as axioms, step as conjecture to a temporary .p file
    run prover (subprocess, timeout)
    executable not found        -> "unknown"
    timeout                     -> "unknown"
    no "% SZS status" line      -> "unknown"
    Theorem | Unsatisfiable | ContradictoryAxioms -> True
    CounterSatisfiable | Satisfiable              -> False
    any other status (Timeout, GaveUp, ...)       -> "unknown"
```

Figure 9: Result mapping of the ATP fallback.

```
def check_proof_file_with_timeout(proof_path, problem_path, timeout):
    queue = Queue()
    process = Process(target=_check_proof_worker,
                      args=(proof_path, problem_path, queue))
    process.start()
    process.join(timeout)                 # wait at most ‘timeout‘ seconds
    if process.is_alive():                # limit exceeded
        process.terminate()
        process.join()
        return "timeout", None
    try:
        return queue.get_nowait()         # ("result", dict) or ("error", msg)
    except Empty:                         # worker died without a result
        return "error", f"exited with code {process.exitcode}"
```

Figure 10: Timeout handling of the driver (excerpt from `main.py`).

the premises are inconsistent. Only `CounterSatisfiable` and `Satisfiable` are reported as a refutation of the step. Every other status, for instance `Timeout`, `GaveUp`, or `ResourceOut`, and missing output count as unresolved, because they say nothing about the step. The prover receives only the parents of the step, never the original conjecture or the rest of the proof. The calls are sequential, and each call may use the full per-proof time limit. Two sound shortcuts avoid prover calls: an `add_hypothesis` step of the form $H \Rightarrow \varphi$ with parent φ, and a final `consequence` step to `$false` whose parents are complementary. All other steps of these rules go to the prover.

6.10 Resource Control

Each proof is checked in a separate process created with Python's `multiprocessing` module. The parent waits for the result for the configured number of seconds (30 by default, configurable on the command line) and terminates the worker if the limit is exceeded. The worker passes its result back through a queue, and exceptions are caught and returned as an error message. Figure 10 shows the mechanism.

A worker that dies without a result (for example, because the operating system killed it for memory) is reported as an error and not as a verdict. The prover is started from inside the worker as a child process, so terminating the worker does not by itself stop a running prover. However, each prover call has its own time limit (`--time_limit` and the timeout of `subprocess.run`), so that an

orphaned prover ends shortly after. Process isolation contains crashes and memory exhaustion of a single proof and allows non-cooperative code, such as a hanging ATP call, to be terminated reliably.

Inside the worker, the checking runs in a dedicated thread with a stack of 64 MB and a recursion limit of 100 000 (`deep_recursion.py`). This is necessary because both the parser's transformer and the checkers traverse formulas recursively, and the benchmark contains stress tests with hundreds of nested negations or function applications. Raising only the recursion limit would risk a hard interpreter crash, because each Python frame also needs space on the operating system's thread stack. The dependency analysis uses an iterative search and is therefore independent of proof length.

6.11 Test Suite

The test suite consists of 293 `pytest` test functions in 26 files. Most files exercise one module in isolation, for example `test_skolem_checker` or `test_clausal_checker`, which is possible because the checkers are functions of the step and its parents. In addition, there are tests for the parser and the AST, a parametrized provenance test over the example proofs (`test_all_examples`), tests for the tri-state ATP result (`test_external_atp_checker`, `test_main_atp_tristate`), edge-case tests, and a mutation-based test (`test_mutation_fuzzing`) that perturbs seed proofs and checks that the verifier reacts with the expected outcome or issue category. A GitHub Actions workflow runs the suite on Python 3.12 for every push and pull request and fails if the line coverage is below 85 % (NFR7). Of the 293 tests, two are skipped when no external prover is installed: the whole-suite test over the Extra set (Section 7.3) and the unmutated baseline of the mutation test need the prover for the final step of each proof. All other tests run without a prover, which keeps the continuous integration independent of it. The tests fall into four groups. *Unit tests* call one checker with hand-written formulas (for example, an instantiation that captures a variable). *Regression tests* reproduce each defect that was found (Section 6.15) with the smallest formula pair that triggers it. They were written before the correction and fail on the old code. *Property-style tests* perturb seed proofs and check that the verdict changes in the expected way. *End-to-end tests* run the driver on whole proofs and compare the report with the expectation. The statement coverage of the checker package is 87.2 %, and 90.4 % for the whole `src` directory (Section 7.5).

6.12 Tooling for Reporting and Evaluation

The command-line interface in `main.py` checks either a single proof or, by default, all proofs of the directory `ProoVer2026/PRV` (another directory with `-d`). For each proof it prints a report grouped by check and finishes with an SZS line, and for a batch it prints counts of the three outcomes and a per-proof summary table with the failed checks. The scoring module (`src/scoring`) compares the outcomes with expected results given in CSV files and computes the points of the ProoVer scheme ($+2$, $+1$, 0, -1, -10) per proof and in total, together with a breakdown of errors by check category. The performance module (`src/performance`) records wall-clock and CPU time per operation category and writes them to a CSV file. The scripts in `scripts/` automate the experiments of Chapter 7. `gen_extra_tests.py` generates the rule-level proofs of the Extra set from templates, and `exp_extra_tests.py` runs them and compares the verdicts with the ground truth. `eval_all.py` is the driver of the evaluation: it checks every proof in a fresh process (`_eval_worker.py`) several times, parses the branch log of the driver to count how each step was decided, measures wall-clock time and peak memory with `resource.getrusage`, applies the mutation operators, runs

the malformed-input cases, and writes all tables and macros of the evaluation chapter as LaTeX files, so that no number in that chapter is typed by hand. The experiments with the Skolemization checker and the compatibility of the prover versions have their own scripts (`exp_skolem_mutants.py`, `exp_vampire_compat.py`). `gen_dag_figure.py` draws the dependency graph of a proof with Graphviz and colours each node by the check that validates it (Figure 4).

6.13 Selected Algorithms

This section shows the core of three algorithms as they appear in the code, because small details in them decide whether the verifier is sound: template matching with a capture check, unification with an occurs check, and the standardization apart of the premises of a resolution step. Each was corrected after a defect was found by the Extra set or by the final review (Section 6.15), and the excerpts show the corrected form.

Template matching. The checks for `instantiate` and `existential_gen` must decide whether a conclusion is an instance of a quantified parent. The quantified variables of the parent are the *abstract variables*: they may be mapped to any term, but each to the same term everywhere. All other variables, constants, and symbols must match exactly, and variables bound inside the formula must correspond one to one (Figure 11). The parameter `local_bound` carries the variables that are bound by quantifiers *inside* the instance on the path to the current position. If an abstract variable were mapped to a term that contains such a variable, the term would be captured by the inner quantifier and change its meaning. For example, instantiating x in $\forall x \exists y\, p(x,y)$ with the term y yields $\exists y\, p(y,y)$, in which the substituted y has been captured by the quantifier. This formula is not a consequence of the parent (take $p(x,y)$ to mean $x \neq y$).

Unification. The resolution and paramodulation checks need a most general unifier of two literals. The implementation uses substitutions as dictionaries from variable names to terms, which are extended while the arguments are unified from left to right (Figure 12). The function `_walk` follows the bindings of a variable until it reaches an unbound variable or a non-variable term. The occurs check rejects a binding $x \mapsto t$ if x occurs in t (after applying the current substitution). Without it, $p(x)$ and $p(f(x))$ would unify, and a step that depends on this unification would be accepted although no resolution step is possible.

Resolution. Figure 13 shows the structure of the resolution check. The universal prefixes of the two parents are flattened into variable lists and clause bodies. The variables of the second parent are renamed so that they differ from every name that occurs in the first parent. This is necessary because the variables of two clauses are independent even if they have the same name. Without the renaming, resolving $\forall x\,(p(x) \vee q(x))$ with $\forall x\,(\neg p(f(x)) \vee r(x))$ would fail to unify $p(x)$ with $p(f(x))$ because of a spurious occurs-check failure, and conversely clauses that happen to share names could be unified wrongly. For each pair of complementary literals the checker computes a unifier, resolves it into idempotent form (the bindings are built in triangular form, $x \mapsto f(z), z \mapsto a$), applies it to the remaining literals, removes duplicates up to alpha-equivalence, rebuilds the clause with the universal prefix, and compares it with the conclusion. The comparison tolerates a different order of the quantified variables (`_matches_up_to_quantifier_permutation`), because the order

```
def match_template_to_instance(template, instance, abstract_vars,
                               bound_map=None, substitutions=None,
                               local_bound=frozenset()):
    if isinstance(template, Variable):
        mapped = bound_map.get(template.name)
        if mapped is not None:                # bound inside the formula
            return isinstance(instance, Variable) and instance.name == mapped
        if template.name in abstract_vars:
            if local_bound and term_vars(instance) & local_bound:
                return False                  # variable capture
            existing = substitutions.get(template.name)
            if existing is None:
                substitutions[template.name] = instance
                return True
            return nodes_equal(existing, instance, {})
        return isinstance(instance, Variable) and template.name == instance.name
    if type(template) is not type(instance):
        return False
    ...                                       # constants, terms, atoms,
    ...                                       # connectives: compare, recurse
    if isinstance(template, QuantifiedFormula):
        ...                                   # same quantifier and arity;
        ...                                   # extend bound_map with the bound
        ...                                   # variables; recurse with
        ...                                   # local_bound | instance.variables
```

Figure 11: Template matching with capture check (excerpt of `formula_utils.py`, abbreviated).

of the variables in a universal prefix does not matter. The search tries all pairs. Since the clauses in the benchmark have at most a few literals, the quadratic search is not a bottleneck (Section 7.6).

Negation of the conjecture. The checker for the negated conjecture must accept the formulas that a prover writes for the negation of the conjecture and reject all others. It computes the expected formula by negating the conjecture and pushing the negation inward through quantifiers and connectives: the negation of a universally quantified formula is an existentially quantified negation, the negation of a conjunction a disjunction of negations, and so on, and the negation of a negation $\neg A$ is A. The checker accepts the stated formula if it is alpha-equivalent to the plain negation $\neg F$ or to the pushed form. The case of a conjecture with several negations shows why the details matter: for $F = \neg\neg Y$, the pushed form is $\neg Y$ (removing one negation), and not Y (removing two). Accepting Y would let a proof start from a formula equivalent to the conjecture itself, so that the verifier would accept a derivation of a contradiction from the conjecture and call it a refutation of its negation (Section 6.15).

6.14 Worked Examples

This section follows two proofs through the verifier: `PRV005`, which is valid (one of the nine proofs of the original set that were later set aside, Section 7.1, but a clear example), and `PRV008`, which is invalid because of a subtle error in the Skolemization. Both refute the conjecture $\exists x \exists y\, s(x, y)$ from

```
def _occurs(name, term, subst):
    term = _walk(term, subst)
    if isinstance(term, Variable):
        return term.name == name
    if isinstance(term, FunctionTerm):
        return any(_occurs(name, arg, subst) for arg in term.args)
    return False

def _unify_terms(t1, t2, subst):
    t1, t2 = _walk(t1, subst), _walk(t2, subst)
    if isinstance(t1, Variable) and isinstance(t2, Variable) \
            and t1.name == t2.name:
        return subst
    if isinstance(t1, Variable):
        if _occurs(t1.name, t2, subst): return None      # occurs check
        return {**subst, t1.name: t2}
    if isinstance(t2, Variable):
        if _occurs(t2.name, t1, subst): return None
        return {**subst, t2.name: t1}
    if isinstance(t1, FunctionTerm) and isinstance(t2, FunctionTerm):
        if t1.functor != t2.functor or len(t1.args) != len(t2.args):
            return None
        for a, b in zip(t1.args, t2.args):
            subst = _unify_terms(a, b, subst)
            if subst is None: return None
        return subst
    ...                                                  # constants
```

Figure 12: Unification with occurs check (excerpt of `clausal_checker.py`).

the axiom

$$\forall x \exists y \left(s(x,y) \wedge \forall x\, t(x,y) \right),$$

in which the inner quantifier binds the same name x as the outer one.

The valid proof. `PRV005` has three derived steps. The step `neg` states $\neg\exists x \exists y\, s(x,y)$ with the rule `negated_conjecture`. The step `sk` Skolemizes the axiom `a1`: the existential variable y is replaced by $sK0(x)$, and the checker finds that y lies in the scope of exactly one universal variable, x, so that the Skolem term $sK0(x)$ is correct. The conclusion is

$$\forall x \left(s(x, sK0(x)) \wedge \forall x_1\, t(x_1, sK0(x)) \right);$$

the inner variable was renamed to x_1, so that the term $sK0(x)$ in the inner conjunct still refers to the outer x. Finally, `bot` derives `$false` from `sk` and `neg` with the generic rule `consequence`. The driver processes the steps in file order. The axiom and the conjecture pass the provenance check, `neg` the negated-conjecture check, and `sk` the Skolem checker, including the freshness of `sK0`, which neither occurs in the problem nor was introduced earlier. The step `bot` is the only one that needs the prover: it is sent as the problem "`sk, neg` $\vdash$ `$false`", and the answer `ContradictoryAxioms` takes 11 ms. The global checks (no cycle, unique names, final `$false`) pass, and the verdict is

```
def check_resolution(step, first_parent, second_parent):
    issues = _check_inference_metadata(step, RESOLUTION)
    vars1, body1 = flatten_prefix(first_parent.formula, UNIVERSAL)
    vars2, body2 = flatten_prefix(second_parent.formula, UNIVERSAL)
    vars2, body2 = _rename_apart(vars2, body2,        # standardize apart
                                 _collect_all_names(first_parent.formula))
    lits1, lits2 = _flatten_disjuncts(body1), _flatten_disjuncts(body2)
    for i, l1 in enumerate(lits1):
        for j, l2 in enumerate(lits2):
            (a1, neg1), (a2, neg2) = _strip_negation(l1), _strip_negation(l2)
            if neg1 == neg2: continue                 # not complementary
            subst = _unify_atoms(a1, a2, {})
            if subst is None: continue
            subst = _idempotent(subst)                 # resolve triangular form
            rest = [substitute(l, subst) for k, l in enumerate(lits1) if k != i]
            rest += [substitute(l, subst) for k, l in enumerate(lits2) if k != j]
            resolvent = _build_resolvent(dedupe_alpha(rest), order)
            if _matches_up_to_quantifier_permutation(resolvent, step.formula):
                return issues                          # success
    issues.append(ClausalIssue(step.name, "no pair of complementary, "
                  "unifiable literals produces the stated conclusion"))
    return issues
```

Figure 13: Structure of the resolution check (excerpt of `clausal_checker.py`, abbreviated).

`VerifiedGood`.

The invalid proof. `PRV008` contains the same Skolemization, but with the conclusion

$$\forall x \left(s(x, sK0(x)) \wedge \forall x\, t(x, sK0(x)) \right),$$

in which the inner quantifier still binds x. In this formula, the second occurrence of $sK0(x)$ is in the scope of the inner $\forall x$, so that its argument refers to the inner variable and not to the outer one. The conclusion therefore says that $t(x, sK0(x))$ holds for all x, which is not the formula that the rule prescribes: the Skolemization of the axiom keeps $t(x', sK0(x))$ for all x' with the argument of the Skolem function fixed by the outer variable. The error is easy to overlook, because the stated formula looks like the Skolemized axiom, and the shadowing of the name x is the only difference. The Skolem checker detects this when it compares the result of the replacement with the stated conclusion: it substitutes $sK0(x)$ for y in the parent and renames the bound variables apart before the comparison, with the result that the substituted term in the inner conjunct contains a free x and is not alpha-equivalent to the stated formula. The checker reports “The resulting formula is not a correct Skolemization of the parent formula” for the step `sk`. All other checks pass, and the verdict is `VerifiedBad`. The remaining steps of the proof are not affected by the error. The example shows why the checker validates each Skolemization step against the rule and not only against the final refutation: the defect is in one step whose conclusion cannot be judged by asking whether the proof as a whole is plausible.

The report. For every proof the driver prints a report grouped by check, and finishes with the SZS status. For a valid proof, each group shows a line `[OK] All ... steps verified correctly` and the final line is `% SZS status VerifiedGood`. For an invalid proof, the group of the failed check lists each affected step with the reason, as in the case of `PRV008` above, and the final line is `% SZS status VerifiedBad`. The same data are available to the tests as the issue maps, which makes it possible to assert on the cause of a rejection and not only on the verdict.

6.15 Defects Found During Testing

Writing the Extra set (Section 7.3) exposed three defects in the checkers, none of which any earlier test or benchmark proof had triggered. We describe them here, together with the defects found later, because they show where a verifier typically goes wrong. None of the corrections changed a verdict on the ProoVer set.

Doubly negated conjecture. The function that computes the negation of a conjecture removed *two* negations from a conjecture of the form $\neg\neg Y$ and returned Y instead of $\neg Y$. As a result, the checker accepted Y, a formula equivalent to the conjecture itself, as the negated conjecture. A proof that derives a contradiction from the conjecture and the axioms, which shows that the conjecture is *false*, would then have been accepted as a refutation of the negated conjecture, that is, as a proof of the conjecture. The correction removes exactly one negation, and the accepted forms are the plain negation and the one-level pushed form. A regression test uses the conjecture $\neg\neg p$ with the formulas p and $\neg p$ as candidate negations.

Variable capture in `instantiate`. The matching function allowed an abstract variable to be mapped to a term that contains a variable bound by a quantifier inside the formula, so that instantiating $\forall x\,\exists y\,p(x,y)$ with y gave $\exists y\,p(y,y)$. The correction is the parameter `local_bound` of Figure 11. The first version of the correction had no effect: it passed the extended set only in the recursive calls of the term cases, but not in the call for quantified formulas, which is where the set grows. A regression test with exactly this formula pair found this.

Resolution without standardizing apart and without occurs check. Resolution used the variable names of both parents unchanged and a unifier without occurs check. Two clauses that share a variable name by accident therefore constrained each other, and circular bindings were accepted. The corrections are the renaming of Figure 13 and the occurs check of Figure 12.

Polarity below implications. The Skolem checker flipped the polarity of a quantifier below a negation, but not in the antecedent of an implication. A step that Skolemized `?[X]: p(X)` in the antecedent of `(?[X]: p(X)) => r` was therefore accepted, although the quantifier is effectively universal there and cannot be eliminated. The defect was found when the measurements of the single-step comparison (Section 8.5) disagreed with the entailment oracle on the step `s1` of `TST064`, and not by the benchmark. The corrected traversal derives the polarity of the operands from the connective (implication, reverse implication, `~|`, `~&`), and rejects a quantifier below `<=>` or `<~>`, where both polarities occur. Four regression tests (existential and universal in the antecedent, existential in the consequent, equivalence) cover it.

Defects found in the final code review. A review of the code before the release found further defects, none of which changes a verdict on the ProoVer or Extra set. First, the driver checked the provenance of formulas with the roles `axiom` and `conjecture`, but a formula with the role `hypothesis` (or `definition`, `lemma`) and a `file(...)` source was not checked at all, so that a proof could introduce an arbitrary assumption under this role. The driver now checks every formula with a `file(...)` source against the problem, including its role, and treats a formula with an `introduced(...)` source, which has no premises, as undecidable. Second, the ATP fallback reported every SZS status other than a confirmation as a refutation, so that a prover that stopped at its own time limit (`Timeout`) rejected the step instead of leaving it unresolved. In the measured runs all refutations had the status `CounterSatisfiable`, so no result is affected. Third, the grammar rule for `introduced(...)` sources matched the introduction type as an anonymous literal that Lark discards, so that the transformer failed on every such source and the proof was rejected as unparsable. Fourth, the review of the individual checkers found four soundness defects in cases that the benchmark rarely or never exercises:

- Resolution applied the unifier in a single pass, although the unification algorithm builds it in triangular form. A binding such as $\{X \mapsto f(Z), Z \mapsto a\}$ therefore produced the resolvent $\forall Z\, q(f(Z))$ instead of $q(f(a))$, and a conclusion that is too strong was accepted.
- The capture check of the template matching (Figure 11) lost the set of bound variables below a negation, so that `instantiate` accepted $\exists Y \neg p(Y,Y)$ from $\forall X \exists Y \neg p(X,Y)$.
- `instantiate` could rename a kept universal variable to a name that is bound further inside the formula.
- `existential_gen` accepted the generalization of a single operand of a disjunction, which is only valid for a conjunction.

In addition, `existential_gen` used the variable mapping in the wrong direction, which only caused valid steps to be rejected, and a Skolemization step without a status was not reported. Each correction comes with regression tests that fail on the old code. The comparison on all 285 proofs shows no change of a verdict on the ProoVer and Extra sets.

Lessons. The first three defects all have to do with variable names and scopes, which is the usual source of trouble in first-order proof checking. Each could be found by a small, hand-written test case, but not by the benchmark, because the proofs of the benchmark are produced by provers and use fresh variable names consistently. This supports two design decisions, namely the rule-level test suite with deliberately injected defects and the principle that a rule check is conservative. It also shows the limits of the approach. A defect of this kind is only found if a test exists for it, so the checkers should be developed from the definition of each rule and its side conditions, as in Section 5.8, and not from the examples at hand. The polarity defect is of the same kind, because it concerns a side condition that the benchmark never exercises.

6.16 Deviations from the Design

The implementation does not yet cover the design completely:

- The dispatcher is an `if/elif` chain instead of a registry, so a new rule needs a new function and a new branch in the driver.
- Every ATP call may use the full per-proof limit instead of the remaining budget, so a single slow call can use up the time of the whole proof (Section 7.6).
- For `weaken`, `commute`, and `excluded_middle`, a prover call without result is reported as a failure of the step and not as an undecided step. This is the conservative choice.
- `new_symbols` annotations of kind `herbrand` are parsed but not checked separately.
- The `rewrite` rule only accepts steps whose conclusion equals the parent, so genuine rewrites would be rejected.

The global check that the derivation ends in `$false` (FR9) and the check of unique step names, which were missing in an earlier version, are implemented (Section 6.5).

One further limitation concerns the correctness of an individual check rather than the architecture. A quantifier below an equivalence or an exclusive disjunction occurs in both polarities, so its Skolemization is not defined, and the Skolem checker rejects such a step. An earlier version flipped the polarity below a negation but not in the antecedent of an implication and therefore classified such steps wrongly (Section 6.15). The correction is covered by tests. In addition, the grammar follows TPTP and treats a word with an upper-case initial as a variable. A premise name such as `S0` or a Skolem symbol such as `SK0` therefore leads to a parser error, and the proof is rejected (Section 7.9). This is the intended behavior and not a limitation.

7 Evaluation

This chapter evaluates the verifier against the research questions of Section 1.2. Section 7.1 describes the benchmarks, the metrics, the external prover, and the experimental setup. Section 7.2 reports correctness and score, Section 7.3 the results of the rule-level test suite, and Section 7.4 the coverage of the inference rules (RQ1). Section 7.5 reports the properties of the architecture (RQ2), Sections 7.6 to 7.8 run time, robustness, and ablations (RQ3), and Sections 7.9 and 7.10 analyze the remaining errors and the threats to validity.

7.1 Setup

Benchmarks. The evaluation uses three proof sets. The *ProoVer set* consists of the 100 problem/proof pairs that were initially provided for ProoVer 2026, of which 47 are valid and 53 contain a deliberate error according to the expected results. Of these, only 91 pairs (directory `PRV`, 44 valid, 47 invalid) form the actual benchmark. The remaining 9 pairs (directory `Deprecated`) no longer appear in the results published by the organizers and were apparently not scored in the competition, whose scores refer to the 91 pairs. Their expected results are only known from the version of the benchmark distributed before the results, so that a verdict on them cannot be scored reliably. We therefore treat the score on the 91 pairs as the primary result and report the full set of 100 pairs only for completeness. Where a deprecated proof serves as an example below (`PRV005`, `PRV057`), this is noted. The expected results of the 91 pairs are taken from the file `results.csv` that the organizers

Table 15: Groups of the TST set.

Group	Proofs	Purpose
`ax`	11	Provenance: modified, renamed, or re-roled axioms and conjectures, missing `file` directive
`c`	13	Correct proofs with edge cases: Skolemization, renamed steps, unordered steps, nested quantifiers
`cp`	3	Conjecture used as premise, inconsistent axioms
`nc`	11	Negated conjecture: wrong status, parent, rule, role, or formula; missing or duplicated
`sk`	25	Skolemization: arity, scope, freshness, polarity, wrong result, wrong number of parents or symbols
`st`	13	Structure: no `$false`, invalid `thm` step, cycles, undefined premises, unknown status

provide with the benchmark (Pro 2026), those of the deprecated pairs from the earlier version of the benchmark. The *TST set* (directory `Test`) consists of 76 proofs written for this thesis to test individual checks (Table 15). Each has an expected result of *correct* or *incorrect*. Five are *edge cases* whose expected result is open (for example two negated conjectures, or a quantifier block in a different order), and are excluded from the agreement counts. The *Extra set* (directory `Extra`) consists of 109 proofs that were generated for the rules that the TST set covers only marginally (resolution, paramodulation, reflexivity, transitivity, and the propositional rules). It is described in Section 7.3.

Metrics. The primary metric is the total score under the ProoVer 2026 scheme (Section 4.2): $+1$ for a correctly accepted valid proof, $+2$ for a correctly rejected invalid proof, 0 for abstention, -1 for a wrongly rejected valid proof, and -10 for a wrongly accepted invalid proof. The verifier reports one of the three verdicts *verified*, *failed*, and *unknown* (SZS: `VerifiedGood`, `VerifiedBad`, `Unknown`). A proof is *unknown* if the only findings are steps that could not be decided (no checker for the rule, or no answer of the prover). Unknown and a time-out both score 0 points. Unless stated otherwise, the numbers refer to this default policy. The option `--binary-verdict` maps unknown to failed (the *binary policy*), and Section 7.8 compares the two. In addition we report the verdict matrix, the wrong acceptances separately, the run time per proof, the share of steps that are decided without the external prover, and the peak memory.

External prover. All steps that no internal checker decides are delegated to Vampire (Kovács & Voronkov 2013) in its CASC mode, and we use no other prover. The reason is that Vampire is the reference system for this task. It is the most successful system of the CASC competition (Sutcliffe 2016): it won all eight divisions of CASC-30 in 2025 (Sutcliffe 2025) and has collected a large number of trophies in the first-order divisions since 1999, so that it is generally regarded as the state of the art in automated theorem proving for first-order logic. For a verifier that treats the prover as a source of *negative* information as well (a step that Vampire cannot prove is rejected), the strength of the prover directly determines how many valid steps are wrongly rejected, which is why we chose the strongest available system instead of comparing several. The verifier addresses it through one

Table 16: Verdicts on the ProoVer set (number of proofs) and resulting score.

Expected \ verdict	verified	failed	unknown	timeout	Points
valid (47)	47	0	0	0	+47
invalid (53)	0	52	1	0	+104
Total	47	52	1	0	+151

command line (variables `EXTERNAL_ATP_BINARY` and `EXTERNAL_ATP_COMMAND`) and reads only the SZS status, so that another prover could be used with minimal changes, but this is not evaluated here. We used Vampire 5.0.1 (Release build, commit 1b13eaf on 2026-01-18 12:14:50 +0000).

Environment. All measurements were made on a laptop with 12th Gen Intel(R) Core(TM) i7-1255U (12 threads, 8 GB RAM available to the virtual machine) under Linux 5.15.167.4-microsoft-standard-WSL2 (WSL 2 on Windows), with Python 3.12.3 and `lark` 1.3.1, with the time limit of 30 s per proof. Each proof was checked in a fresh worker process and run 3 times. Run times are medians of these repetitions. Every run includes the start-up of the interpreter and the import of the verifier (median 1.05 s), because the competition measures the whole process. The measurements were made on 2026-10-21 with commit f74ab9c. All numbers in this chapter are produced by `scripts/eval_all.py`, which writes the tables and macros used here into `thesis/generated`.

7.2 Correctness and Score

Table 16 shows the verdicts of the verifier on the ProoVer set. The verifier accepts 47 of the 47 valid proofs, rejects 52 of the 53 invalid proofs, answers *unknown* for 1 invalid proof, runs into the time limit on 0 proofs, and accepts no invalid proof. On the 91 proofs of the actual benchmark, the primary result, the score is 136 of 138 points. Including the 9 deprecated proofs, whose expected results come only from the earlier version of the benchmark, it is 151 out of a maximum of $47 + 2 \cdot 53 = 153$ points (98.7 %). The verdicts on the deprecated proofs all agree with the expected results given for them. The two missing points stem from one proof, `PRV097`: it is invalid, and the verifier finds no error, but it also cannot verify the step with the rule `trivial` (status `esa`), for which neither a checker nor a prover justification exists, and therefore answers unknown instead of failed. With the binary policy the proof is rejected, and the score is 153 (138 on the competition proofs), that is, the maximum. All other verdicts are optimal. One proof (`PRV099`) is close to the limit for the prover and is discussed in Section 7.9. Two invalid proofs (`PRV091` and the deprecated `PRV057`) are rejected because they are not well-formed TSTP, which is the intended defect of these proofs. Section 7.9 discusses them.

The central safety property is that no invalid proof is accepted, because a wrong acceptance costs ten points. It holds for the ProoVer set (0 wrong acceptances) under both policies. On the TST set, the verdicts agree with the expectation in 70 of 71 proofs (excluding the 5 edge cases, Table 17). The remaining proof, `TST043`, is accepted although the test author marked it as incorrect: it derives `$false` from the axioms and the negated conjecture in a single step, which is a valid inference, only of coarse granularity. This is a matter of the definition of a correct proof rather than of unsound reasoning, and we discuss it in Section 7.9. A second proof, `TST072`, which imports an axiom of the

Table 17: Results on the TST set by group (agreeing / total without edge cases).

	ax	c	cp	nc	sk	st	Total
Verdict as expected	9/9	13/13	2/2	10/10	25/25	11/12	70/71

problem with the role `hypothesis`, was accepted in the first runs because the driver did not check formulas with this role at all. This defect is corrected (Section 6.15), and the proof is now rejected.

7.3 Rule-Level Test Suite

The TST set concentrates on provenance, the negated conjecture, and Skolemization, the points on which the competition proofs are most likely to contain errors. It therefore says little about the checkers of the other rules. To test them, we generated the Extra set with `scripts/gen_extra_tests.py`: for each of 20 rules, the generator produces a small problem and a proof that applies the rule correctly, and several variants with a specific defect (a wrong premise, an inverted conclusion, a missing side condition, a capture of a variable). Overall the set contains 35 valid proofs, 66 invalid proofs, and 8 edge cases, that is, proofs that are sound but unusual (for example a reflexivity step in an unusual orientation). Each case is a proof together with its problem file, and the set is run through the same batch mode of `main.py` as the other sets (`--dir ProoVer2026/Extra`). Of the 109 cases, 101 agree with the expectation (the edge cases not counted), there is no wrong acceptance and no wrong rejection, and the score is 167 points. The median run time is 1.33 s.

Table 18 shows that the verifier accepts all 35 valid proofs and rejects all 66 invalid proofs, and that in every case the rejection comes from the check of the rule itself, not only from the final `$false` step: the rule checker reports an error for 66 of the 66 invalid proofs. Of the 8 edge cases, 2 are accepted and 6 are rejected. We regard the rejections as cautious behavior, because they concern formulations that no competition prover produces.

The result above is the state after three corrections that the first run of the Extra set made necessary. Before them, the verifier accepted an invalid proof in which a doubly negated conjecture was “negated” by adding a negation, accepted a variable capture by `instantiate`, and accepted a resolution step whose premises shared a variable name or whose unifier was circular. We describe these defects and their corrections in Section 6.15. The corrections do not change any verdict on the ProoVer set, which we confirmed by re-running it with and without them. This also illustrates a limitation: the ProoVer set could not detect these defects, because it contains no proof that exercises them. The same holds for the defects found later, the polarity of a quantifier in the antecedent of an implication and the unchecked role `hypothesis` (Section 6.15). We compared the old and the corrected version on all 285 proofs of the four directories: the corrected Skolem check additionally rejects the step `s1` of `TST064`, a proof that was already rejected, and the corrected provenance check rejects `TST072`, which was wrongly accepted before. No other finding and no verdict on the ProoVer and Extra sets changes.

7.4 Rule Coverage (RQ1)

Table 19 lists, for each inference rule occurring in the ProoVer set, how many proofs and steps use it, and how its steps were decided: by the internal checker, by the external prover after a failed

Table 18: Results on the Extra set by rule. “By the rule check” counts the invalid proofs for which the checker of the rule itself, and not only the final step, reports the error.

	valid		invalid			
Rule	cases	accepted	cases	rejected	by the rule check	edge
add_hypothesis	1	1	1	1	1	–
commute	3	3	3	3	3	–
conjunction	1	1	2	2	2	1
consequence	0	0	1	1	1	–
copy	1	1	2	2	2	–
double_negation	1	1	2	2	2	–
duplicate	1	1	2	2	2	–
excluded_middle	1	1	4	4	4	1
existential_gen	2	2	4	4	4	–
instantiate	4	4	4	4	4	–
modus_ponens	3	3	5	5	5	–
paramodulation	4	4	8	8	8	2
reflexivity	1	1	4	4	4	1
remove_double_negation	1	1	2	2	2	–
rename_variable	1	1	2	2	2	–
resolution	4	4	9	9	9	1
rewrite	1	1	2	2	2	–
split_conjunct	1	1	2	2	2	1
transitivity	2	2	4	4	4	–
weaken	2	2	3	3	3	1
Total	35	35	66	66	66	8

internal check, by the external prover because no internal checker exists, or not at all. The 100 proofs contain 15590 derived steps of 35 different rules. Of these steps, 15457 (99.1 %) are decided by an internal checker, 132 are decided by the prover alone, 1 step needs the fallback, and no step of an accepted proof is unresolved. One further step, in the invalid proof PRV097, uses the rule trivial with the status esa, for which neither a checker nor a prover justification exists. The driver does not log this step, so it is not part of the table.

Figure 14 shows the same data from the perspective of the proofs: for each rule, the number of proofs that use it and whether its steps are decided internally or by the prover. The two views differ considerably. Counted by steps, modus_ponens and skolemize dominate, because the three large proofs consist almost entirely of them. Counted by proofs, consequence is the most frequent rule (95 of 100 proofs), followed by skolemize (47) and instantiate (30), and apart from consequence, which has no syntactic criterion, every rule that occurs in more than five proofs has an internal checker. Of the 15 rules without an internal checker, only consequence and add_hypothesis occur in more than four proofs. The others occur in at most four proofs, most of them in a single one. This long tail is where delegation is worth it: a checker for each of these rules would add code, and therefore trusted code, for a handful of steps, whereas the prover decides them with the same mechanism as consequence.

Table 19: Rules in the ProoVer set and how their steps were decided.

Rule	Proofs	Steps	Internal	ATP fallback	ATP only	Unresolved
modus_ponens	13	10012	10012	0	0	0
skolemize	47	5086	5086	0	0	0
consequence	95	96	0	0	96	0
instantiate	30	68	68	0	0	0
existential_gen	14	45	45	0	0	0
split_conjunct	7	33	33	0	0	0
duplicate	1	30	30	0	0	0
weaken	8	27	26	1	0	0
rename_variable	9	26	26	0	0	0
conjunction	11	24	24	0	0	0
excluded_middle	7	22	22	0	0	0
double_negation	7	17	17	0	0	0
commute	7	16	16	0	0	0
copy	12	12	12	0	0	0
reflexivity	7	11	11	0	0	0
add_hypothesis	5	11	0	0	11	0
remove_double_negation	5	9	9	0	0	0
resolution	5	8	7	0	1	0
horn	4	8	0	0	8	0
paramodulation	4	5	5	0	0	0
rewrite	1	4	4	0	0	0
instantiate_mp	2	3	3	0	0	0
assume	3	3	0	0	3	0
alpha	1	2	0	0	2	0
ex_falso	2	2	0	0	2	0
de_morgan	1	1	0	0	1	0
contrapositive	1	1	0	0	1	0
disjunctive_syllogism	1	1	0	0	1	0
transitivity	1	1	1	0	0	0
case_analysis	1	1	0	0	1	0
reassociate	1	1	0	0	1	0
over_generalize	1	1	0	0	1	0
tautology	1	1	0	0	1	0
simplify	1	1	0	0	1	0
project	1	1	0	0	1	0
Total (35 rules)	100	15590	15457	1	132	0

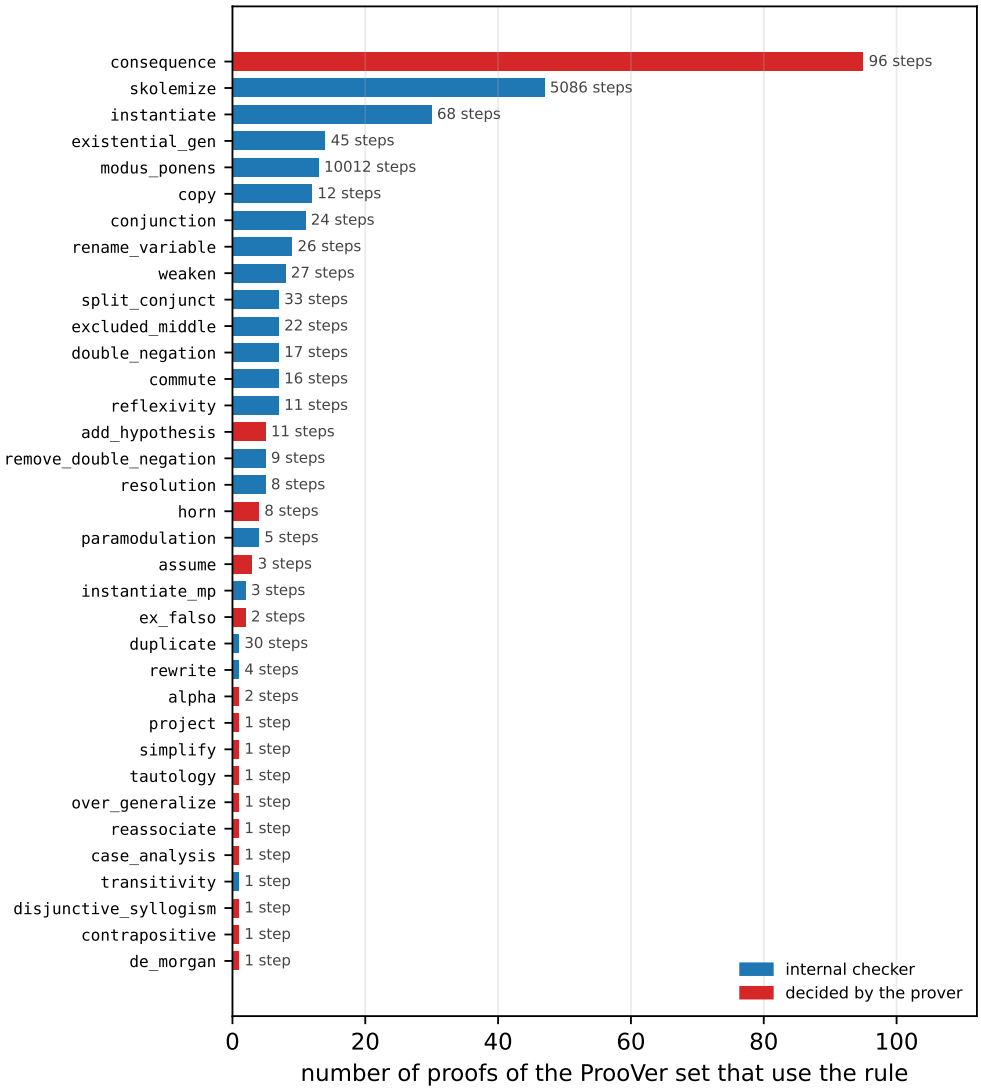


Figure 14: Inference rules of the ProoVer set by the number of proofs that use them. Blue rules have an internal checker, red rules are decided by the external prover. The labels give the number of steps.

The aggregate share is dominated by three proofs (`PRV071`–`PRV073`) with 10 005 steps each, which consist almost entirely of `modus_ponens` and `skolemize` steps. We therefore also give the median over proofs: in half of the proofs, at least 50 % of the steps are decided internally, and the remaining steps are almost always the final `consequence` step (it occurs in 95 proofs) and `add_hypothesis`. This is intended: `consequence` is a generic "follows from" step for which no syntactic criterion exists. Besides these, 13 further rules (among them `horn`, `de_morgan`, `contrapositive`, `disjunctive_syllogism`, and `over_generalize`) have no internal checker and are decided by the prover. Most of them occur in a single proof. In answer to RQ1, a verifier that covers `skolemize`, `instantiate`, `existential_gen`, `modus_ponens`, `split_conjunct`, `conjunction`, and the structural rules checks the bulk of the benchmark itself, and delegation to a prover handles

Table 20: Size and statement coverage of the modules (measured with `pytest-cov`).

Module	Lines	Statements	Coverage
`skolem_checker.py`	619	266	82.7 %
`clausal_checker.py`	500	310	84.5 %
`formula_utils.py`	270	144	81.2 %
`negated_conjecture_checker.py`	263	114	90.4 %
`external_atp_checker.py`	197	119	84.9 %
`junction_checker.py`	171	76	92.1 %
`propositional_checker.py`	161	72	87.5 %
`alpha_eq.py`	155	85	97.6 %
`instantiate_checker.py`	129	67	88.1 %
`dependency_checker.py`	125	61	100.0 %
`structural_checker.py`	98	53	94.3 %
`file_dir.py`	94	44	95.5 %
`existential_gen_checker.py`	74	36	86.1 %
Checker package	2856	1447	87.2 %
All of `src`	4251	2194	90.4 %

the long tail of rules.

7.5 Architecture Metrics (RQ2)

To assess whether the architecture keeps the trusted part small and extensible, we report two measures from the implementation and one from the tests. Table 20 gives the size of each checker module and its statement coverage by the unit tests. The checker package has 2856 lines, ranging from 74 to 619 lines per module (the largest is `skolem_checker`). The driver `main.py` adds 1231 lines. The tests cover 87.2 % of the statements of the checker package and 90.4 % of the whole `src` directory. The share of steps that are decided without the prover (Section 7.4) is the second measure: it determines how much of a verdict depends on the prover.

Modularity matters in practice because each module can be tested alone: the *rule-level* tests of Section 7.3 call the checkers directly with a few formulas, without parsing a proof or starting a prover. We have not measured the effort to add a new rule, because the driver dispatches by a chain of conditions and not by the registry described in the design (Section 6.4). Adding a rule therefore means one new function and one new branch in the driver.

7.6 Run Time and Memory (RQ3)

Figure 15 plots the wall-clock time of each proof of the ProoVer set against its number of steps. The median time per proof is 1.48 s, the 90th percentile 1.85 s, and the maximum 10.92 s. Only 1 proof needs more than 10 s: `PRV099`, whose time goes into a single prover call. The slowest of the three proofs with 10 005 steps needs 6.9 s. The peak memory of the worker process is 59 MB at the median and 173 MB at the maximum. Of the median run time of 1.48 s, about 1.05 s are start-up of the interpreter and the import of the verifier, which is slow under WSL 2 because the files are read

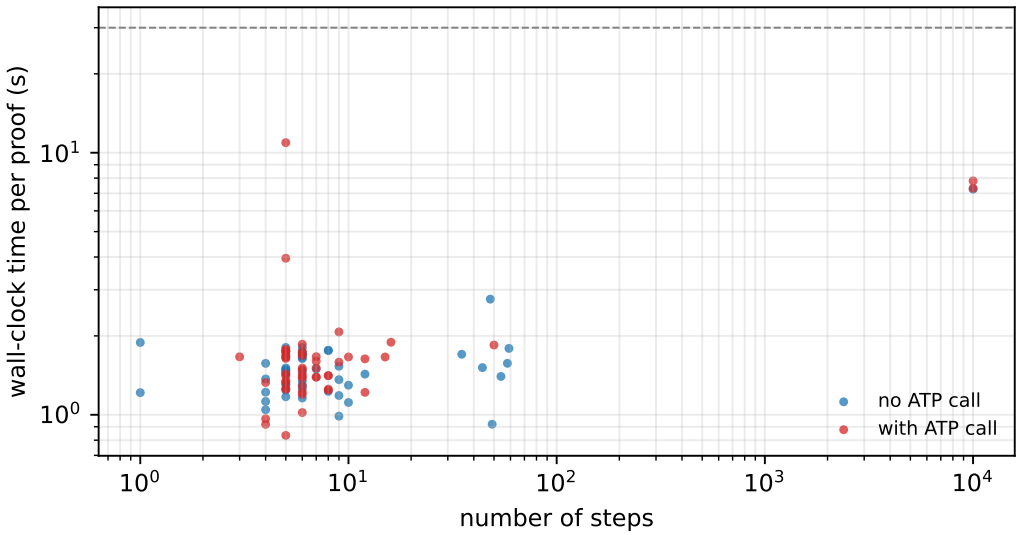


Figure 15: Wall-clock time per proof of the ProoVer set against the number of steps. Proofs in which the external prover was called are drawn in red. The dashed line is the time limit.

Table 21: The six slowest proofs of the ProoVer set (median of the runs). Times in the columns are wall-clock times of the phases, "MB" is the peak resident memory.

Proof	Steps	ATP calls	Start-up	Parsing	ATP	Rule checks	Total	MB
`PRV099+1.s`	5	2	1.24 s	100 ms	9.15 s	429 ms	10.92 s	173
`PRV073+1.s`	10005	1	1.16 s	3.38 s	82 ms	3.18 s	7.81 s	96
`PRV071+1.s`	10005	1	1.10 s	2.55 s	94 ms	3.58 s	7.32 s	96
`PRV072+1.s`	10005	0	1.21 s	2.67 s	0.0 ms	3.38 s	7.26 s	100
`PRV081+1.s`	5	1	5.65 s	350 ms	498 ms	0.0 ms	3.96 s	59
`PRV012+1.s`	48	0	2.71 s	77 ms	0.0 ms	0.0 ms	2.76 s	26

from the Windows file system. The checks themselves are therefore cheap for the typical proof.

The external prover is not the bottleneck for the typical proof. The 61 prover calls in the ProoVer set take 239 ms on average and 82 ms at the median, and altogether 8.2 % of the total time. This contradicts the assumption we had at the beginning of the work, based on preliminary measurements of single proofs, that the prover would dominate the run time. The mean is, however, determined by a few outliers: `PRV099` alone spends 8.5 s in its prover call, and no other call takes longer than 0.3 s. Table 21 shows where the time goes for the slowest proofs. `PRV099` is the only proof that is dominated by the prover. The step `hard` is a resolution step that the internal check does not decide, and Vampire confirms it (`ContradictoryAxioms`) after 8.5 s. The three large proofs spend about 2.5 s in parsing and between 3.0 and 3.6 s in the rule checks, and less than 0.1 s in the prover. Parsing and the rule checks are therefore the first things to improve for very large proofs, and the prover is the first for hard single steps.

Table 22: Mutation operators and kill rates. “Survivors still valid” are mutants that the verifier accepts and that Vampire confirms to follow from the premises.

Mutation operator	Mutants	Killed	Survivors still valid	Survivors not valid	Kill rate (without valid)
Change connective ($\wedge \leftrightarrow \vee$)	21	19	2	0	100.0 %
Swap sides of an implication	21	18	3	0	100.0 %
Swap quantifier kind ($\forall \leftrightarrow \exists$)	20	19	1	0	100.0 %
Negate an atom	82	81	1	0	100.0 %
Remove a premise	71	69	2	0	100.0 %
Cite another premise	82	72	10	0	100.0 %
Change status `thm` to `cth`	82	37	45	0	100.0 %
Total	379	315	64	0	100.0 %

7.7 Robustness (RQ3)

Robustness is evaluated in three ways.

Mutation test. We generate variants of the valid proofs by applying one mutation each to a derived step and run the verifier on the variant (Table 22). The operators change a connective, swap the sides of an implication, swap a quantifier, negate an atom, delete a premise, cite a different earlier step as premise, or change the status `thm` to `cth`. Per proof and operator, up to two applicable steps were mutated, in 46 valid proofs with at most 80 steps. This gives 379 mutants. A mutant that the verifier accepts is not necessarily a miss, because many mutations preserve validity (swapping the operands of a symmetric connective, for example). We therefore check each surviving mutant with Vampire: if the mutated step still follows from its premises, it is counted as “still valid”, and otherwise as a real miss. Of the 379 mutants, 315 are rejected, 64 survive and are still valid, and 0 survive and are not valid. The kill rate on the mutants that are no longer valid is 100.0 %. Figure 16 shows the outcome per operator. The operators that change the formula itself (connective, implication, quantifier, atom) are killed almost completely, and their few survivors are mutations that preserve validity, such as swapping the operands of a symmetric connective or negating an atom of a tautology. Removing or redirecting a premise is also detected in most cases. The survivors of redirection are steps whose conclusion also follows from the other premise, typically because the proof contains two similar formulas. No bar has a part for accepted mutants that are no longer valid. The majority of the survivors (45 of 64) come from the status change: the verifier ignores the status `cth` of a step if the step is entailed by its premises, which is correct for the competition, where status is informational. Two limits apply. The oracle checks entailment but not the side conditions of rules, so it is blind to defects such as a dropped Skolem dependency. The mutation experiment therefore does not substitute for the TST group `sk`. In addition, the operators are simple, so the result shows that obvious local errors are not missed, not that all errors are detected.

Malformed input. Table 23 shows the outcome for ten kinds of broken input derived from `PRV010+1.s`. In no case does the verifier crash or hang. 3 inputs end in a parser error, which the driver reports as an error (and the competition scores like a rejection), and the others are rejected by the checks. One input is accepted: a file in which the rule name of one step was replaced by an unknown name. This is intended, because an unknown rule is decided by the prover from the

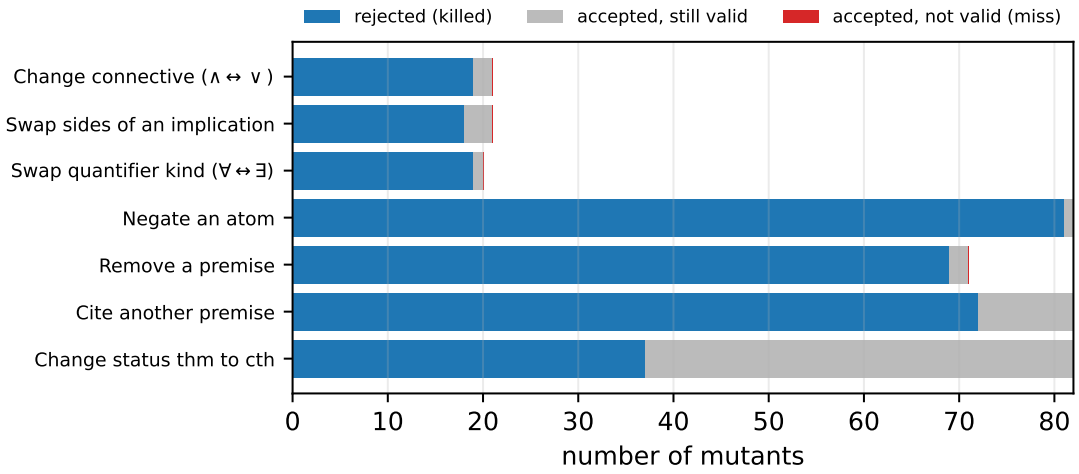


Figure 16: Outcome of the mutation test per operator: mutants rejected by the verifier, mutants accepted that still follow from their premises, and mutants accepted that do not (none).

Table 23: Outcome for malformed inputs. `error` means that the parser raised an error and the proof was not verified.

Input	Outcome	Time (s)
empty file	`VerifiedBad`	0.05
truncated in the middle of a formula	`error`	0.05
binary garbage	`error`	0.03
unknown inference rule	`VerifiedGood`	0.12
missing inference annotation	`VerifiedBad`	0.05
unknown status	`error`	0.02
duplicate step names	`VerifiedBad`	0.04
deeply nested negations (3000)	`VerifiedBad`	0.06
deeply nested terms (3000)	`VerifiedBad`	0.13
only comments	`VerifiedBad`	0.03

premises and the conclusion, and the step is in fact entailed. The two deeply nested inputs (3 000 levels) are handled because the verifier runs in a thread with a larger stack (Section 6.10).

Missing prover. We repeat the ProoVer run with a prover command that does not exist (Table 24). No invalid proof is accepted (0 wrong acceptances), which verifies the fail-closed policy empirically (NFR1). Only 22 of the 47 valid proofs are still accepted, because nearly every proof ends with a `consequence` step. The score under the binary policy falls from 153 to 103 points. Under the default policy, the 30 proofs with an undecidable step are unknown and the score is 118 points. This shows how strongly the verifier depends on the prover for acceptance. Rejections, in contrast, mostly do not depend on it: the internal checks alone find the error in a large part of the invalid proofs.

Table 24: ProoVer set with and without the external prover. The three-valued score counts steps for which no verdict could be reached as uncertain (zero points).

	valid verified	invalid verified	invalid rejected	Score (binary)	Score (three-valued)	uncertain
with prover	47	0	53	+153	+151	1
without prover	22	0	53	+103	+118	30

7.8 Ablations

Internal checkers against the prover. The design checks syntactically first and delegates only when necessary (Table 12). To see what the internal checkers add, we compare them with the prover step by step. From the valid and invalid proofs of the ProoVer set (proofs up to 400 steps), we sampled up to eight steps per proof, for rules with an internal checker, and asked Vampire whether the step follows from its premises (Table 25). Of the 155 steps, 146 are accepted by the checker and confirmed by the prover, so the two agree. In 0 cases the prover does not confirm a step that the checker accepts, and in all of them the answer is a timeout after 10 s: a `rename_variable` step in `PRV008`, a `conjunction` step in `PRV012`, and an `instantiate_mp` step in `PRV018`. The steps are trivial for the internal checker, but Vampire needs more than 10 s for them in this form. No step is refused by the prover with a counter-model. This is a second reason for the internal checkers: a verifier that relies on the prover alone would not verify these proofs within a short per-call limit. In two cases the checker rejects a step that the prover accepts (a `modus_ponens` step in `PRV023` and a `rewrite` step in `PRV024`, both in invalid proofs): the conclusion follows from the premises, but not by the rule that the step cites. The internal checkers are therefore stricter than entailment, which is intended, because the task is to verify the stated inference and not only that the conclusion is true. A prover alone could not make this distinction. The cost of a prover call is moderate in our data (median 68 ms), so run time is not the main argument for the internal checkers. The arguments are precision, independence from the prover, and robustness against steps that are hard for it.

We did not run a configuration in which *all* steps are sent to the prover, because Skolemization steps, which are not logical consequences, cannot be checked by entailment at all, and because the single-step comparison already answers the question of what the internal checks add.

Three-valued verdict. The verifier distinguishes the verdicts verified, failed, and unknown (Section 6.4). A proof is unknown only if its sole findings are steps that could not be decided (prover time-out or no answer, no checker for the rule). Every step that is refuted by a checker or by the prover makes the proof failed. The default policy gives 0 points for unknown, and the option `--binary-verdict` maps it to failed. To quantify the difference, we score both policies on the same runs (Table 24). On the ProoVer set with the prover, the score is 151 for the default policy and 153 for the binary policy. The default policy is lower by two points because one invalid proof (`PRV097`), whose step with the rule `trivial` is not decided, is an abstention instead of a correct rejection. Without the prover, the order is reversed: 118 points under the default policy against 103 under the binary policy, since most proofs are unknown (30 of 100), and a wrongly rejected valid proof costs one point each, whereas an abstention costs nothing. Abstaining is therefore worthwhile when a missing answer is likely for valid proofs, which is the case when the prover is unavailable or too slow, and it costs points when the undecided step is in an invalid proof, as with `PRV097`. With the prover available, the cost in our data is two points of 153. We keep the three-valued verdict as

Table 25: Single steps, internal verdict against Vampire. “Prover *no*” includes steps for which Vampire did not finish within 10 s (status `Timeout`).

		internal accepts		internal rejects		
Rule	Steps	prover *yes*	prover *no*	prover *yes*	prover *no*	unknown
`instantiate`	39	38	0	0	1	0
`existential_gen`	16	16	0	0	0	0
`modus_ponens`	13	11	0	1	1	0
`copy`	11	11	0	0	0	0
`conjunction`	10	9	0	0	0	1
`excluded_middle`	10	10	0	0	0	0
`resolution`	8	7	0	0	1	0
`duplicate`	8	8	0	0	0	0
`rename_variable`	7	6	0	0	0	1
`split_conjunct`	6	6	0	0	0	0
`paramodulation`	5	4	0	0	1	0
`weaken`	4	4	0	0	0	0
`rewrite`	4	3	0	1	0	0
`remove_double_negation`	3	3	0	0	0	0
`commute`	3	3	0	0	0	0
`instantiate_mp`	3	2	0	0	0	1
`reflexivity`	2	2	0	0	0	0
`double_negation`	2	2	0	0	0	0
`transitivity`	1	1	0	0	0	0
Total	155	146	0	2	4	3

the default because it is the more honest report (“this step is wrong” against “this step could not be checked”), it limits the loss if the prover behaves differently on the hidden benchmark, and it never turns an unverifiable step into an acceptance. The binary policy is available for a setting in which every invalid proof must be rejected.

7.9 Error Analysis

1 of the 100 proofs of the ProoVer set ends with a verdict that is not the best possible one (Table 26): the invalid proof `PRV097`, whose verdict is unknown instead of failed, as described in Section 7.2. The step `trivial` with status `esa` is a gap in the rule coverage and not an error of the checks. A rule that accepts the step would turn the unknown into a verdict, but we cannot decide from one example which semantics the organizers intend, and a guess could cause a wrong acceptance. In one case, however, the result depends on the speed of the prover. In an earlier run on a two-core virtual machine with Vampire 5.1.0, the proof `PRV099` exceeded the time limit of 30 s: the single call for the step `hard` used the whole remaining budget, and the proof counted as an abstention, which would have cost two points. On the machine of this chapter, with Vampire 5.0.1, the same call takes 8.5 s. Because a single prover call is allowed to use the whole time limit, one hard step can consume the budget of the proof. A shorter limit per prover call (for example 10 s) would let the

Table 26: Causes of suboptimal verdicts on the ProoVer set.

Cause	Proofs	Examples
Step not decidable (verdict Unknown)	1	PRV097

verifier reject such a proof in time instead of abandoning it, but would reject a valid proof if its step needed more time, as for PRV099 here. Which choice is better depends on the unknown hardware of the competition. The longest prover call outside PRV099 takes 0.3 s.

The parser errors (2 proofs, PRV057 and PRV091) are counted as correct, because the proofs are invalid and the verifier answers failed. In both cases the parse error is the intended defect and not a weakness of the grammar: in TPTP, a word that starts with an upper-case letter is a variable, so that a step name or a function symbol must start with a lower-case letter or be quoted. PRV057 cites a premise S0, which is a variable and not a step name (and no such step exists), and PRV091 introduces the Skolem symbol SK0, which is a variable and not a function symbol. Both proofs are therefore not well-formed TSTP, and rejecting them is the correct verdict for a reason that the specification gives. Accepting these names would not make a valid proof verifiable, but it would risk accepting the deliberately malformed ones, so the grammar stays strict. The verdict is, however, given by the parser and not by a check of the proof. The diagnostic "unexpected token" does not tell the user which defect was intended.

There is no wrong rejection of a valid proof in the ProoVer set. The accepted TST proof (Section 7.2), TST043, shows that the verifier does not enforce a granularity of steps, because each step is a valid consequence. Whether this is acceptable depends on the competition rules.

7.10 Threats to Validity

Internal validity. The verifier was developed against the ProoVer 2026 sample proofs, so the ProoVer set is not an independent test set. The TST and Extra sets were written by the author and reflect the author's understanding of the rules. The Extra set found three defects that the other sets did not, which shows that the score on the ProoVer set alone is not sufficient evidence of correctness. The expected results of the ProoVer set come from the organizers and are not verified independently. The competition itself leaves some edge cases open. We mitigate this by reporting the mutation experiments, which do not depend on the benchmark, and by reporting wrong acceptances separately. The mutation oracle is itself an entailment check by Vampire, so its verdicts on the survivors are only as reliable as the prover. A timeout of the oracle is counted as a miss, not as a valid mutant, so the oracle errs on the cautious side.

External validity. The benchmark contains FOF proofs from a fixed set of generators. The results do not transfer to proofs from other provers with different rule vocabularies or to other logics. The share of steps checked internally (99.1 %) is mainly a property of the benchmark generators, as a few proofs with many steps dominate it. The median per proof is 50 %. The scoring scheme and the 30 s limit are those of one competition.

Construct validity. The correctness of the accepted proofs relies on the correctness of the rule checkers, which is supported by tests but not proved, and on the soundness of Vampire. Run times

depend on the hardware and on WSL 2, where start-up is slow. The relations, namely that the prover takes a small part of the time for typical proofs while parsing and rule checks dominate for the large proofs, are less sensitive to this than the absolute values. The verdict of the prover can differ between versions and machines when a step is close to the time limit, as the example of `PRV099` shows. The evaluation uses Vampire 5.0.1 (Release build, commit 1b13eaf on 2026-01-18 12:14:50 +0000).

7.11 Summary

On the 91 proofs of the benchmark, the verifier reaches 136 of 138 points under the default three-valued policy (151 of 153 including the deprecated proofs, 138 and 153 with the binary policy), without a wrong acceptance (RQ1: 35 rules occur, 99.1 % of the derived steps are checked without the prover, and the rest are mostly the final `consequence` step). The architecture keeps the checkers small and separately testable (RQ2): 2856 lines in the checker package, 87.2 % statement coverage, and a rule-level test suite of 109 proofs, on which the verifier has no wrong verdict. The median time per proof is 1.48 s and the peak memory 173 MB at most. The prover is called 61 times and takes 8.2 % of the time (RQ3). Without the prover, no invalid proof is accepted, but almost no valid proof is verified. Against mutations of valid proofs, every mutant that is no longer valid is rejected. The open points are the sensitivity of `PRV099` to the speed of the prover and the rule `trivial` with status `esa`, which makes `PRV097` unknown.

8 Discussion

This chapter interprets the results of Chapter 7, discusses what the verifier does and does not guarantee, and lists its limitations.

8.1 Interpretation of the Results

The verifier reaches a score of 136 of 138 points on the 91 proofs of the benchmark (151 of 153 including the deprecated proofs) and does not accept any invalid proof. The second property matters more than the first: with a penalty of -10 for a wrong acceptance, a single such error costs as much as ten correct acceptances. The verdict matrix shows that the only loss on this benchmark is the two points for the invalid proof `PRV097`, for which the verifier answers unknown. This should not be read as a statement that the verifier is complete: the benchmark was available during the development, and the proofs that the verifier rejects do so for reasons that it was built to detect. The Extra set of Section 7.3 found defects that the benchmark could not, and on 6 of the 8 generated edge cases the verifier is more cautious than necessary (Section 7.3). If the verifier loses points on unseen proofs, the fail-closed policy of Section 4.2 makes it lose them in the cheap direction.

The ablation of the verdict rule (Section 7.8) shows the price of this policy. The default three-valued verdict, in which a proof that fails only because a step cannot be decided is an abstention, scores 151 points instead of the 153 points of the binary policy (`--binary-verdict`), because one invalid proof (`PRV097`), whose step with an unknown rule cannot be decided, is an abstention instead of a correct rejection. Without the prover, the three-valued rule is clearly better (118 against 103 points), because most valid proofs become undecidable and would otherwise be wrongly rejected.

The default therefore costs two points on this benchmark and protects against a much larger loss if the prover fails on the competition machine. The benefit of abstaining therefore depends on how often a missing answer occurs for valid proofs compared with invalid proofs, and on the availability of the prover.

The rule coverage (Section 7.4) shows that 99.1 % of the derived steps are decided by internal checks, and only 132 steps (the final `consequence` step and `add_hypothesis`, mostly) need the external prover. The run times (Section 7.6) are nevertheless not dominated by the prover: its calls take 8.2 % of the total time, with a mean of 239 ms, while the start-up of the interpreter alone takes 1.05 s on the test machine. The one visible exception is `PRV099`, which shows that a single hard prover call can matter more than all other checks together. The order "cheap checks first" is therefore mainly a matter of precision and independence from the prover, and has little effect on the run time. It is also the reason why steps that Vampire does not confirm within 10 s but that are trivial for the internal checker do not endanger the verdict (Section 7.8).

8.2 Scoring Policy and Its Limits

The policy of Section 4.2 follows from the scoring scheme, but it rests on assumptions that deserve a comment. The probability p that a proof is valid is not observable, so the thresholds $2/3$ and $10/11$ cannot be applied literally. The verifier approximates them by the discrete rule that acceptance requires positive evidence for every step and rejection requires a concrete defect. This is conservative on both sides. It would be possible to accept proofs with a small unresolved remainder if the remainder is judged harmless, but this requires a calibrated estimate of p that the verifier does not have. The policy is also tied to this competition: with a different penalty the optimal behavior changes, and in a setting where a wrong acceptance is not much worse than a wrong rejection, abstention would be less attractive. Finally, the policy trades points for safety when the external prover is missing or too slow: invalid proofs whose defect is only visible to the prover become *uncertain* instead of being rejected, and the verifier forgoes the $+2$ for them.

8.3 Trusted Computing Base and Soundness

The soundness argument of Section 5.10 is conditional: it holds if the rule checkers are correct and the external prover is sound. In terms of the trusted computing base (Section 2.1), the following components therefore have to be trusted for the property that every accepted proof is valid:

- The *parser and normalization*, since a parse error that changes the meaning of a formula would change what is checked.
- The *alpha-equivalence* check, which also merges quantifier prefixes and flattens junctions before it compares, and which therefore equates formulas that are not literally identical.
- The *rule checkers*, in particular the Skolem checker, the largest module, and the clausal checker.
- The *serialization of formulas to TPTP* for the external prover, because a defect there would make the prover validate a different formula than the one in the proof.
- The *external prover* and its interpretation through the SZS status line.

- The *verdict aggregation*, which decides how the individual results are combined.

Compared with a checker that sends every step to a prover, this list is longer, because the internal checks are part of it. Compared with the prover that produced the proof, it is much shorter. Measured against the de Bruijn criterion (Section 2.1), the verifier is a partial success. Its own trusted code, about 2856 lines of checkers plus the parser and the shared utilities, is small enough to be read and tested rule by rule, but it is an order of magnitude larger than the kernel of an LCF-style system, and every proof whose last step is a `consequence` step also depends on Vampire, a full saturation prover. For such a proof the trusted part is not smaller than the system that could have produced the proof, which is exactly what the criterion wants to avoid. The competition rules accept this compromise, since they explicitly allow external provers for unspecified rules. A stricter verifier would have to replace the prover calls by certificates that a kernel checks, as GDV-LP does with LambdaPi (Sutcliffe et al. 2025). This is the trade-off stated in Section 3.7: the thesis exchanges a larger trusted checker for less dependence on the prover and for better diagnostics. The evaluation supports the correctness of the checkers by tests, by the mutation experiment, and by the benchmark, but it does not prove it. A way to reduce the trusted part further would be to restrict the internal checks to the rules where they are simple (structural, propositional, and the Skolem conditions) and to delegate the rest, or to validate the checks themselves against a proof assistant.

8.4 Design Trade-offs

Internal checks and ATP fallback. The competition rules allow any step except Skolemization and the import steps to be checked by an external prover. A single step that derives `$false` directly from the axioms and the negated conjecture is then delegated completely (the test group `st08` contains such a “monolithic” step). For such proofs the verifier is exactly as strong as the prover, and the internal checks contribute only the global conditions. This is correct with respect to the rules, but it shows that the quality of the verifier on such inputs lies in the global checks, not in the step checks.

Modularity. Rule-local checkers made it possible to test each rule in isolation and to attribute every error to a step and a reason. The price is that some knowledge, such as the freshness of Skolem symbols, cannot be local to one step and lives in the driver. The size of the dispatcher in `main.py` is a sign that the modular idea has been applied to the checkers but not yet to the dispatch. The registry of Section 5.5 addresses this.

Python. The implementation language favors readability and fast development, which was valuable for a thesis with many rules. It costs run time for the internal checks and complicates deep recursion, which needed a dedicated thread with a larger stack. The measurements show that Python is fast enough for this task: the prover takes only 8.2 % of the total time, and the pure Python cost matters only for the very large proofs, where parsing takes about 2.5 s and the rule checks between 3.0 and 3.6 s (Section 7.6).

Process isolation. Running each proof in its own process makes timeouts and crashes harmless and enforces the time limit, but it adds a startup cost per proof and makes sharing state, such as a cache of ATP results, harder.

Table 27: Verifier on unmodified Vampire proofs (142 proofs).

Outcome	Proofs
Rejected by the parser (`introduced(...)` annotations)	66
Parsed, verdict *failed*	75
Parsed, timeout (60 s)	1
Parsed, verdict *verified*	0

8.5 Transfer to Other Provers: A Case Study with Vampire

The verifier was designed for the ProoVer 2026 setting, in which the rule vocabulary, the proof style, and the kinds of defects in invalid proofs are fixed by the organizers. This subsection asks how much of the design survives when the proofs do not come from the competition but from an automated prover. We use Vampire (Riazanov & Voronkov 2002; Kovács & Voronkov 2013) for two small experiments and then discuss why a verifier that performs well on the competition set is unlikely to transfer to the output of other provers without further work.

Setup. Vampire 5.0.1 was run in CASC mode with a limit of 30 s on all 176 problem files of the benchmark (script `scripts/run_vampire_all.sh`). It reports *Theorem* for 139 problems, *CounterSatisfiable* for 19, *ContradictoryAxioms* for 3, and times out on 15 (13 of them in the TST group). The 142 problems with a proof (Theorem and ContradictoryAxioms) are the input of Experiment A. The 19 CounterSatisfiable outputs contain a saturation instead of a proof. They correspond to problems whose conjecture does not follow from the axioms and are not used.

Experiment A: Can the verifier read Vampire's proofs? Table 27 summarizes what happens when the 142 proofs are given to the verifier unchanged. No proof is verified. This is not surprising, but the reasons differ and are worth looking at:

- **Syntax.** Vampire records Skolem symbols and definitions as separate formulas with the source `introduced(definition,...)` and refers to input formulas as `file('Problems/X.p',unknown)`. The grammar and the provenance check of the verifier assume the competition conventions (66 files cannot be parsed, and the provenance check fails for almost all others). Part of the parse failures were caused by a defect in the grammar rule for `introduced(...)`, corrected after this experiment (Section 6.15). Since the formulas introduced in this way cannot be justified by the verifier, the correction would turn these rejections into abstentions, not into acceptances. This part is engineering and could be removed by a normalizing front end.

- **Vocabulary.** Vampire's 142 proofs use 22 distinct inference names. The competition corpus uses 39. Only two names occur in both (`resolution` and `negated_conjecture`), and even for them the meaning differs: a Vampire `resolution` step works on clauses, a ProoVer `resolution` step on the formulas of the competition. 22 153 inferences were counted in total. The 24.2 % that carry a name known to the verifier are almost all `resolution` steps, and no single proof consists only of such steps. Most steps belong to the preprocessing and splitting

Table 28: Internal Skolem check against an entailment oracle ($C \models P$ with Vampire). Numbers are mutants rejected / accepted.

Step kind	Steps	Skolem check (rej./acc.)	Oracle (rej./acc.)
unchanged (control)	116	0 / 116	1 / 115
drop_arg: missing dependency	25	25 / 0	0 / 25
universal: universal variable eliminated	56	56 / 0	26 / 30
negate_atom: wrong formula	108	108 / 0	106 / 2
drop_conjunct: wrong formula	11	11 / 0	5 / 6
flawed steps of the reference proofs	35	35 / 0	11 / 22 (+2 ill-formed)

machinery of the prover (`ennf_transformation`, `cnf_transformation`, `avatar_*`, `sat_conversion`, `forward_subsumption_resolution`).

- **Status information.** In the competition format nearly every step carries `thm`, `esa`, or `cth`, and the verifier uses this to decide what may be delegated to a prover (Section 5.9). In Vampire's output only `negated_conjecture` and `skolemisation` carry a status. The other steps have none, so under the fail-closed policy none of them may be delegated.

The proofs are not large on this benchmark (median 11 formulas against 6 in the reference proofs, maximum 20 007 against 10 006), because the ProoVer problems are small. For harder problems the size of a prover's proof is not bounded by what a human or the organizers would write down, which matters for the run-time results of Section 7.6.

Experiment B: Skolemization and an entailment oracle. The previous experiment mostly measures format differences. To separate this from the question of whether the *checks* would carry over, we isolate the one rule that cannot be checked by entailment: Skolemization (Section 2.4). We take the Skolemization steps of the 176 reference proofs. There are 5 141 such steps, and the internal check accepts 5 106 of them. Since 5 000 of them belong to the single, very large proof PRV072, we use at most ten steps per proof and mutation kind. From these steps we derive mutants with a known defect and compare two verdicts for each: (i) the internal Skolem check, and (ii) an *entailment oracle* that asks Vampire whether the conclusion C entails the parent P. This is the only entailment that holds for a correct step and it is what a verifier built around a generic prover would test. The mutants are *drop_arg* (a required dependency is removed from the Skolem term, which makes the step unsound), *universal* (a universally quantified variable is "Skolemized"), *negate_atom* (an atom of C is negated), and *drop_conjunct* (a conjunct of C is removed). The unchanged steps serve as a control. Table 28 shows the expected pattern. Defects that change the *formula* (*negate_atom*) are found by both. The two mutants that the oracle accepts negate an atom in a tautology ($a = a$) and are harmless. The defect that is specific to Skolemization, a missing dependency, is not found by the oracle in any of the 25 cases: a Skolem term with too few arguments makes C stronger, so $C \models P$ still holds, but the step is not equisatisfiable. Of the 30 *universal* mutants accepted by the oracle, 27 eliminate a variable that does not occur in the body, which is semantically harmless. The internal check rejects them because the rule does not allow it. In the measured version, the oracle also rejects one unchanged step that the internal check accepts (`TST064`, step `s1`). The corrected check rejects it as well. The step Skolemizes an existential quantifier in the antecedent of an implication,

i.e. at a negative position, and the internal check of the measured version did not flip the polarity for implications. This defect is corrected in the final version (Section 6.15), which rejects the step. The 35 Skolemization steps that the internal check of the measured version rejects in the reference proofs (17 of them in the PRV set, where all affected proofs are labeled incorrect) fall into the same classes: the oracle recognizes 11, accepts 22, and cannot read two (a free variable in the Skolem term, rejected by Vampire's parser). The 22 accepted steps are defects such as a wrong number of introduced symbols, a mismatching annotation, `thm` instead of `esa`, or a wrong argument list. None of them is visible to an entailment test.

Why a good competition verifier transfers poorly. The two experiments point to four reasons.

1. *The calculus is part of the benchmark.* The ProoVer proofs use a small set of local rules in the style of natural deduction (`modus_ponens`, `instantiate`, `split_conjunct`, ...) that can be matched syntactically or discharged by a single small prover call. The verifier is effectively a collection of matchers for exactly these rules. A prover's proof is a refutation over clauses obtained after normalization and splitting. Its steps are not related to the above rules by name or by shape, so each rule needs a new checker or a translation layer.

2. *The invalid proofs are drawn from a known fault model.* The defects in the competition set (wrong Skolem term, wrong status, missing negation of the conjecture, circular dependency, wrong instance) are the ones the checkers target. This explains the high score. Prover proofs would be wrong for other reasons, for example an unsound inference implementation, an incorrect definition introduced during clausification, or a wrong splitting decision, and those are not covered.

3. *Entailment is not enough.* Experiment B shows that rules which preserve only satisfiability (Skolemization, and also the definition-introducing clausification and AVATAR steps of modern saturation provers) cannot be validated by asking a prover for entailment. A verifier that works well because most competition steps are `thm` steps loses this advantage on output where a large share of the steps is `esa`-like or has no status at all.

4. *The trusted base becomes circular.* The verifier delegates `thm` steps to Vampire. For Vampire's own proofs this means that the correctness of the prover is partly assumed in order to check the prover, which weakens the independence that the architecture of Section 8.3 is meant to provide. Certificate-based approaches (Wetzler et al. 2014; Andreotti et al. 2023) avoid this by making the prover emit a proof in a calculus small enough to be checked without a second prover.

Consequences. A direct extension of the verifier to Vampire or to another saturation prover would need (a) a normalizing front end for the output syntax, (b) native checkers for the clausification and splitting rules, which are the bulk of the steps, and (c) an argument for how satisfiability-preserving steps are validated, for example through the Skolem and definition conditions used here rather than through entailment. A cheaper way to obtain a verifiable proof is to request a proof in a restricted calculus from the prover, or to reconstruct the proof from Vampire's output with the external prover as the oracle for small, `thm`-like steps. This is the approach followed by proof-reconstruction tools such as Sledgehammer (Böhme & Nipkow 2010). Other provers, for example E (Schulz et al. 2019),

emit their own rule vocabularies. The observations above should apply to them in a similar way, but we did not run them.

Threats to validity. The mutants are synthetic, and their ground truth follows from the construction, not from an independent oracle. The mutation kind *universal* in particular contains semantically harmless cases, as noted above. The *drop_arg* class is small (25 steps), because only few reference Skolem terms have an argument that is actually used. Only one entailment test was used as oracle. A model finder or a stronger equisatisfiability argument might detect more defects. The failures in Table 27 partly stem from provenance conventions and syntax and would shrink with a normalizing front end. Only the vocabulary and status differences are fundamental. Finally, all results are for one prover version and one benchmark whose problems are small.

8.6 Limitations

- **Logic and format.** The verifier handles FOF problems and TSTP proofs with the rule vocabulary of the ProoVer 2026 benchmarks. CNF, typed, and higher-order input are out of scope, and rules outside the supported set are delegated to the prover or lead to abstention.
- **Weak rule checks.** Because the demodulating equation of a `rewrite` step is missing in the benchmark format, the check accepts only a conclusion that equals its parent up to renaming. This is sound but incomplete: a genuine rewrite that changes the formula would be rejected, which costs points on a valid proof but never accepts an invalid one.
- **Symbols from instantiation.** Annotations of Herbrand symbols are parsed but not checked separately. This is sound for `thm` steps, since instantiation with an arbitrary term is entailed by the parent, but it means that the annotation carries no checked information.
- **Rules without a semantics.** The rule `trivial` with status `esa` (`PRV097`) has no checker and cannot be delegated, so the proof is *unknown* instead of rejected. The same holds for any other unknown rule with a status other than `thm` or `cth`.
- **Sequential prover calls.** Each unresolved step starts its own prover process, one after another. Proofs with many such steps are therefore slow. Parallel calls and a shared time budget would help.
- **Benchmark.** The ProoVer set contains 100 proofs, the TST set 76 proofs written by the author, and the Extra set 109 generated proofs. This is enough to find defects, but not to estimate rare failure rates, and the results do not carry over to proofs from other provers (Section 7.10).
- **Trust in tests.** Correctness of the checkers is supported by tests and mutations, not proved.

9 Conclusion and Future Work

9.1 Summary

This thesis designed and implemented a Python-based verifier for TSTP derivations for the ProoVer 2026 competition. The starting point was the observation that the competition's scoring scheme is highly asymmetric: a wrongly accepted proof costs ten points, a correct acceptance gains one. From the expected score of the three possible answers, we derived a fail-closed policy with a three-valued verdict: the verifier accepts only if every step is validated positively, rejects only if it has found a concrete defect, and abstains otherwise. The verifier realizes this policy as a modular pipeline: a parser, global checks of provenance, dependencies, and the negated conjecture, rule-local checkers for structural, propositional, quantifier, and clausal rules, a dedicated check for Skolemization including the freshness of the introduced symbols, and a restricted fallback to an external prover for the remaining steps. A soundness argument shows that acceptance implies unsatisfiability of the axioms together with the negated conjecture, under the assumption that the rule checkers are correct and the prover is sound.

9.2 Answers to the Research Questions

RQ1: Which rules and fragments must be supported? The ProoVer set uses 35 distinct inference rules. Of these, 20 are checked internally, which covers 99.1 % of all 15590 derived steps (the share is dominated by three very long proofs; the median over proofs is 50 %); the remaining 15 rules, among them `consequence`, are delegated to the prover. Skolemization is the one rule that has to be checked internally, since it preserves only satisfiability and cannot be validated by entailment, and the checks for quantifier scope, polarity, and freshness are needed to reject the subtle errors in the invalid proofs.

RQ2: How does modularity keep the trusted part small and extensible? Every rule check is a function of a step and its parents and returns structured issues. The checkers can be tested in isolation, which is reflected in a test suite of 293 tests with a statement coverage of 87.2 % for the checker package, and an error can be attributed to a step and a reason. The trusted part consists of the parser, the alpha-equivalence check, the rule checkers, the serialization to the prover, and the verdict aggregation, in total about 4 500 lines (2856 of them in the checker package), plus the external prover as an assumption (Section 8.3). The dispatcher is still a chain of conditions, so that a new rule needs one function and one branch in the driver; the registry of the design remains future work (Section 6.16).

RQ3: How do correctness, run time, and memory behave? On the 91 proofs of the benchmark, the verifier reaches 136 of 138 points under the default three-valued verdict (138 with the binary option; 151 of 153 including the 9 deprecated proofs) with 0 wrong acceptances, a median run time of 1.48 s and a maximum of 10.92 s; 0 proofs hit the time limit, and the peak memory is 173 MB. The mutation experiment rejects all 315 mutants that are no longer valid (of 379; the 64 survivors are still valid), and with the prover removed the verifier still makes no wrong acceptance, which confirms the fail-closed property empirically. For typical proofs the run time is dominated by the

start-up of the interpreter, for the three very long proofs by parsing and rule checks, and the prover takes 8.2 % of the time; the exception is `PRV099`, whose single prover call takes about 8.5 s.

9.3 Contributions Revisited

The thesis contributes (i) a verdict policy for proof checking derived from the scoring scheme, together with a soundness argument that accounts for the different semantics of `thm` and `esa` steps; (ii) a modular architecture and its implementation with a dedicated Skolem check and a restricted ATP fallback; and (iii) an evaluation on the competition benchmark and on a suite of mutated and hand-written proofs that measures correctness, coverage, run time, and robustness, with an analysis of the remaining limitations.

9.4 Future Work

Several directions follow from the limitations of Section 8.6.

- **Registry-based dispatch.** Replace the `if`/`elif` dispatcher by the registry of the design, so that adding a rule needs one entry and one function.
- **Stronger checks for weak rules.** Reconstruct the missing demodulating equation of `rewrite` steps (a plain entailment test is not enough, because it would accept steps that follow from the parent but not by the cited rule), clarify the semantics of the rule `trivial` with status `esa` with the organizers, and add checks for Herbrand symbols once their meaning is fixed by the specification.
- **Counter-model search.** Use a model finder to reject steps that are not entailed, as Nörgler does, which would turn more unresolved steps into defects and thereby recover the $+2$ for invalid proofs.
- **Parallel and budgeted prover calls.** Run the prover calls for independent steps in parallel under a shared time budget, and cache results across steps.
- **Smaller trusted base.** Move closer to the de Bruijn criterion: reduce the rule checkers to a small LCF-style kernel of primitive operations, validate the internal checks themselves, for example by formalizing the Skolem conditions in a proof assistant, or replace prover calls by certificates that a small kernel can check, following the approaches discussed in Chapter 3.
- **Broader input.** Support CNF proofs and further rule vocabularies, and compare the verifier with GDV and Nörgler on a common benchmark.

A Reproduction

This appendix describes how to reproduce the results of the thesis. The source code, the proofs, and the scripts are in the repository of the thesis; the commit used for the measurements is f74ab9c.

Table 29: Command-line options of the verifier.

Option	Meaning
`-p`, `--problem`	Problem file; by default `Problems/<name>.p` next to the proof.
`-t`, `--timeout`	Wall-clock limit per proof in seconds (default 30).
`-d`, `--dir`	Batch mode: directory with proofs; the expected results are chosen from the directory name (`-e` overrides).
`--prefix`	Batch mode: only check proofs whose file name starts with the prefix.
`--binary-verdict`	Print `VerifiedBad` instead of `Unknown` when a step cannot be decided.

Table 30: Statuses printed by the verifier.

Status	Meaning
`VerifiedGood`	Every check succeeded.
`VerifiedBad`	A definite defect was found, or the proof could not be processed.
`Unknown`	No defect was found, but a step could not be decided (prover without result, or no checker for the rule).
`Timeout`	The time limit was exceeded.

A.1 Requirements

The verifier needs Python 3.11 or newer (the measurements used Python 3.12.3) and the library `lark`. The external prover is Vampire (the measurements used Vampire 5.0.1 (Release build, commit 1b13eaf on 2026-01-18 12:14:50 +0000)). Vampire is called through the environment variables `EXTERNAL_ATP_BINARY` (path of the binary) or `EXTERNAL_ATP_COMMAND` (complete command), and is found in the search path by default. Under Windows, the verifier is run in WSL.

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt -r requirements-dev.txt
export EXTERNAL_ATP_BINARY=/usr/local/bin/vampire
```

A.2 Checking Proofs

A single proof is checked by passing the proof file; the problem file is looked up in the subdirectory `Problems` next to the proof, or given with `-p`. Without a proof argument, the verifier checks all proofs of a directory (default `ProoVer2026/PRV`; further sets are `ProoVer2026/Test`, `ProoVer2026/Extra`, and `ProoVer2026/Deprecated`) and prints a summary with the score.

```
python main.py ProoVer2026/PRV/PRV010+1.s
python main.py proof.s -p problem.p -t 30
python main.py -d ProoVer2026/Extra
python main.py --binary-verdict ProoVer2026/PRV/PRV010+1.s
```

The last line of the output is the SZS status. Table 30 lists the statuses.

Table 31: Parts of the evaluation driver `scripts/eval_all.py`.

Part	Content
`runs`	Verdicts, steps, prover calls, time, and memory per proof (Sections 7.2 and 7.6).
`noatp`	The same without prover (ablation).
`internal`	Internal checkers against Vampire on single steps.
`mutation`	Mutations of accepted proofs with entailment check of the survivors.
`inputs`	Malformed inputs (robustness).
`extra`	Generated rule test suite.
`coverage`	Test coverage and lines of code per module.
`tables`	Regenerates tables, macros, and the figures from the raw data.

A.3 Tests and Evaluation

The test suite is run with `pytest`; the coverage figures of Section 7.5 come from the same run. The evaluation driver runs the verifier in a fresh process for each proof and writes raw data to `results/eval` and the tables and macros of Chapter 7 to `thesis/generated`.

```
pytest
python scripts/eval_all.py --parts all --reps 3
python scripts/gen_appendix_tables.py
```

The parts of the driver can be run separately with `--parts`; Table 31 lists them. The run time depends on the machine; the full run needs the prover for all parts except `noatp`.

B Example Reports

The report lists one block per check, and ends with the SZS status. The following output is the (shortened) report for the invalid proof `PRV002+1`, in which the negated conjecture is not the negation of the conjecture; the blocks without finding are omitted and the formulas are abbreviated.

```
Checking proof: ProoVer2026/PRV/PRV002+1.s
Against problem: ProoVer2026/PRV/Problems/PRV002+1.p

[FAIL] Negated Conjecture Issues:
  - neg: formula is not the correct negation of parent
    conjecture; expected one of: [...], got: [...]

% SZS status VerifiedBad
```

C Results per Proof

Tables 32, 33, and 34 list the status and the run time of each proof of the three benchmark groups. The time is the median over the repetitions of the whole process, including the start-up of the

interpreter. The column “Result” compares the status with the expected result; “abstain” denotes `Unknown` or `Timeout`, which score zero.

Table 32: Results for each proof of the ProoVer set (official proofs and samples).

Proof	Expected	Status	Result	Time (s)	ATP calls
PRV000+1	correct	VerifiedGood	ok	1.33	1
PRV001+1	correct	VerifiedGood	ok	1.12	0
PRV002+1	incorrect	VerifiedBad	ok	0.97	1
PRV003+1	incorrect	VerifiedBad	ok	0.92	1
PRV004+1	correct	VerifiedGood	ok	0.83	1
PRV005+1	correct	VerifiedGood	ok	1.70	1
PRV006+1	incorrect	VerifiedBad	ok	1.66	1
PRV007+1	incorrect	VerifiedBad	ok	0.99	0
PRV008+1	incorrect	VerifiedBad	ok	1.11	0
PRV009+1	incorrect	VerifiedBad	ok	1.02	1
PRV010+1	correct	VerifiedGood	ok	0.92	0
PRV011+1	correct	VerifiedGood	ok	1.40	0
PRV012+1	correct	VerifiedGood	ok	2.76	0
PRV013+1	correct	VerifiedGood	ok	1.79	0
PRV014+1	correct	VerifiedGood	ok	1.85	1
PRV015+1	correct	VerifiedGood	ok	1.51	0
PRV016+1	correct	VerifiedGood	ok	1.57	0
PRV017+1	correct	VerifiedGood	ok	1.63	2
PRV018+1	correct	VerifiedGood	ok	1.43	0
PRV019+1	correct	VerifiedGood	ok	1.59	1
PRV020+1	correct	VerifiedGood	ok	1.53	0
PRV021+1	correct	VerifiedGood	ok	1.76	0
PRV022+1	correct	VerifiedGood	ok	2.07	2
PRV023+1	incorrect	VerifiedBad	ok	1.81	0
PRV024+1	incorrect	VerifiedBad	ok	1.76	0
PRV025+1	incorrect	VerifiedBad	ok	1.46	0
PRV026+1	incorrect	VerifiedBad	ok	1.67	1
PRV027+1	correct	VerifiedGood	ok	1.68	0
PRV028+1	incorrect	VerifiedBad	ok	1.75	1
PRV029+1	correct	VerifiedGood	ok	1.50	0
PRV030+1	correct	VerifiedGood	ok	1.68	1
PRV031+1	incorrect	VerifiedBad	ok	1.43	1
PRV032+1	incorrect	VerifiedBad	ok	1.30	0
PRV033+1	correct	VerifiedGood	ok	1.23	0
PRV034+1	correct	VerifiedGood	ok	1.30	0
PRV035+1	incorrect	VerifiedBad	ok	1.24	0
PRV036+1	incorrect	VerifiedBad	ok	1.74	0
PRV037+1	incorrect	VerifiedBad	ok	1.37	0
PRV038+1	incorrect	VerifiedBad	ok	1.43	1
PRV039+1	incorrect	VerifiedBad	ok	1.05	0

Proof	Expected	Status	Result	Time (s)	ATP calls
PRV040+1	correct	VerifiedGood	ok	1.39	1
PRV041+1	correct	VerifiedGood	ok	1.26	0
PRV042+1	correct	VerifiedGood	ok	1.66	1
PRV043+1	correct	VerifiedGood	ok	1.75	1
PRV044+1	correct	VerifiedGood	ok	1.75	1
PRV045+1	correct	VerifiedGood	ok	1.49	0
PRV046+1	incorrect	VerifiedBad	ok	1.35	0
PRV047+1	incorrect	VerifiedBad	ok	1.31	0
PRV048+1	correct	VerifiedGood	ok	1.17	0
PRV049+1	incorrect	VerifiedBad	ok	1.47	0
PRV050+1	incorrect	VerifiedBad	ok	1.81	0
PRV051+1	incorrect	VerifiedBad	ok	1.67	0
PRV052+1	incorrect	VerifiedBad	ok	1.31	0
PRV053+1	incorrect	VerifiedBad	ok	1.51	0
PRV054+1	incorrect	VerifiedBad	ok	1.16	0
PRV055+1	correct	VerifiedGood	ok	1.22	0
PRV056+1	incorrect	VerifiedBad	ok	1.22	1
PRV057+1	incorrect	Error	ok	1.89	0
PRV058+1	incorrect	VerifiedBad	ok	1.41	1
PRV059+1	correct	VerifiedGood	ok	1.24	1
PRV060+1	correct	VerifiedGood	ok	1.25	1
PRV061+1	correct	VerifiedGood	ok	1.22	3
PRV062+1	correct	VerifiedGood	ok	1.66	2
PRV063+1	correct	VerifiedGood	ok	1.66	1
PRV064+1	correct	VerifiedGood	ok	1.89	4
PRV065+1	incorrect	VerifiedBad	ok	1.34	1
PRV066+1	incorrect	VerifiedBad	ok	1.25	1
PRV067+1	incorrect	VerifiedBad	ok	1.65	0
PRV068+1	incorrect	VerifiedBad	ok	1.43	0
PRV069+1	incorrect	VerifiedBad	ok	1.78	1
PRV070+1	correct	VerifiedGood	ok	1.46	0
PRV071+1	incorrect	VerifiedBad	ok	7.32	1
PRV072+1	incorrect	VerifiedBad	ok	7.26	0
PRV073+1	correct	VerifiedGood	ok	7.81	1
PRV074+1	incorrect	VerifiedBad	ok	1.51	1
PRV075+1	incorrect	VerifiedBad	ok	1.41	1
PRV076+1	correct	VerifiedGood	ok	1.60	2
PRV077+1	incorrect	VerifiedBad	ok	1.38	1
PRV078+1	incorrect	VerifiedBad	ok	1.49	1
PRV079+1	correct	VerifiedGood	ok	1.20	1
PRV080+1	incorrect	VerifiedBad	ok	1.26	1
PRV081+1	correct	VerifiedGood	ok	3.96	1
PRV082+1	correct	VerifiedGood	ok	1.64	1
PRV083+1	incorrect	VerifiedBad	ok	1.72	0
PRV084+1	incorrect	VerifiedBad	ok	1.86	1
PRV085+1	correct	VerifiedGood	ok	1.41	1

Proof	Expected	Status	Result	Time (s)	ATP calls
PRV086+1	incorrect	VerifiedBad	ok	1.35	1
PRV087+1	incorrect	VerifiedBad	ok	1.29	1
PRV088+1	correct	VerifiedGood	ok	1.40	1
PRV089+1	correct	VerifiedGood	ok	1.36	0
PRV090+1	incorrect	VerifiedBad	ok	1.18	0
PRV091+1	incorrect	Error	ok	1.21	0
PRV092+1	incorrect	VerifiedBad	ok	1.39	1
PRV093+1	incorrect	VerifiedBad	ok	1.70	1
PRV094+1	incorrect	VerifiedBad	ok	1.51	1
PRV095+1	incorrect	VerifiedBad	ok	1.72	0
PRV096+1	correct	VerifiedGood	ok	1.64	0
PRV097+1	incorrect	Unknown	abstain	1.57	0
PRV098+1	correct	VerifiedGood	ok	1.70	0
PRV099+1	incorrect	VerifiedBad	ok	10.92	2

Table 33: Results for each proof of the TST set (own test proofs).

Proof	Expected	Status	Result	Time (s)	ATP calls
TST001+1	correct	VerifiedGood	ok	1.19	0
TST002+1	correct	VerifiedGood	ok	1.22	0
TST003+1	correct	VerifiedGood	ok	1.28	0
TST004+1	correct	VerifiedGood	ok	1.24	0
TST005+1	correct	VerifiedGood	ok	1.30	0
TST006+1	correct	VerifiedGood	ok	1.27	0
TST007+1	correct	VerifiedGood	ok	1.37	0
TST008+1	incorrect	VerifiedBad	ok	1.61	0
TST009+1	incorrect	VerifiedBad	ok	1.57	0
TST010+1	incorrect	VerifiedBad	ok	1.24	0
TST011+1	incorrect	VerifiedBad	ok	1.29	0
TST012+1	incorrect	VerifiedBad	ok	1.21	0
TST013+1	incorrect	VerifiedBad	ok	1.18	0
TST014+1	incorrect	VerifiedBad	ok	1.22	0
TST015+1	incorrect	VerifiedBad	ok	1.30	0
TST016+1	incorrect	VerifiedBad	ok	1.41	0
TST017+1	incorrect	VerifiedBad	ok	1.19	0
TST018+1	incorrect	VerifiedBad	ok	1.28	0
TST019+1	incorrect	VerifiedBad	ok	1.15	0
TST020+1	incorrect	Unknown	abstain	1.18	0
TST021+1	incorrect	VerifiedBad	ok	1.24	0
TST022+1	incorrect	VerifiedBad	ok	1.28	0
TST023+1	incorrect	VerifiedBad	ok	1.75	1
TST024+1	incorrect	VerifiedBad	ok	1.65	1
TST025+1	incorrect	VerifiedBad	ok	1.33	0
TST026+1	incorrect	VerifiedBad	ok	1.46	1

Proof	Expected	Status	Result	Time (s)	ATP calls
TST027+1	incorrect	VerifiedBad	ok	1.24	0
TST028+1	incorrect	VerifiedBad	ok	1.28	0
TST029+1	incorrect	VerifiedBad	ok	1.26	0
TST030+1	incorrect	VerifiedBad	ok	1.36	0
TST031+1	incorrect	VerifiedBad	ok	1.40	0
TST032+1	incorrect	VerifiedBad	ok	1.30	0
TST033+1	incorrect	VerifiedBad	ok	1.32	0
TST034+1	incorrect	VerifiedBad	ok	1.28	0
TST035+1	incorrect	VerifiedBad	ok	1.27	0
TST036+1	incorrect	VerifiedBad	ok	1.33	0
TST037+1	incorrect	VerifiedBad	ok	1.49	1
TST038+1	incorrect	VerifiedBad	ok	1.56	0
TST039+1	incorrect	VerifiedBad	ok	1.25	0
TST040+1	incorrect	Error	ok	1.24	0
TST041+1	incorrect	VerifiedBad	ok	1.29	0
TST042+1	incorrect	VerifiedBad	ok	1.21	0
TST043+1	incorrect	VerifiedGood	wrong accept	1.23	1
TST044+1	incorrect	VerifiedBad	ok	1.30	0
TST045+1	incorrect	VerifiedBad	ok	1.33	0
TST046+1	incorrect	VerifiedBad	ok	1.26	0
TST047+1	correct	VerifiedGood	ok	1.23	0
TST048+1	correct	VerifiedGood	ok	1.29	0
TST049+1	correct	VerifiedGood	ok	1.26	0
TST050+1	correct	VerifiedGood	ok	1.19	0
TST051+1	correct	VerifiedGood	ok	1.28	0
TST052+1	correct	VerifiedGood	ok	1.57	1
TST053+1	incorrect	VerifiedBad	ok	1.55	0
TST054+1	incorrect	VerifiedBad	ok	1.34	0
TST055+1	incorrect	VerifiedBad	ok	1.28	0
TST056+1	incorrect	VerifiedBad	ok	1.21	0
TST057+1	incorrect	VerifiedBad	ok	1.22	0
TST058+1	incorrect	VerifiedBad	ok	1.21	0
TST059+1	incorrect	VerifiedBad	ok	1.38	0
TST060+1	incorrect	VerifiedBad	ok	1.23	0
TST061+1	incorrect	VerifiedBad	ok	1.27	0
TST062+1	incorrect	VerifiedBad	ok	1.21	0
TST063+1	incorrect	VerifiedBad	ok	1.25	0
TST064+1	incorrect	VerifiedBad	ok	1.56	0
TST065+1	incorrect	VerifiedBad	ok	1.29	1
TST066+1	incorrect	VerifiedBad	ok	1.07	0
TST067+1	incorrect	VerifiedBad	ok	1.21	0
TST068+1	edge	VerifiedGood	edge case	1.39	0
TST069+1	incorrect	VerifiedBad	ok	1.22	0
TST070+1	incorrect	VerifiedBad	ok	1.28	0
TST071+1	edge	VerifiedGood	edge case	1.26	0
TST072+1	incorrect	VerifiedBad	ok	1.25	0

Proof	Expected	Status	Result	Time (s)	ATP calls
TST073+1	edge	VerifiedGood	edge case	1.23	0
TST074+1	edge	VerifiedBad	edge case	1.31	0
TST075+1	incorrect	VerifiedBad	ok	1.36	1
TST076+1	edge	VerifiedBad	edge case	1.32	1

Table 34: Results for each proof of the Extra set (generated rule-level test proofs).

Proof	Expected	Status	Result	Time (s)	ATP calls
EXT001+1	correct	VerifiedGood	ok	1.28	0
EXT002+1	incorrect	VerifiedBad	ok	1.31	0
EXT003+1	incorrect	VerifiedBad	ok	1.20	0
EXT004+1	incorrect	VerifiedBad	ok	1.20	0
EXT005+1	incorrect	VerifiedBad	ok	1.33	0
EXT006+1	incorrect	VerifiedBad	ok	1.20	0
EXT007+1	correct	VerifiedGood	ok	1.24	0
EXT008+1	incorrect	VerifiedBad	ok	1.50	0
EXT009+1	edge	VerifiedBad	edge case	1.47	0
EXT010+1	correct	VerifiedGood	ok	1.27	0
EXT011+1	incorrect	VerifiedBad	ok	1.30	0
EXT012+1	incorrect	VerifiedBad	ok	1.30	0
EXT013+1	incorrect	VerifiedBad	ok	1.38	0
EXT014+1	incorrect	VerifiedBad	ok	1.42	0
EXT015+1	incorrect	VerifiedBad	ok	1.35	0
EXT016+1	incorrect	VerifiedBad	ok	1.38	0
EXT017+1	correct	VerifiedGood	ok	1.21	0
EXT018+1	incorrect	VerifiedBad	ok	1.24	0
EXT019+1	correct	VerifiedGood	ok	1.32	0
EXT020+1	incorrect	VerifiedBad	ok	1.22	0
EXT021+1	correct	VerifiedGood	ok	1.30	0
EXT022+1	incorrect	VerifiedBad	ok	1.30	0
EXT023+1	incorrect	VerifiedBad	ok	1.57	0
EXT024+1	incorrect	VerifiedBad	ok	1.59	0
EXT025+1	incorrect	VerifiedBad	ok	1.27	0
EXT026+1	correct	VerifiedGood	ok	1.23	0
EXT027+1	incorrect	VerifiedBad	ok	1.42	0
EXT028+1	incorrect	VerifiedBad	ok	1.24	0
EXT029+1	incorrect	VerifiedBad	ok	1.22	0
EXT030+1	incorrect	VerifiedBad	ok	1.25	0
EXT031+1	correct	VerifiedGood	ok	1.30	0
EXT032+1	incorrect	VerifiedBad	ok	1.20	0
EXT033+1	incorrect	VerifiedBad	ok	1.27	0
EXT034+1	correct	VerifiedGood	ok	1.32	0
EXT035+1	incorrect	VerifiedBad	ok	1.50	1

Proof	Expected	Status	Result	Time (s)	ATP calls
EXT036+1	incorrect	VerifiedBad	ok	1.34	1
EXT037+1	incorrect	VerifiedBad	ok	1.43	1
EXT038+1	edge	VerifiedGood	edge case	1.56	1
EXT039+1	correct	VerifiedGood	ok	1.24	0
EXT040+1	incorrect	VerifiedBad	ok	1.25	1
EXT041+1	incorrect	VerifiedBad	ok	1.29	1
EXT042+1	correct	VerifiedGood	ok	1.26	0
EXT043+1	incorrect	VerifiedBad	ok	1.32	1
EXT044+1	correct	VerifiedGood	ok	1.26	0
EXT045+1	incorrect	VerifiedBad	ok	1.36	1
EXT046+1	incorrect	VerifiedBad	ok	1.41	1
EXT047+1	incorrect	VerifiedBad	ok	1.30	1
EXT048+1	incorrect	VerifiedBad	ok	1.20	0
EXT049+1	correct	VerifiedGood	ok	1.38	0
EXT050+1	incorrect	VerifiedBad	ok	1.70	0
EXT051+1	incorrect	VerifiedBad	ok	1.32	0
EXT052+1	incorrect	VerifiedBad	ok	1.58	0
EXT053+1	correct	VerifiedGood	ok	1.64	0
EXT054+1	incorrect	VerifiedBad	ok	1.66	0
EXT055+1	correct	VerifiedGood	ok	1.38	0
EXT056+1	incorrect	VerifiedBad	ok	1.50	0
EXT057+1	correct	VerifiedGood	ok	1.33	0
EXT058+1	correct	VerifiedGood	ok	1.39	0
EXT059+1	incorrect	VerifiedBad	ok	1.56	0
EXT060+1	incorrect	VerifiedBad	ok	1.66	0
EXT061+1	correct	VerifiedGood	ok	1.38	0
EXT062+1	incorrect	VerifiedBad	ok	1.32	0
EXT063+1	incorrect	VerifiedBad	ok	1.35	0
EXT064+1	incorrect	VerifiedBad	ok	1.25	0
EXT065+1	correct	VerifiedGood	ok	1.24	0
EXT066+1	incorrect	VerifiedBad	ok	1.65	0
EXT067+1	correct	VerifiedGood	ok	1.56	0
EXT068+1	incorrect	VerifiedBad	ok	1.59	0
EXT069+1	incorrect	VerifiedBad	ok	1.61	0
EXT070+1	edge	VerifiedBad	edge case	1.30	0
EXT071+1	correct	VerifiedGood	ok	1.27	0
EXT072+1	incorrect	VerifiedBad	ok	1.31	0
EXT073+1	incorrect	VerifiedBad	ok	1.24	0
EXT074+1	correct	VerifiedGood	ok	1.25	0
EXT075+1	incorrect	VerifiedBad	ok	1.29	0
EXT076+1	incorrect	VerifiedBad	ok	1.30	0
EXT077+1	correct	VerifiedGood	ok	1.31	0
EXT078+1	incorrect	VerifiedBad	ok	1.34	0
EXT079+1	incorrect	VerifiedBad	ok	1.22	0
EXT080+1	correct	VerifiedGood	ok	1.25	0
EXT081+1	incorrect	VerifiedBad	ok	1.43	0

Proof	Expected	Status	Result	Time (s)	ATP calls
EXT082+1	incorrect	VerifiedBad	ok	1.43	0
EXT083+1	correct	VerifiedGood	ok	1.20	0
EXT084+1	incorrect	VerifiedBad	ok	1.28	0
EXT085+1	incorrect	VerifiedBad	ok	1.18	0
EXT086+1	correct	VerifiedGood	ok	1.19	0
EXT087+1	incorrect	VerifiedBad	ok	1.23	0
EXT088+1	incorrect	VerifiedBad	ok	1.42	0
EXT089+1	correct	VerifiedGood	ok	1.34	0
EXT090+1	incorrect	VerifiedBad	ok	1.48	1
EXT091+1	correct	VerifiedGood	ok	1.45	0
EXT092+1	incorrect	VerifiedBad	ok	1.36	0
EXT093+1	correct	VerifiedGood	ok	1.34	0
EXT094+1	incorrect	VerifiedBad	ok	1.25	0
EXT095+1	incorrect	VerifiedBad	ok	1.53	0
EXT096+1	correct	VerifiedGood	ok	2.64	0
EXT097+1	edge	VerifiedBad	edge case	1.45	0
EXT098+1	edge	VerifiedBad	edge case	1.45	0
EXT099+1	correct	VerifiedGood	ok	1.35	0
EXT100+1	incorrect	VerifiedBad	ok	1.35	0
EXT101+1	correct	VerifiedGood	ok	1.35	0
EXT102+1	incorrect	VerifiedBad	ok	1.52	0
EXT103+1	edge	VerifiedBad	edge case	1.28	0
EXT104+1	edge	VerifiedGood	edge case	1.39	1
EXT105+1	edge	VerifiedBad	edge case	1.25	0
EXT106+1	correct	VerifiedGood	ok	1.66	0
EXT107+1	correct	VerifiedGood	ok	1.37	0
EXT108+1	correct	VerifiedGood	ok	1.63	0
EXT109+1	incorrect	VerifiedBad	ok	1.78	1

List of Figures

1 The ProoVer 2026 proof `PRV000+1.s` (comments and SZS delimiters omitted). Axiom and conjecture are imported from the problem file, `neg` negates the conjecture, and one `consequence` step derives `$false` from `neg` and `a1`. 11
2 Expected score of accepting, rejecting, and abstaining as a function of the probability p that the proof is valid, under the ProoVer 2026 scoring scheme. Accepting beats abstaining only for $p > 10/11$, rejecting only for $p < 2/3$. 24
3 Architecture view of the verifier modules and their main dependencies. Parsing, checking, and verdict aggregation run in a worker process with a wall-clock limit. . 28
4 Dependency graph of the valid benchmark proof `PRV089`. Each node is an annotated formula with its name, role, formula, and justification and each edge leads from a parent to the step derived from it. The color shows which check of the verifier validates the node. Generated from the proof file with `scripts/gen_dag_figure.py`. 30
5 Validation flow for a `skolemize` step. 35
6 Decision flow for aggregating step-level outcomes into the final proof verdict. . . . 37
7 Call sequence of `check_proof_file` including parser, global checks, rule dispatch, ATP fallback, and verdict aggregation. 42
8 Structure of the alpha-equivalence check (pseudocode, simplified). 43
9 Result mapping of the ATP fallback. 46
10 Timeout handling of the driver (excerpt from `main.py`). 46
11 Template matching with capture check (excerpt of `formula_utils.py`, abbreviated). 49
12 Unification with occurs check (excerpt of `clausal_checker.py`). 50
13 Structure of the resolution check (excerpt of `clausal_checker.py`, abbreviated). . 51
14 Inference rules of the ProoVer set by the number of proofs that use them. Blue rules have an internal checker, red rules are decided by the external prover. The labels give the number of steps. 60
15 Wall-clock time per proof of the ProoVer set against the number of steps. Proofs in which the external prover was called are drawn in red. The dashed line is the time limit. 62
16 Outcome of the mutation test per operator: mutants rejected by the verifier, mutants accepted that still follow from their premises, and mutants accepted that do not (none). 64

List of Tables

1 Logical symbols of the FOF language and their TPTP syntax (Sutcliffe 2026c). . . 9
2 Taxonomy of checks for TSTP refutations, after Taprogge et al. (2026b). The abbreviations are those of the paper. 14
3 Systems entered in ProoVer 2026, as described in their system descriptions. “Ext.” lists external tools the system relies on at run time. 18
4 Checks of GDV, Nörgler, and this verifier. “n.s.”: not stated in the sources. “added later”: missing in the studied version of GDV, added afterwards. 20
5 Comparison of the closest related systems with this thesis, according to their system descriptions (Pro 2026)(n/a: not stated in the description.) 22

6 Functional requirements. 24
7 Non-functional requirements. 25
8 Traceability from requirements to modules and tests. 25
9 Rule families and the principle of their checks. 31
10 Inference rules occurring in `PRV010+1.s` and their intended reading. The readings are derived from the benchmark proofs, since the competition does not specify the rules in full. 32
11 Verdicts, printed SZS status, and score under the ProoVer 2026 scheme. 38
12 Main design decisions and the alternatives considered. 39
13 Module structure of the implementation (approximate line counts). 40
14 Rule families, their internal checks, and the behavior on a failed check. 45
15 Groups of the TST set. 55
16 Verdicts on the ProoVer set (number of proofs) and resulting score. 56
17 Results on the TST set by group (agreeing / total without edge cases). 57
18 Results on the Extra set by rule. “By the rule check” counts the invalid proofs for which the checker of the rule itself, and not only the final step, reports the error. . . 58
19 Rules in the ProoVer set and how their steps were decided. 59
20 Size and statement coverage of the modules (measured with `pytest-cov`). 61
21 The six slowest proofs of the ProoVer set (median of the runs). Times in the columns are wall-clock times of the phases, “MB” is the peak resident memory. 62
22 Mutation operators and kill rates. “Survivors still valid” are mutants that the verifier accepts and that Vampire confirms to follow from the premises. 63
23 Outcome for malformed inputs. `error` means that the parser raised an error and the proof was not verified. 64
24 ProoVer set with and without the external prover. The three-valued score counts steps for which no verdict could be reached as uncertain (zero points). 65
25 Single steps, internal verdict against Vampire. “Prover *no*” includes steps for which Vampire did not finish within 10 s (status `Timeout`). 66
26 Causes of suboptimal verdicts on the ProoVer set. 67
27 Verifier on unmodified Vampire proofs (142 proofs). 71
28 Internal Skolem check against an entailment oracle ($C \models P$ with Vampire). Numbers are mutants rejected / accepted. 72
29 Command-line options of the verifier. 77
30 Statuses printed by the verifier. 77
31 Parts of the evaluation driver `scripts/eval_all.py`. 78
32 Results for each proof of the ProoVer set (official proofs and samples). 79
33 Results for each proof of the TST set (own test proofs). 81
34 Results for each proof of the Extra set (generated rule-level test proofs). 83

References

2026. Proceedings of ProoVer 2026. https://proover-competition.github.io/assets/ProoVer2026_proceedings.pdf. Accessed: 2026-10-19.

ANDREOTTI, BRUNO; HANNA LACHNITT; und HANIEL BARBOSA. 2023. Carcara: An efficient proof checker and elaborator for SMT proofs in the Alethe format. *Tacas 2023*, *LNCS*, Ausg. 13993, 367–386. Springer.

BARENDREGT, HENK, und FREEK WIEDIJK. 2005. The challenge of computer mathematics. *Philosophical Transactions of the Royal Society A* 363.2351–2375.

BODINGBAUER, JONAS; MÁRTON HAJDU; LAURA KOVÁCS; AXEL POLACZEK; und MICHAEL RAWSON. 2026. Lean on Vampire proofs (short paper).

BÖHME, SASCHA, und TOBIAS NIPKOW. 2010. Sledgehammer: Judgement day. *Automated reasoning – IJCAR 2010*, *LNCS*, Ausg. 6173, 107–121. Springer.

BUTT, MIRIAM, und WILHELM GEUDER (Hg.) 1998. *The projection of arguments: Lexical and compositional factors*. Stanford, CA: CSLI Publications.

CAILLER, JULIE, und SIMON GUILLOUD. 2026. ProoVer: Website of the ProoVer competition. https://proover-competition.github.io/. Annual competition for proof checkers in the TPTP ecosystem. Accessed: 2026-10-09.

CHIHANI, ZAKARIA; DALE MILLER; und FABIEN RENAUD. 2013. Foundational proof certificates in first-order logic. *Automated deduction – cade-24*, hrsg. von Maria Paola Bonacina, *Lecture Notes in Computer Science*, Ausg. 7898, 162–177. Springer.

CROFT, WILLIAM. 1998. Event structure in argument linking. In Butt & Geuder, 21–63.

DELAHAYE, DAVID; DAMIEN DOLIGEZ; FRÉDÉRIC GILBERT; PIERRE HALMAGRAND; und OLIVIER HERMANT. 2013. Zenon modulo: When achilles outruns the tortoise using deduction modulo. *Logic for programming, artificial intelligence, and reasoning – LPAR-19*, *LNCS*, Ausg. 8312, 274–290. Springer.

DONOHUE, MARK. 2009. Geography is more robust than linguistics. Science e-letter, 13 August 2009. URL http://www.sciencemag.org/cgi/eletters/324/5926/464-c.

DORIAN, NANCY C. (Hg.) 1989. *Investigating obsolescence*. Cambridge: Cambridge University Press.

EBNER, GABRIEL; STEFAN HETZL; GISELLE REIS; MARTIN RIENER; SIMON WOLFSTEINER; und SEBASTIAN ZIVOTA. 2016. System description: GAPT 2.0. *Automated reasoning – IJCAR 2016*, *LNCS*, Ausg. 9706, 293–301. Springer.

FITTING, MELVIN. 1996. *First-order logic*, 109–136. New York, NY: Springer New York. URL https://doi.org/10.1007/978-1-4612-2360-3_5.

GORDON, MICHAEL J.; ROBIN MILNER; und CHRISTOPHER P. WADSWORTH. 1979. *Edinburgh LCF: A mechanised logic of computation*, *Lecture Notes in Computer Science*, Ausg. 78. Springer.

GROPEN, JESS; STEVEN PINKER; MICHELLE HOLLANDER; RICHARD GOLDBERG; und RONALD WILSON. 1989. The learnability and acquisition of the dative alternation in English. *Language* 65.203–57.

HALE, KENNETH, und JOSIE WHITE EAGLE. 1980. A preliminary metrical account of Winnebago accent. *International Journal of American Linguistics* 46.117–32.

HARRISON, JOHN. 2009. HOL Light: An overview. *Theorem proving in higher order logics (tphols 2009)*, *Lecture Notes in Computer Science*, Ausg. 5674, 60–66. Springer.

HASPELMATH, MARTIN. 1993. *A grammar of lezgian*. Walter de Gruyter.

HYMES, DELL H. 1974a. *Foundations in sociolinguistics: An ethnographic approach*. Philadelphia: University of Pennsylvania Press.

HYMES, DELL H. (Hg.) 1974b. *Studies in the history of linguistics: Traditions and paradigms*. Bloomington: Indiana University Press.

HYMES, DELL H. 1980. *Language in education: Ethnolinguistic essays*. Washington, DC: Center for Applied Linguistics.

KOVÁCS, LAURA, und ANDREI VORONKOV. 2013. First-order theorem proving and Vampire. *Cav 2013*, *LNCS*, Ausg. 8044, 1–35. Springer.

MINER, KENNETH. 1990. Winnebago accent: The rest of the data. Lawrence: University of Kansas, MS.

MORGAN, TJH; NT UOMINI; LE RENDELL; L CHOUINARD-THULY; SE STREET; HM LEWIS; CP CROSS; C EVANS; R KEARNEY; I DE LA TORRE; ET AL. 2015. Experimental evidence for the co-evolution of hominin tool-making teaching and language. *Nature communications* 6.

NONNENGART, ANDREAS, und CHRISTOPH WEIDENBACH. 2001. Computing small clause normal forms. *Handbook of automated reasoning*, hrsg. von John Alan Robinson und Andrei Voronkov, Ausg. 1, 335–367. Elsevier and MIT Press.

PERLMUTTER, DAVID M. 1978. Impersonal passives and the unaccusative hypothesis. *Berkeley Linguistics Society* 4.157–89.

POLLACK, ROBERT. 1998. How to believe a machine-checked proof. *Twenty five years of constructive type theory*, hrsg. von Giovanni Sambin und Jan M. Smith. Oxford University Press.

POSER, WILLIAM. 1984. *The phonetics and phonology of tone and intonation in Japanese*. Cambridge, MA: MIT Dissertation.

PRINCE, ELLEN. 1991. Relative clauses, resumptive pronouns, and kind-sentences. Paper presented at the annual meeting of the Linguistic Society of America, Chicago.

RIAZANOV, ALEXANDRE, und ANDREI VORONKOV. 2002. The design and implementation of VAMPIRE. Ausg. 15, 91–110.

RICE, KEREN. 1989. *A grammar of Slave*. Berlin: Mouton de Gruyter.

ROBINSON, JOHN ALAN, und ANDREI VORONKOV (Hg.) 2001. *Handbook of automated reasoning*. Elsevier and MIT Press.

SALTZMAN, ELLIOT; HOSUNG NAM; JELENA KRIVOKAPIC; und LOUIS GOLDSTEIN. 2008. A task-dynamic toolkit for modeling the effects of prosodic structure on articulation. *Proceedings of the 4th International Conference on Speech Prosody (Speech Prosody 2008)*, Campinas, 175–84. URL `http://aune.lpl.univ-aix.fr/~sprosig/sp2008/papers/3inv.pdf`.

VAN DER SANDT, ROB A. 1992. Presupposition projection as anaphora resolution. *Journal of Semantics* 9.333–77.

SCHULZ, STEPHAN; SIMON CRUANES; und PETAR VUKMIROVIĆ. 2019. Faster, higher, stronger: E 2.3. *Cade-27*, *LNCS*, Ausg. 11716, 495–507. Springer.

SCHURR, HANS-JÖRG; MATHIAS FLEURY; HANIEL BARBOSA; und PASCAL FONTAINE. 2021. Alethe: Towards a generic SMT proof format (extended abstract). *Proceedings of the 7th workshop on proof exchange for theorem proving (PxTP 2021)*, *EPTCS*, Ausg. 336, 49–54.

SINGLER, JOHN VICTOR. 1992. Review of Melanesian English and the Oceanic substrate, by Roger M. Keesing. *Language* 68.176–82.

SPROUSE, JON; MATT WAGERS; und COLIN PHILLIPS. 2011. A test of the relation between working memory capacity and syntactic island effects. *Language*, to appear.

STOCKWELL, ROBERT P. 1993. Obituary of Dwight L. Bolinger. *Language* 69.99–112.

SUNDELL, TIMOTHY R. 2009. Metalinguistic disagreement. Ann Arbor: University of Michigan, MS. URL `http://faculty.wcas.northwestern.edu/~trs341/papers.html`.

SUTCLIFFE, G. 2006a. Proceedings of the 3rd IJCAR ATP System Competition. Seattle, USA.

SUTCLIFFE, G. 2006b. Report on the 19th International FLAIRS Conference. *AI Magazine* 27.111–112.

SUTCLIFFE, G. 2006c. Semantic Derivation Verification: Techniques and Implementation. *International Journal on Artificial Intelligence Tools* 15.1053–1070.

SUTCLIFFE, G. 2006d. The CADE-20 Automated Theorem Proving Competition. *AI Communications* 19.173–181.

SUTCLIFFE, G. 2007. TPTP, TSTP, CASC, etc. *Proceedings of the 2nd International Computer Science Symposium in Russia*, hrsg. von V. Diekert, M. Volkov, und A. Voronkov, Nr. 4649 in Lecture Notes in Computer Science, 7–23. Springer-Verlag.

SUTCLIFFE, G. 2008a. The SZS Ontologies for Automated Reasoning Software. *Proceedings of the LPAR Workshops: Knowledge Exchange: Automated Provers and Proof Assistants, and the 7th International Workshop on the Implementation of Logics*, hrsg. von G. Sutcliffe, P. Rudnicki, R. Schmidt, B. Konev, und S. Schulz, Nr. 418 in CEUR Workshop Proceedings, 38–49.

SUTCLIFFE, G. 2010. The TPTP World - Infrastructure for Automated Reasoning. *Proceedings of the 16th International Conference on Logic for Programming Artificial Intelligence and Reasoning*, hrsg. von E. Clarke und A. Voronkov, Nr. 6355 in Lecture Notes in Artificial Intelligence, 1–12. Springer-Verlag.

SUTCLIFFE, G. 2016. The CADE ATP System Competition - CASC. *AI Magazine* 37.99–101.

SUTCLIFFE, G. 2024. Stepping Stones in the TPTP World. *Proceedings of the 12th International Joint Conference on Automated Reasoning*, hrsg. von C. Benzmüller, M. Heule, und R. Schmidt, Nr. 14739 in Lecture Notes in Artificial Intelligence, 30–50.

SUTCLIFFE, GEOFF. 2000. System description: SystemOnTPTP. *Cade-17*, *LNCS*, Ausg. 1831, 406–410. Springer.

SUTCLIFFE, GEOFF. 2006e. Semantic derivation verification: Techniques and implementation. *International Journal on Artificial Intelligence Tools* 15.1053–1070.

SUTCLIFFE, GEOFF. 2008b. The SZS ontologies for automated reasoning software. *Proceedings of the combined KEAPPA – IWIL workshops*, hrsg. von Piotr Rudnicki, Geoff Sutcliffe, Boris Konev, Renate Schmidt, und Stephan Schulz, *CEUR Workshop Proceedings*, Ausg. 418, 38–49.

SUTCLIFFE, GEOFF. 2017. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning* 59.483–502.

SUTCLIFFE, GEOFF. 2025. The 30th IJCAR automated theorem proving system competition (CASC-30). `https://tptp.org/CASC/30/`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026a. Semantic derivation verification (GDV). `https://tptp.org/Seminars/GDV/Contents.html`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026b. The SZS ontologies. `https://szs.tptp.org`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026c. The TPTP language: Syntax in BNF. `https://tptp.org/TPTP/SyntaxBNF.html`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026d. TPTP quick guide: The TPTP format for derivations. `https://tptp.org/UserDocs/QuickGuide/Derivations.html`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026e. The TSTP solution library. `https://tptp.org/TSTP/`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF; FRÉDÉRIC BLANQUI; und GUILLAUME BUREL. 2025. Proof verification with GDV and LambdaPi – it's a matter of trust. *Proceedings of the international FLAIRS conference*, Ausg. 38. URL `https://journals.flvc.org/FLAIRS/article/view/138642`.

SUTCLIFFE, GEOFF; STEPHAN SCHULZ; KOEN CLAESSEN; und ALLEN VAN GELDER. 2006. Using the TPTP language for writing derivations and finite interpretations. *Ijcar 2006*, *LNCS*, Ausg. 4130, 67–81. Springer.

TAPROGGE, MELANIE; FRÉDÉRIC BLANQUI; und ALEXANDER STEEN. 2026a. Formal verification of proofs from automated theorem provers for higher-order logic. URL `https://arxiv.org/abs/2609.24594`.

TAPROGGE, MELANIE; HAPPY SARIYANTO; und ALEXANDER STEEN. 2026b. A light-weight proof checker for tstp refutations. *Practical aspects of automated reasoning*, *CEUR Workshop Proceedings*, Ausg. 4241. URL `https://ceur-ws.org/Vol-4241/`.

TIERSMA, PETER M. 1993. Linguistic issues in the law. *Language* 69.113–37.

WETZLER, NATHAN; MARIJN J. H. HEULE; und WARREN A. HUNT JR. 2014. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. *Theory and applications of satisfiability testing – SAT 2014*, *LNCS*, Ausg. 8561, 422–429. Springer.

WILSON, DEIRDRE. 1975. *Presuppositions and non-truth-conditional semantics*. London: Academic Press.

YIP, MOIRA. 1991. Coronals, consonant clusters, and the coda condition. *The special status of coronals: Internal and external evidence*, hrsg. von Carole Paradis und Jean-François Prunet, 61–78. San Diego, CA: Academic Press.